

计算机图形学 期末复习文档

使用说明

本文档按期末考核的三大题方向组织：**第一大题 渲染（变换与着色）**、**第二大题 几何（贝塞尔与网格）**、**第三大题 动画（弹簧系统）**。10 道小题分布在三大题中。

答题三段式（按这个顺序写，方便改卷给分）：

1. **写思路**——先说明用什么方法、分几步、为什么这样做（拿步骤分）；
2. **写公式**——把要用的矩阵 / 向量公式的一般形式写出来（拿公式分）；
3. **套数据计算**——最后代入题目给的具体数值算出结果（拿结果分）。

全文统一约定（务必记牢，和试卷一致）：

- 采用**齐次坐标 + 列向量右乘**。点 $P = (x, y, z, 1)^T$ ，方向 $d = (x, y, z, 0)^T$ （第 4 维为 1 是点、为 0 是向量）。
- 先做变换 A 再做变换 B ，则总矩阵为 BA （**右边的矩阵先作用**）。
- 相机默认看向 $-z$ 方向；旋转角从轴正方向看**逆时针为正**。

0. 预备知识（零基础入门）

这一部分假设你完全没接触过图形学。后面三大题会反复用到的所有基本概念，都先在这里用大白话讲清楚。看不懂后面公式时，回到这里对照。

0.1 计算机图形学是什么

一句话：研究怎么用计算机生成图像（画面）的方法。

你在屏幕上看到的每一帧游戏画面、每一部动画电影、每一张 CAD 工程图，本质都是一个由很多小彩色方块（像素，pixel）拼成的图。图形学要解决的核心问题就是：

给定一个三维场景（物体的形状、材质、灯光、相机），如何算出屏幕上每个像素应该是什么颜色？

它的主要应用领域：信息可视化、工业 / 建筑设计（CAD）、模拟与动画（电影 / 游戏）、人机交互界面。

0.2 成像四要素

不管哪种成像方式，要拍出 / 算出一张图，都离不开四样东西。记住这四个，整门课的结构就清楚了：

要素	含义	对应本课内容
几何 (Geometry)	物体长什么形状、在哪里	变换、贝塞尔、网格
材质 (Material)	物体表面是什么质感（光滑 / 粗糙 / 透明）	着色（漫反射 / 镜面 / 折射）
观察者 (Viewer)	相机 / 眼睛在哪、朝哪看	相机变换、投影
光源 (Light)	有哪些灯、什么颜色、多亮	光照模型、光线跟踪

直观理解：把它想象成拍照——你要有被拍的东西（几何+材质）、相机（观察者）、打光（光源），才能得到一张照片。

0.3 坐标系：x, y, z 与右手系

坐标系就是用数字描述"位置"的方法。

- **二维 (2D)**: 平面上一个点用两个数 (x, y) 表示。 x 是水平方向, y 是竖直方向。
- **三维 (3D)**: 空间里一个点用三个数 (x, y, z) 表示。多出来的 z 表示"深度" (远近)。

右手系: 图形学 (OpenGL) 默认用右手坐标系。判断方法——伸出右手, 四指从 x 轴正方向**握向** y 轴正方向, 大拇指指的方向就是 z 轴正方向。

本课一个重要默认: 相机 (眼睛) 放在原点时, **朝 $-z$ 方向看**, 也就是 z 越小的物体离相机越远 (在你前方), z 为正的在你背后。这个约定会在相机变换、投影里反复用到。

旋转角的正方向: 从某根轴的**正方向往原点看**, **逆时针**转为正角度。

0.4 点与向量的区别

这两个东西坐标看起来一样 (都是几个数), 但意义完全不同, 是新手最容易混的地方:

- **点 (Point)**: 表示一个**位置**, 比如"物体在哪"。例: $(1, 2, 3)$ 表示空间中某个具体地点。
- **向量 (Vector)**: 表示一个**方向 + 长度 (位移)**, 没有"在哪"的概念, 只有"往哪走、走多远"。例: $(0, 0, 5)$ 表示"沿 z 正方向移动 5 个单位"。

它们之间的运算规则:

- 点 - 点 = 向量 (从一个位置指向另一个位置)。
- 点 + 向量 = 点 (从一个位置出发走一段, 到达新位置) ——**平移**就是这个。
- 向量 + 向量 = 向量 (方向叠加)。
- 点 + 点 = **没有意义** (两个"地点"相加是什么? 没意义)。

后面用**齐次坐标** (下一节) 就是为了在数学上区分这两者。

0.5 齐次坐标: 第四维 w 是干什么的

这是整门课最关键的"约定", 务必理解。

问题来源: 平移 (把物体挪个位置) 这个操作, 没法用普通的 3×3 矩阵乘法表示 (矩阵乘以零向量还是零向量, 原点没法被挪走)。但旋转、缩放可以用矩阵。我们希望**所有变换都能统一写成"矩阵 $\times$ 坐标"**, 方便连续相乘。

解决办法: 给三维坐标 (x, y, z) **多加一维 w** , 变成四维的 (x, y, z, w) , 这就是**齐次坐标 (Homogeneous Coordinates)**。用 4×4 矩阵, 平移就也能用矩阵乘法表示了。

第四维 w 的含义 (最重要):

w 的值	表示	写法
$w = 1$	这是一个点（位置）	$(x, y, z, 1)$
$w = 0$	这是一个向量（方向）	$(x, y, z, 0)$

这样数学就自动满足 0.4 节的规则：两个点相减 $(x_1, y_1, z_1, 1) - (x_2, y_2, z_2, 1) = (\dots, 0)$ ，第四维变成 0，结果自动是个向量。而平移矩阵对 $w = 0$ 的向量不起作用（向量没有位置，挪它没意义）——这些都"免费"地由 w 帮你处理好了。

透视除法：在透视投影里，变换后 w 可能不等于 1（会变成和深度有关的值）。这时要把所有分量除以 w 还原成普通坐标 $(x/w, y/w, z/w)$ ，这一步叫**透视除法**，正是它实现了"近大远小"的效果（见 0.12 和第 5 节）。

记忆口诀：点的 w 是 1，向量的 w 是 0；透视完了除以 w 。

0.6 矩阵与变换：为什么用矩阵表示几何变换

矩阵可以理解为一张数字表格。一个 4×4 矩阵乘以一个坐标列向量，会得到一个新坐标——这就完成了一次**几何变换**（移动 / 旋转 / 放缩物体）。

为什么非要用矩阵？三个好处：

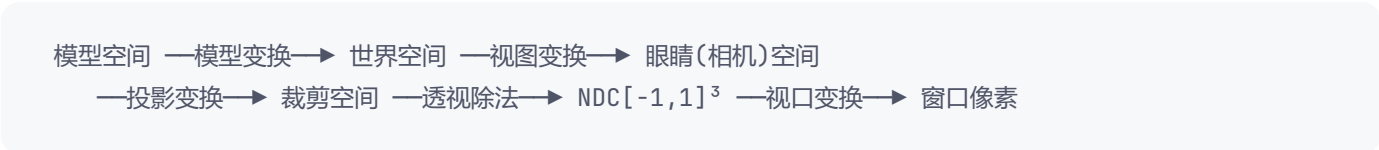
- 1. **统一**：平移、旋转、缩放、投影全都写成"矩阵 $\times$ 坐标"，形式一致。
- 2. **可复合**：想"先旋转再平移"，只要把两个矩阵**相乘**成一个总矩阵 M ，对成千上万个顶点只需各乘一次 M ，省得分步算。
- 3. **可逆**：求逆矩阵就能"撤销"一个变换。

两个关键约定（本课固定用法）：

- **列向量右乘**：坐标写成竖着的列向量，矩阵写在它**左边**： $P' = MP$ 。
- **变换顺序**：先做 A 再做 B ，合起来写作 BA （**靠近坐标的、最右边的先作用**）。因为 $BA P = B(A P)$ ：先用 A 作用 P ，再用 B 作用结果。
- **矩阵乘法不能交换**： $AB \neq BA$ 。所以"先转后移"和"先移后转"结果不同，顺序千万别写反。

0.7 顶点要经过的几个坐标空间

一个顶点从"建模时的坐标"到"屏幕上的像素"，要换好几次"坐标系"。这是第一大题的主线，先混个眼熟（细节见第 1 节）：



逐个用大白话解释：

空间	大白话
模型空间 (Model / Object)	物体自己的坐标系，建模时以物体中心为原点
世界空间 (World)	把物体摆进整个场景后的统一坐标系
眼睛 / 相机空间 (Eye / View)	以相机为原点、相机朝 $-z$ 看时的坐标系
裁剪空间 (Clip)	投影后、还没除 w 的坐标，用来裁掉视野外的东西
NDC (规范化设备坐标)	除完 w 后，落在 $[-1, 1]^3$ 标准立方体里的坐标
窗口空间 (Window / Screen)	最终的像素行列位置

模型变换 + 视图变换合起来叫模型视图变换 (ModelView)，是一个矩阵 $M = V \cdot C$ 。这是考试反复出现的词。

0.8 角度、弧度与三角函数

旋转要用到三角函数，先复习两个常用值：

- $\cos 30^\circ = \frac{\sqrt{3}}{2} \approx 0.866, \sin 30^\circ = \frac{1}{2} = 0.5。$
- $\cos 45^\circ = \sin 45^\circ = \frac{\sqrt{2}}{2} \approx 0.707。$
- $\cos 60^\circ = \frac{1}{2}, \sin 60^\circ = \frac{\sqrt{3}}{2}。$
- $\cos 90^\circ = 0, \sin 90^\circ = 1。$

两个会用到的性质： $\cos(-\theta) = \cos \theta$ (偶函数)， $\sin(-\theta) = -\sin \theta$ (奇函数)。这就是"逆旋转 $R(-\theta)$ 只需把所有 $\sin$ 变号"的原因 (第 2.5 节)。

弧度制： $180^\circ = \pi$ 弧度。编程里三角函数 (如 C/C++ 的 `sin`) 的参数是弧度，记得 弧度 = 角度 $\times \pi/180$ 。

0.9 几何的"零件"：顶点 / 边 / 面 / 网格 / 法向

计算机里的三维物体，绝大多数是用多边形网格拼出来的 (想象一个由许多小三角形 / 四边形拼成的球面)。

- 顶点 (Vertex)**：一个 3D 点，是物体的"角"。
- 边 (Edge)**：连接两个顶点的线段。

- **面 (Face)**: 由几条边围成的多边形 (最常见是三角形)。
- **网格 (Mesh)**: 一堆顶点 + 它们的连接关系 (拓扑), 组成整个物体表面。
- **法向 (Normal)**: 垂直于表面、指向"外侧"的**单位向量**, 告诉光照计算"这个面朝哪个方向"。**顶点的法向**通常取它周围几个面法向的平均。光照、着色都离不开法向。
- **像素 (Pixel)**: 屏幕上最小的彩色方块, 最终图像就是像素的网格。
- **帧 (Frame)**: 一张完整画面。动画就是每秒快速播放很多帧 (电影 24 帧 / 秒)。

0.10 颜色: RGB 与加色 / 减色

人眼有三种感色细胞, 所以用**三原色**就能表示绝大多数颜色。

- **RGB (加色系统)**: 红、绿、蓝三种光**叠加**。全亮 = 白, 全灭 = 黑。屏幕、投影仪用这套。本课光照计算就是对 R、G、B 三个通道**分别算**。
- **CMY (减色系统)**: 青、品红、黄三种颜料**吸收**光。叠加越多越暗。打印机用这套。

本课里一个颜色 / 光强常写成三元组, 如光亮度 $L = [L_r, L_g, L_b]$; 所有光照公式都对三个分量各算一遍。

0.11 渲染、光栅化、着色、流水线的关系

这几个词经常一起出现, 先理清:

- **渲染 (Rendering)**: 从三维场景算出最终二维图像的**整个过程**的总称。
- **绘制流水线 (Pipeline)**: 渲染被拆成一串固定的处理阶段 (顶点变换 → 投影 → 裁剪 → 视口 → 光栅化 → 着色 ...), 像工厂流水线一样一道道处理, 这是实时渲染 (游戏) 的主流方式。
- **光栅化 (Rasterization)**: 把一个几何图元 (如三角形) 转换成它覆盖的那一片**像素**的过程。
- **着色 / 明暗处理 (Shading)**: 计算每个像素 / 顶点**颜色**的过程 (用光照模型)。

两大绘制范式 (贯穿全课):

范式	思路	循环写法
光栅化流水线	对每个 物体 , 找它覆盖哪些像素并上色	<code>for each object: render(object)</code>
光线跟踪	对每个 像素 , 发射光线找最近物体并上色	<code>for each pixel: assign_color(pixel)</code>

光栅化快、适合实时; 光线跟踪慢但能自然产生阴影、反射、折射 (第 10 节)。

0.12 投影直观理解: 平行 vs 透视

投影就是把三维场景"拍扁"成二维图像的方式，分两大类：

- **平行（正交）投影**：投影线互相平行。物体**不会近大远小**，远近一样大。能保持长度和角度，可以直接量尺寸——所以工程图、CAD 三视图用它。
- **透视投影**：投影线汇聚到一个点（相机）。物体**近大远小**（diminution），更接近人眼 / 照片的真实观感——动画、游戏用它。透视里平行的线会交于**灭点（vanishing point）**。

技术上，透视的"近大远小"就是靠 0.5 节的**透视除法**实现的：越远的点 w （与深度相关）越大，除完之后坐标越往中间缩，看起来就越小。

0.13 名词速查（中英对照）

中文	英文	一句话
齐次坐标	Homogeneous Coordinates	加第 4 维 w 的坐标，点 $w=1$ 、向量 $w=0$
模型视图变换	ModelView Transform	模型变换 + 视图变换的合成矩阵
视图 / 相机变换	View Transform	把世界搬到以相机为原点的坐标系
投影变换	Projection	把视景物规范化到 $[-1, 1]^3$
视口变换	Viewport Transform	把 $[-1, 1]^3$ 映射到屏幕像素
法向	Normal	垂直于表面的单位向量
漫反射	Diffuse	粗糙面把光均匀散开
镜面反射	Specular	光滑面产生高光
环境光	Ambient	模拟多次反射的均匀背景光
着色	Shading	算颜色 / 明暗
网格	Mesh	顶点 + 连接关系组成的表面
细分	Subdivision	加点把网格变光滑（Catmull-Clark）
贝塞尔曲线	Bézier Curve	由控制点决定的光滑曲线
控制点	Control Point	拉扯曲线形状的"把手"
质点-弹簧系统	Mass-Spring System	用质点和弹簧模拟布料等柔体
关键帧	Keyframe	手工指定的关键画面，中间靠插值

0.14 向量的运算：点乘与叉乘（从例子推导）

向量是图形学的"螺丝刀"，两种乘法贯穿全课。**先别背公式，跟着例子想为什么。**

点乘（内积）：测"两个方向有多一致"

先看一个问题：一束光照在桌面上，光线方向和桌面法向夹角越小（越正对着照），桌面应该越亮；夹角到 90° （光擦着表面）就几乎不亮。我们需要一个数，能反映"两个方向有多一致"。这个数就是**点乘**。

推导直觉：在二维里，把单位向量 a 放在 x 轴上 $a = (1, 0)$ ，另一个单位向量 b 与它夹角 θ ，则 $b = (\cos \theta, \sin \theta)$ 。把对应分量相乘再相加： $1 \cdot \cos \theta + 0 \cdot \sin \theta = \cos \theta$ 。正好等于夹角余弦！把它推广到任意长度、三维，就得到：

$$a \cdot b = a_x b_x + a_y b_y + a_z b_z = \|a\| \|b\| \cos \theta$$

怎么用：

- 求夹角： $\cos \theta = \frac{a \cdot b}{\|a\| \|b\|}$ ；若 a, b 都是单位向量，直接 $\cos \theta = a \cdot b$ 。
- 判方向：点乘 > 0 夹角小于 90° （同侧）； $= 0$ 垂直； < 0 反向（背光）。
- 例：法向 $n = (0, 0, 1)$ 、光向 $l = (0, 0, 1)$ ，则 $l \cdot n = 1$ （正对，最亮）；若 $l = (1, 0, 0)$ ，则 $l \cdot n = 0$ （擦过，不亮）。这正是漫反射公式 $\max(l \cdot n, 0)$ 的来历（第8节）。

叉乘（外积）：造一个"同时垂直于两者"的方向

先看一个问题：已知三角形面上两条边的方向，想求这个面"朝哪"——也就是同时垂直于这两条边的**法向**。需要一种运算，输入两个向量、输出一个垂直于它俩的新向量。这就是**叉乘**。

【最好记的形式：行列式定义式】把 x, y, z 三个坐标轴的单位向量记作 i, j, k ，叉乘就是下面这个 3×3 行列式（第一行放三个单位向量，第二行放 a ，第三行放 b ）：

$$a \times b = \begin{vmatrix} i & j & k \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$

怎么算：按第一行展开（每个单位向量配它的"代数余子式"，注意中间项带负号）：

$$a \times b = i \begin{vmatrix} a_y & a_z \\ b_y & b_z \end{vmatrix} - j \begin{vmatrix} a_x & a_z \\ b_x & b_z \end{vmatrix} + k \begin{vmatrix} a_x & a_y \\ b_x & b_y \end{vmatrix}$$

$(2 \times 2 \text{ 行列式 } \begin{vmatrix} p & q \\ r & s \end{vmatrix} = ps - qr, \text{ 主对角减副对角。})$ 把三个二阶行列式算出来，就是展开后的分量式：

$$a \times b = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

为什么用行列式形式记：分量式那三项符号容易记乱，而行列式"第一行摆 i, j, k 、下面两行摆 a, b 、按第一行展开"是一套固定动作，考场上照搬就行，不会记错下标顺序。

验证一下： x 轴 $\times$ y 轴应得 z 轴。取 $a = (1, 0, 0), b = (0, 1, 0)$ ，代入得 $(0 \cdot 0 - 0 \cdot 1, 0 \cdot 0 - 1 \cdot 0, 1 \cdot 1 - 0 \cdot 0) = (0, 0, 1) \checkmark$ ，正是 z 轴，符合右手系。

性质与用途：

- $\|a \times b\| = \|a\| \|b\| \sin \theta$ ，等于两向量张成的平行四边形面积。
- **不满足交换律：** $a \times b = -(b \times a)$ （交换顺序，法向反向）。所以求法向时边的取法顺序要一致。
- 主要用途：求面 / 顶点法向、构造正交基（`gluLookAt`，第 4 节）。

归一化（单位化）：把向量变成长度为 1、只保留方向： $\hat{a} = a / \|a\|$ 。法向、相机基都必须归一化。

0.15 矩阵与正交基（从例子推导）

矩阵 $\times$ 向量 = 变换后的新向量

先理解“矩阵乘向量”在算什么。把一个 2×2 矩阵的**两列**看成“新的 x 轴落点”和“新的 y 轴落点”，矩阵乘以坐标，就是按这两个新基把坐标重新组合。

例子：矩阵 $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$ 乘以点 $(1, 1)$ ：

$$\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + 3 \cdot 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$$

x 被放大 2 倍、 y 放大 3 倍——这正是“缩放”。**结论：矩阵的每一列，就是变换后各坐标轴的去向。**想让 x 轴转到某方向，就把那个方向写进矩阵第一列。第 2 节所有变换矩阵都是这么“拼”出来的。

矩阵 $\times$ 矩阵 = 把两个变换合成一个： $M = BA$ 表示“先做 A 再做 B ”，因为 $Mp = B(Ap)$ 。这就是复合变换的数学基础（0.6 节）。

正交基与正交矩阵

正交基：三个互相垂直、长度都为 1 的向量（如 x, y, z 轴）。它们能干净地张成空间，不拉伸不扭曲。**构造套路**（考相机变换必用）：取一个已知方向 $\rightarrow$ 叉乘出第二个垂直方向 $\rightarrow$ 再叉乘出第三个 $\rightarrow$ 全部归一化。

正交矩阵：每一列都是单位向量且互相垂直的矩阵，满足

$$R^T R = I \implies R^{-1} = R^T$$

即它的**逆等于转置**。**所有旋转矩阵都是正交矩阵**——这就是第 2.5 节“旋转的逆 = 转置、只需把 $\sin$ 变号”的根本原因。

准备好这些直觉后，就可以进入第一大题了。后面每个矩阵，都先有“为什么长这样”的推导，再给最终形式。

第一大题：渲染（变换 + 着色）

这是分值最重的一题，核心是**把一个点从模型空间一路算到窗口像素**，再算它的**颜色**。务必能"自己实现"整条流水线，不依赖 OpenGL 函数。（用到的向量 / 矩阵工具见预备知识 0.14、0.15。）

1. 顶点变换流水线总览

【先假设你不懂】 一个用建模软件做好的物体，它的顶点坐标是"以物体自己为中心"记的。可它最终要变成屏幕上某个像素的位置。这中间要"翻译"好几次坐标系——就像描述"杯子在哪"，可以说"在桌子左上角"（物体自己的参照）、"在房间东墙边"（世界参照）、"在我正前方 1 米"（相机参照），同一个位置换不同参照说法不同。流水线就是把这些"翻译"一步步做完。

【一个顶点的旅程（先看全景，细节在 2-6 节）】

$$\begin{array}{ccccccc} P & \xrightarrow{\text{模型视图变换 } M} & P_e & \xrightarrow{\text{投影变换 } N} & P_c & \xrightarrow{\text{透视除法}} & P_n & \xrightarrow{\text{视口变换 } D} & P_w \\ \text{模型空间} & & \text{眼睛空间} & & \text{裁剪/规范空间} & & \text{NDC } [-1,1]^3 & & \text{窗口像素} \end{array}$$

逐站说明它在干嘛：

- **模型视图变换** $M = V \cdot C$ ：先用模型变换 C 把物体摆到世界里（第 2、3 节），再用视图变换 V 把相机摆好（第 4 节，等价于反向移动整个世界）。出来的是"以相机为原点"的眼睛空间坐标 P_e 。
- **投影变换** N ：把眼前看得见的那块空间规范化到边长为 2、中心在原点的标准立方体 $[-1, 1]^3$ （第 5 节）。
- **透视除法**：把齐次坐标第 4 维 w 除回去， $(x, y, z, w) \rightarrow (x/w, y/w, z/w)$ 。正交 $w = 1$ 不用除；透视 $w = -z_e$ 必须除（这步实现近大远小）。
- **视口变换** D ：把 $[-1, 1]^3$ 映射到屏幕窗口的实际像素位置（第 6 节）。

考点：给一个点的坐标，要求自己一路算到它在窗口中的行列——就是把 M, N, D 依次乘上去（对应 OpenGL 的 `gluProject`）。这条链路的完整写法是：先算模型 / 视图矩阵，再算投影矩阵，最后算视口矩阵。**先记住这条主线，再逐节学每一步的矩阵怎么来。**

1.1 先把模型视图矩阵 $M = VC$ 讲清楚

【先看一个点到底经历了什么】 一个模型自己的点 P ，一开始只知道它在模型内部的位置，比如"这个顶点在模型中心右边 1 个单位"。这还不是世界里的位置，因为模型可能被平移、旋转、缩放过。先用**模型矩阵** C 处理它，得到世界坐标；再用**视图矩阵** V 把世界坐标换成相机眼里的坐标，得到眼睛坐标 P_e 。所以计算顺序是

$$P \xrightarrow{C} P_{\text{world}} \xrightarrow{V} P_e$$

写成列向量右乘，就是

$$P_e = V(CP) = (VC)P$$

因此我们把 VC 合成一个矩阵，叫**模型视图矩阵**，记作

$$M = VC, \quad P_e = MP$$

这里的 $P_e = MP$ 不是新规则，而是把两步计算压缩成一步。原本要先算 $P_{\text{world}} = CP$ ，再算 $P_e = VP_{\text{world}}$ ；把第一步代入第二步，就得到 $P_e = V(CP)$ 。由于矩阵乘法满足结合律，可以先把 V 和 C 合成 $M = VC$ ，再让 M 去乘点 P 。所以做题时通常先算总矩阵 M ，再做一次矩阵乘点：

$$\begin{pmatrix} x_e \\ y_e \\ z_e \\ 1 \end{pmatrix} = M \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

也就是说， M 的每一行分别和点 $P = (x, y, z, 1)^T$ 做点乘，就得到眼睛坐标里的 x_e, y_e, z_e 。

【为什么不是 CV 】 因为本文档统一使用列向量，矩阵靠右的先作用。 $M = VC$ 的意思不是"先写 V 就先做 V "，而是最右边的 C 先碰到点 P ，所以先做模型变换；然后左边的 V 再作用，所以再做相机变换。把顺序写成 CV 就会变成"先把模型点拿去做相机变换，再摆到世界里"，这和真实流水线不一致。

【怎么手算 VC 】 如果

$$C = \begin{pmatrix} a_{11} & a_{12} & a_{13} & c_x \\ a_{21} & a_{22} & a_{23} & c_y \\ a_{31} & a_{32} & a_{33} & c_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

那么 VC 就按普通矩阵乘法去算。第一行等于 V 的第一行 $(1, 0, 0, -1)$ 分别去乘 C 的四列，所以得到

$$(a_{11}, a_{12}, a_{13}, c_x - 1)$$

第二行、第三行、第四行也同样按行乘列算，完整结果是

$$VC = \begin{pmatrix} a_{11} & a_{12} & a_{13} & c_x - 1 \\ a_{21} & a_{22} & a_{23} & c_y \\ a_{31} & a_{32} & a_{33} & c_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2. 模型变换：平移 / 旋转 / 缩放与逆变换

2.1 平移 Translation

【先假设你不懂】 把一个点挪到那个位置，不是很简单吗—— x 加点、 y 加点就行。为什么还要搞个矩阵？

【从例子出发】 点 $P = (1, 2, 3)$ 想沿方向 $d = (2, 2, 5)$ 平移。直接加：

$$x' = 1 + 2 = 3, \quad y' = 2 + 2 = 4, \quad z' = 3 + 5 = 8$$

得到 $(3, 4, 8)$ 。这一步谁都会。问题是：旋转、缩放都能写成“矩阵 $\times$ 坐标”，唯独平移是“加法”，形式不统一，没法和别的变换连乘成一个矩阵。

【推导原理】 怎么把“加法”也变成“矩阵乘法”？秘诀就是预备 0.5 的**齐次坐标**——给点补上第四维 $w = 1$ ，写成 $(x, y, z, 1)$ 。再设计一个 4×4 矩阵，让它的**第四列**装上平移量 d 。看乘出来是什么：

$$\begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + d_x \\ y + d_y \\ z + d_z \\ 1 \end{pmatrix}$$

第四维那个 1 把第四列的 d “激活”加了进来——加法被装进了矩阵乘法里！这也顺带解释了为什么**向量** ($w = 0$) **平移不动**：第四维是 0，第四列乘进去是 0，方向没有“位置”可挪，完全合理。

【公式（最终形式）】

$$T(d_x, d_y, d_z) = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

平移有 3 个自由度 (d_x, d_y, d_z) ，本质是“点 + 向量 = 点”。

2.2 旋转 Rotation

【先假设你不懂】 把点绕原点转一个角度 θ ，新坐标怎么算？凭直觉根本写不出来。我们从最简单的二维推。

【从例子出发】 在二维平面上，点 P 到原点距离 r ，与 x 轴夹角 ϕ ，那么

$$x = r \cos \phi, \quad y = r \sin \phi$$

现在绕原点**逆时针**再转 θ ，距离 r 不变、角度变成 $\phi + \theta$ ：

$$x' = r \cos(\phi + \theta), \quad y' = r \sin(\phi + \theta)$$

用和角公式展开 $\cos(\phi + \theta) = \cos \phi \cos \theta - \sin \phi \sin \theta$ 、 $\sin(\phi + \theta) = \sin \phi \cos \theta + \cos \phi \sin \theta$ ，再把 $r \cos \phi = x$ ， $r \sin \phi = y$ 代回去：

$$\boxed{x' = x \cos \theta - y \sin \theta, \quad y' = x \sin \theta + y \cos \theta}$$

【推导原理】 把上面两式写成矩阵（系数排好）就是二维旋转矩阵 $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ 。推广到三维：**绕哪根轴转，哪根轴的坐标分量就不变**，另外两维按上面的二维公式转。例如绕 z 轴转， z 不动， x, y 按二维公式变——把它嵌进 4×4 即可。

【公式（最终形式）】（记 $c = \cos \theta$, $s = \sin \theta$ ）

绕 Z 轴（ z 不变，就是上面二维公式的直接搬入）：

$$R_z(\theta) = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕 X 轴（ x 不变，对 y, z 做二维旋转）：

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕 Y 轴（ y 不变；注意因为 $z \rightarrow x$ 的环绕顺序， $\sin$ 符号与另两者相反）：

$$R_y(\theta) = \begin{pmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

记忆技巧： 绕谁转，矩阵里谁那一行那一列保持单位（对角为 1、其余为 0）。 R_y 的负号在左下角，正好和 R_x, R_z 反着。

验证： 绕 z 转 90° （ $c = 0, s = 1$ ）时， x 轴点 $(1, 0, 0)$ 应转到 y 轴。代入 R_z 得 $(0, 1, 0) \checkmark$ 。

2.3 缩放 Scaling

【先假设你不懂】 "把物体放大 2 倍"在坐标上是什么操作？

【从例子出发】 点 $(1, 1, 1)$ 想让 x 放大 2 倍、 y 放大 3 倍、 z 不变，结果显然是 $(2, 3, 1)$ ——就是每个坐标各乘一个系数。回忆预备 0.15：矩阵的每一列是"该坐标轴的去向"，要让 x 轴方向拉长 2 倍，就把第一列写成 $(2, 0, 0)$ ；以此类推：

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 1 \\ 1 \end{pmatrix} \checkmark$$

【推导原理】 缩放就是对角矩阵：对角线上是各轴的缩放系数，原点是不动点。系数取负数就是沿该轴反射（reflection）（例如 $s_x = -1$ 把物体左右镜像）。

【公式（最终形式）】

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.4 刚体变换 vs 仿射变换

【先假设你不懂】 上面三种变换（平移、旋转、缩放）作用在物体上，有的会"改变形状"，有的不会。我们给它们分个类，搞清楚谁"温柔"、谁会"捏变形"。

【从例子出发】 想象桌上一本书：

- 你把它挪个位置（平移）、转个方向（旋转）——书还是那本书，长宽没变、四个角还是直角。这种"只换位置和朝向、不改变形状大小"的变换叫**刚体变换（Rigid Transformation）**。
- 但如果你把书放大成两倍（缩放），或者像推一摞扑克牌那样把矩形推成"斜的平行四边形"（错切 / Shear）——形状就变了。这类变换叫**仿射变换（Affine Transformation）**。

【什么是错切】 错切是新手容易陌生的一个。它让每一层按高度成比例横向平移，把正方形推成平行四边形。二维沿 x 错切的矩阵（ k 是错切量）：

$$\begin{pmatrix} 1 & k & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + ky \\ y \\ 1 \end{pmatrix}$$

y 越大的点， x 被推得越远——所以竖直的边变成了斜边。注意：平行线推完还是平行线、直线还是直线，只是角度变了。

【原理小结】

类型	包含	保持什么	改变什么
刚体变换	平移、旋转	长度、角度、形状、大小	只改位置和朝向
仿射变换	刚体 + 缩放 + 错切	"直线还是直线、平行还是平行"	可改形状、大小、角度

【为什么要分这个类】 "仿射变换保持直线和平行关系"这条性质后面有大用：第二大题的贝塞尔曲线具有仿射不变性——想对曲线做平移 / 旋转 / 缩放，不必逐点变换，只要对几个控制点做同样的仿射变换，再重新生成曲线即可，结果完全一样（省大量计算）。

2.5 逆变换（按几何意义直接写，别去求逆矩阵）

【先假设你不懂】 "逆变换"就是把一个变换撤销、让物体回到原样——像 Word 里的"撤销（Ctrl+Z）"。数学上，变换 M 的逆记作 M^{-1} ，满足 $M^{-1}M = I$ （先做 M 再做 M^{-1} ，等于什么都没做）。

【从例子出发，逐个想"怎么撤销"】

- **平移**：往右挪了 $(2, 2, 5)$ ，想撤销就往回挪 $(-2, -2, -5)$ 。验证：点 $(1, 2, 3) \xrightarrow{+(2,2,5)} (3, 4, 8) \xrightarrow{+(-2,-2,-5)} (1, 2, 3)$ ✓ 回到原点。所以 $T^{-1}(d) = T(-d)$ 。
- **旋转**：逆时针转了 30° ，想撤销就顺时针转回 30° ，即转 -30° 。所以 $R^{-1}(\theta) = R(-\theta)$ 。又因旋转矩阵是正交矩阵（预备 0.15），它的逆正好等于**转置**： $R(-\theta) = R^T(\theta)$ 。
- **缩放**：放大了 2 倍，想撤销就缩小到 $\frac{1}{2}$ （乘 0.5）；所以系数取**倒数**： $S^{-1}(s_x, s_y, s_z) = S(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z})$ 。

【公式表】

变换	逆变换	几何含义
平移 $T(d_x, d_y, d_z)$	$T(-d_x, -d_y, -d_z)$	往回挪
旋转 $R(\theta)$	$R(-\theta) = R^T(\theta)$	转回去（正交矩阵逆=转置）
缩放 $S(s_x, s_y, s_z)$	$S(1/s_x, 1/s_y, 1/s_z)$	缩放系数取倒数

利用 $\cos(-\theta) = \cos \theta$, $\sin(-\theta) = -\sin \theta$, $R(-\theta)$ 就是把 $R(\theta)$ 里所有 $\sin$ 变号——结果正好等于它的转置。

【复合变换的逆：顺序要反过来】 如果做了一串变换 $M = BA$ （先 A 后 B ），撤销时必须**反着来**：先撤 B 、再撤 A ，即

$$(BA)^{-1} = A^{-1}B^{-1}$$

生活类比：穿衣服先穿袜子、再穿鞋；脱的时候只能先脱鞋、再脱袜子——撤销的顺序和原来正好相反。这个"顺序反转"非常容易错，务必记牢。

2.6 变换的复合与顺序（最容易错，重点理解）

【先假设你不懂】 一个物体常常要连续做好几个变换（先转、再移、再放大……）。我们能不能把它们合成一个矩阵，一次算完？能——把各变换矩阵**相乘**就行。但这里藏着新手最大的坑：**顺序不能随便换**。

【为什么要合成】 假设要对一个有 10 万个顶点的模型做"先 A 后 B 再 C "。如果先把 C, B, A 乘成一个总矩阵 $M = CBA$ ，那对每个顶点只需算一次 Mp ；否则每个顶点要分三步算，慢三倍。**合成一次、复用多次**，这是用矩阵的核心好处。

【读法：最靠近点的先作用】 写成 $p' = CBAp$ ，要从**右往左**读：

$$p' = CBAp = C(B(Ap))$$

最右边、最贴着眼点 p 的 A 先作用，然后 B ，最后 C 。所以"先做的变换写在右边"。

【关键坑：顺序换了，结果就不一样】因为矩阵乘法不满足交换律 ($AB \neq BA$)，"先转后移"和"先移后转"是两回事。用具体数字看一眼就懂——取二维点 $P = (1, 0)$ ：

- 方案 A：先平移 $(2, 0)$ ，再绕原点转 90°

$$(1, 0) \xrightarrow{+(2,0)} (3, 0) \xrightarrow{\text{转}90^\circ} (0, 3)$$

- 方案 B：先绕原点转 90° ，再平移 $(2, 0)$

$$(1, 0) \xrightarrow{\text{转}90^\circ} (0, 1) \xrightarrow{+(2,0)} (2, 1)$$

结果 $(0, 3) \neq (2, 1)$ ，完全不同！（直觉：方案 A 先把点推远再绕原点转，转出去的弧更大；方案 B 先在原地转好再整体推走。）所以写复合矩阵时，顺序一旦写反，答案就全错。

【欧拉角也是同理】把绕三个坐标轴的旋转复合起来描述任意朝向，叫欧拉角：

$$R(\theta) = R_z(\theta_z) R_y(\theta_y) R_x(\theta_x)$$

调换 x, y, z 的旋转顺序会得到不同的朝向；若想换顺序又保持同样效果，三个角度都得重新算。这也是第 3 节"绕任意轴旋转"为什么必须严格按顺序拼矩阵的原因。

2.7 【例题】把平移 + 旋转 + 缩放串起来算一个点

把 2.1~2.6 用一道题串通：建一个复合矩阵，再作用到点上。

题目（二维，为看清楚）：对点 $P = (1, 0)$ 依次做 ① 沿 x, y 放大 2 倍（缩放 S ）→ ② 绕原点逆时针转 90° （旋转 R ）→ ③ 平移 $(1, 1)$ （平移 T ）。求复合矩阵 M 和变换后的点 P' 。

写思路："先 S 、再 R 、最后 T "，按"先做的写右边"组织复合矩阵，所以 $M = T R S$ 。先合成矩阵，再把齐次点 $P = (1, 0, 1)^T$ 代入矩阵乘法求 P' 。

写公式：二维齐次坐标下，缩放、旋转、平移分别写成矩阵；复合矩阵和点变换公式为

$$M = T R S, \quad P' = M P$$

本题中 90° 时 $\cos = 0, \sin = 1$ ，三个矩阵为

$$S = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

套数据计算：从右往左乘。先 $R S$ ：

$$RS = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -2 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

再 $T(RS)$ (左乘 T 只是把平移列加上 $(1, 1)$):

$$M = \begin{pmatrix} 0 & -2 & 1 \\ 2 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

再作用到 $P = (1, 0)$, 它的齐次形式是 $(1, 0, 1)^T$:

$$P' = M \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 - 2 \cdot 0 + 1 \\ 2 \cdot 1 + 0 + 1 \\ 1 \end{pmatrix} = (1, 3)$$

分步验证: 不合成矩阵而一步步做, 也应得同一答案。

$$(1, 0) \xrightarrow{S \times 2} (2, 0) \xrightarrow{R 90^\circ} (0, 2) \xrightarrow{T+(1,1)} (1, 3) \checkmark$$

合成矩阵和分步走结果一致, 说明 $M = TRS$ 的顺序写对了。若把顺序写成 SRT 则会得到完全不同的错误结果。

3. 绕任意轴 / 任意中心旋转与复合变换

这是变换部分最常考的小题之一。**核心难点:** 第2节的旋转矩阵 R_x, R_y, R_z 只会"绕坐标轴、且过原点"地转。可现实里旋转轴往往**既不在原点、也不是 x/y/z 轴**。怎么办? 答案是——**想办法把"难的情况"搬成"会做的情况", 做完再搬回去**。下面一步步看。

3.1 绕"不在原点"的不动点 p_f 旋转

【先看一个具体问题】 一个钟表的指针, 要绕**钟面中心** $p_f = (1, 1, 0)$ 旋转, 而不是绕原点。但我们手里的 R_z 只会绕**原点**转。如果直接用 R_z , 指针会绕错地方 (绕到原点去了), 整个表盘都飞了。

【为什么不能直接用】 R_z 让所有点绕 $(0, 0)$ 转。 $(1, 1, 0)$ 这个点本该不动 (它是旋转中心), 可 R_z 会把它也转走。所以必须想办法"骗"一下。

【逐步推导】 三步搬运法:

- 1. 先把不动点挪到原点:** 整个物体平移 $-p_f$, 于是 p_f 正好落在原点。变换 $T(-p_f)$ 。
- 2. 现在绕原点转** (这是我们会做的): $R(\theta)$ 。此时 p_f 在原点, 转它原地不动, 正合心意。
- 3. 再挪回去:** 平移 $+p_f$, 把物体送回原来的位置。变换 $T(p_f)$ 。

把三步按"先做的写右边"拼起来:

【公式】

$$M = T(p_f) R(\theta) T(-p_f)$$

读法：最右的 $T(-p_f)$ 先作用（搬到原点），然后 $R(\theta)$ （转），最后 $T(p_f)$ （搬回去）。

为什么这里 $R(\theta)$ 不带下标？ 因为三维旋转其实是**绕一根轴**转的，不动点 p_f 只决定“轴从哪穿过”，还需另外知道“轴朝哪个方向”。3.1 只负责处理“中心不在原点”这一件事，把方向留空，所以写成通用的 $R(\theta)$ ——它是个**占位符**，等你知道轴方向后再填下标：轴平行 z 轴就填 $R_z(\theta)$ ，轴平行 x 轴就填 $R_x(\theta)$ ，轴是斜的就填 3.2 推出的那串复合旋转。

数字直觉：不动点 p_f 走一遍—— $T(-p_f)$ 把它送到原点 $(0,0,0)$ ， R 转它原地不动， $T(p_f)$ 再送回 p_f 。所以它确实没动，符合“它是旋转中心”。

3.2 绕“过原点、但方向是斜的”任意轴旋转

【先看一个具体问题】 旋转轴是一根**斜着**的方向 r （比如指向 $(1,1,1)$ ），过原点。我们只会绕 x, y, z 轴转，不会绕一根斜轴转，怎么办？

【逐步推导】 思路和 3.1 一样——把“斜轴”扳正成我们会的 z 轴。设单位旋转轴 $u = (a, b, c)$ （已归一化， $a^2 + b^2 + c^2 = 1$ ）：

1. 绕 x 轴转 α ，把 u 转进 xz 平面（消掉 y 分量）。看 u 在 yz 平面的投影 $(0, b, c)$ ，长度 $d = \sqrt{b^2 + c^2}$ ，要把它转到 $+z$ 上。 α 由分量直接给出（实战不必求角度本身）：

$$\cos \alpha = \frac{c}{d}, \quad \sin \alpha = \frac{b}{d}$$

验证（代入 2.2 的同一个 $R_x(\alpha)$ 作用在 $(0, b, c)$ 上）：新 $y = \frac{c}{d}b - \frac{b}{d}c = 0\checkmark$ ，新 $z = \frac{b^2+c^2}{d} = d\checkmark$ 。此后轴变为 $(a, 0, d)$ 。

2. 绕 y 轴转 β ，把 $(a, 0, d)$ 扳到 $+z$ ：

$$\cos \beta = d, \quad \sin \beta = -a$$

验证（代入 2.2 的 $R_y(\beta)$ 作用在 $(a, 0, d)$ 上）：新 $x = da + (-a)d = 0\checkmark$ ，新 $z = a^2 + d^2 = 1\checkmark \rightarrow$ 落到 $+z$ 。

3. 绕 z 轴转 θ （现在会做了）： $R_z(\theta)$ 。

4. 把轴转回原来的斜方向：把第 1、2 步反着做一遍 $R_x(-\alpha)R_y(-\beta)$ 。

关键澄清（消除"两套定义"的疑虑）：上面 $R_x(\alpha), R_y(\beta)$ 用的就是 2.2 节那两个原始矩阵， α, β 是货真价实的旋转角，只不过我们不去求角度本身，而是直接由轴分量给出它的 $\cos$ 、 $\sin$ 代进去。所以全程**只有一套** rotate 定义，与 2.2 完全一致。 α 的几何含义是"投影 $(0, b, c)$ 与 $+z$ 轴的夹角"（在 yz 平面内）。

边界坑：若轴本就沿 x 轴（ $b = c = 0$ ），则 $d = 0$ 、 α 公式分母为 0 无定义——此时轴已在 xz 平面，跳过第 1 步、直接做第 2 步即可。

【公式】

$$R(\theta) = \underbrace{R_x(-\alpha)R_y(-\beta)}_{\text{把轴转回去}} R_z(\theta) \underbrace{R_y(\beta)R_x(\alpha)}_{\text{把轴扳到 } z}$$

其中 α, β 即上面由 $u = (a, b, c)$ 的分量算出的两个角（ $\cos \alpha = c/d, \sin \alpha = b/d$ ； $\cos \beta = d, \sin \beta = -a, d = \sqrt{b^2 + c^2}$ ）。

为什么左右两半符号相反、顺序相反？ 因为"转回去（undo）"必须是"扳到 z （align）"的**逆变换**。由 2.5 节"乘积的逆 = 倒序 + 每个角取负"：align = $R_y(\beta)R_x(\alpha) \Rightarrow$ undo = $R_x(-\alpha)R_y(-\beta)$ 。注意 α, β 的正负号已经由上面的 $\cos / \sin$ 公式自动定好了（ $\sin \alpha = b/d$ 的符号跟着 b 走、 $\sin \beta = -a$ 跟着 $-a$ 走），不用再纠结"该 + 还是 -"——照公式代即可。**记忆法：**别背符号，记住"align 与 undo 互为逆（倒序 + 取负）"， α, β 的 $\cos / \sin$ 由轴分量直接给。

看出规律了吗？3.1 和 3.2 是**同一个套路**：把不会做的（中心不在原点 / 轴是斜的）通过变换"搬成"会做的，做完再搬回去。

3.3 绕"任意轴 + 任意中心"旋转（最一般情况）

【组合起来】 轴既是斜的、又不过原点（过轴上某点 p_0 ）。那就把 3.1（处理中心）和 3.2（处理方向）叠在一起：先把 p_0 搬到原点，再按 3.2 绕该方向轴转，最后搬回。

【公式】

$$M = T(p_0) R(\theta) T(-p_0)$$

其中 p_0 是轴上一点， $R(\theta)$ 是 3.2 推出的"绕该斜方向轴"的旋转。

3.4 【例题 1】绕平行于 Z 轴、过点 (1, 1, 0) 的轴逆时针旋转 30°

写思路：旋转轴平行于 z 轴，方向不斜，省去 3.2 的两次对齐；但旋转中心在 (1, 1, 0)，不在原点，所以用 3.1 的“搬到原点—转—搬回”。也就是说，先把轴上点平移到原点，再绕 z 轴旋转，最后平移回原位置。

写公式：把视口矩阵和行列换算公式先写出来。

$$R = T(1, 1, 0) R_z(30^\circ) T(-1, -1, 0)$$

套数据计算：令 $c = \cos 30^\circ = \frac{\sqrt{3}}{2}$, $s = \sin 30^\circ = \frac{1}{2}$ ，逐步相乘。

第一步， $R_z(30^\circ) T(-1, -1, 0)$ （先把旋转作用到“已搬到原点”的坐标上），平移列变为 $R_z \cdot (-1, -1, 0)$ ：

$$R_z(30^\circ) T(-1, -1, 0) = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & s - c \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & -s - c \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

第二步，再左乘 $T(1, 1, 0)$ （搬回去，平移列各加 1）：

$$R = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{3}{2} - \frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} - \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

快捷验证法：纯绕不动点 p_f 的旋转，平移列必等于 $p_f - R_z p_f$ 。这里 $R_z(1, 1, 0) = (c - s, s + c, 0)$ ，故平移列 = $(1 - (c - s), 1 - (s + c), 0) = (\frac{3}{2} - \frac{\sqrt{3}}{2}, \frac{1}{2} - \frac{\sqrt{3}}{2}, 0)$ ，与上面一致。

逆矩阵（撤销这次旋转 = 绕同一条轴转 -30° ）：

$$R^{-1} = T(1, 1, 0) R_z(-30^\circ) T(-1, -1, 0) = \begin{pmatrix} \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 & \frac{1}{2} - \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{3}{2} - \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3.5 【完整例题】绕一根真正“斜”的轴旋转 30°

如果旋转轴刚好平行某个坐标轴，问题会简单很多；但这道题故意选一根三个分量都非零的斜轴，把“先对齐到 z 轴、再旋转、最后转回去”这套机制完整跑一遍。

题目：绕过原点、方向为单位向量 $u = (-\frac{1}{2}, \frac{\sqrt{3}}{4}, \frac{3}{4})$ 的轴，逆时针旋转 $\theta = 30^\circ$ ，求旋转矩阵 R 。

(这根轴是我特意挑的, 好让 α, β 正好都是漂亮的 30° ; 实际考试给什么轴, 就按下面公式照算什么。先验证它是单位向量: $\frac{1}{4} + \frac{3}{16} + \frac{9}{16} = \frac{4+3+9}{16} = 1 \checkmark$ 。)

写思路: 斜轴过原点, 但方向不是坐标轴, 所以先把斜轴对齐到 z 轴, 在 z 轴上做普通旋转, 再用逆变换把轴转回去。计算上先由轴分量求 α, β , 再按五个旋转矩阵的夹心结构相乘。

写公式: 设单位轴 $u = (a, b, c)$, 且 $d = \sqrt{b^2 + c^2}$, 对齐角满足

$$\cos \alpha = \frac{c}{d}, \quad \sin \alpha = \frac{b}{d}, \quad \cos \beta = d, \quad \sin \beta = -a$$

在这种对齐、旋转、还原的顺序下, 绕斜轴旋转的矩阵为

$$R = R_x(-\alpha)R_y(-\beta)R_z(\theta)R_y(\beta)R_x(\alpha)$$

套数据计算: 记 $a = -\frac{1}{2}$, $b = \frac{\sqrt{3}}{4}$, $c = \frac{3}{4}$, 先求 yz 投影长度

$$d = \sqrt{b^2 + c^2} = \sqrt{\frac{3}{16} + \frac{9}{16}} = \sqrt{\frac{12}{16}} = \frac{\sqrt{3}}{2}$$

把本题的轴分量代入上面的公式:

$$\cos \alpha = \frac{c}{d} = \frac{3/4}{\sqrt{3}/2} = \frac{\sqrt{3}}{2}, \quad \sin \alpha = \frac{b}{d} = \frac{\sqrt{3}/4}{\sqrt{3}/2} = \frac{1}{2} \Rightarrow \alpha = 30^\circ$$

$$\cos \beta = d = \frac{\sqrt{3}}{2}, \quad \sin \beta = -a = \frac{1}{2} \Rightarrow \beta = 30^\circ$$

于是 $\alpha = \beta = \theta = 30^\circ$, 三个角都用 $c = \cos 30^\circ = \frac{\sqrt{3}}{2}$, $s = \sin 30^\circ = \frac{1}{2}$ 。

写出五个旋转矩阵。它们都是 2.2 的原矩阵, 齐次补成 4×4 时第四行列为 $0, 0, 0, 1$, 这里只写 3×3 主块:

$$R_x(30^\circ) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ 0 & \frac{1}{2} & \frac{\sqrt{3}}{2} \end{pmatrix} \quad R_y(30^\circ) = \begin{pmatrix} \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \\ -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} \end{pmatrix} \quad R_z(30^\circ) = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

另外两个是 $R_y(-30^\circ), R_x(-30^\circ)$ (把上面的 R_y, R_x 里 $\sin$ 变号即可)。

把三个角代入夹心结构后, 矩阵乘法顺序为:

$$R = \underbrace{R_x(-30^\circ)R_y(-30^\circ)}_{\text{③ 把轴扳回去}} \underbrace{R_z(30^\circ)}_{\text{② 绕 } z \text{ 转}} \underbrace{R_y(30^\circ)R_x(30^\circ)}_{\text{① 把轴扳到 } z}$$

含义: 右边两个先把整个空间拧一下、让斜轴 u 正好躺到 z 轴上 $\rightarrow$ 中间在 z 轴上做我们最熟的 $R_z(30^\circ) \rightarrow$ 左边两个再把空间拧回去。

验证：轴自己必须不动，这是最干净的检验。一根轴绕它**自己**转，应当原地不动，即 $Ru = u$ 。沿夹心从右往左让 u 走一遍：

① $R_x(30^\circ)u$:

$$\begin{cases} y' = \frac{\sqrt{3}}{2} \cdot \frac{\sqrt{3}}{4} - \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8} - \frac{3}{8} = 0 \\ z' = \frac{1}{2} \cdot \frac{\sqrt{3}}{4} + \frac{\sqrt{3}}{2} \cdot \frac{3}{4} = \frac{\sqrt{3}}{8} + \frac{3\sqrt{3}}{8} = \frac{\sqrt{3}}{2} \end{cases} \Rightarrow u \rightarrow \left(-\frac{1}{2}, 0, \frac{\sqrt{3}}{2}\right)$$

y 分量被消掉，轴进了 xz 平面 $\checkmark$ (结果正是 $(a, 0, d)$)。

② $R_y(30^\circ)\left(-\frac{1}{2}, 0, \frac{\sqrt{3}}{2}\right)$:

$$\begin{cases} x' = \frac{\sqrt{3}}{2} \cdot \left(-\frac{1}{2}\right) + \frac{1}{2} \cdot \frac{\sqrt{3}}{2} = -\frac{\sqrt{3}}{4} + \frac{\sqrt{3}}{4} = 0 \\ z' = -\frac{1}{2} \cdot \left(-\frac{1}{2}\right) + \frac{\sqrt{3}}{2} \cdot \frac{\sqrt{3}}{2} = \frac{1}{4} + \frac{3}{4} = 1 \end{cases} \Rightarrow (0, 0, 1)$$

轴已对齐到 $+z$ $\checkmark$ 。

③ $R_z(30^\circ)(0, 0, 1) = (0, 0, 1)$ ——绕 z 轴转， z 轴上的点原地不动 $\checkmark$ 。

④⑤ 再用 $R_y(-30^\circ)R_x(-30^\circ)$ (①②的逆) 把 $(0, 0, 1)$ 一步步送回 u 。

所以 $Ru = u$ ，轴确实不动——机制正确。

若轴不过原点、而过某点 p_0 (最一般的"全量任意旋转")：在外面再套一层平移即可 (3.3 的公式)：

$$M = T(p_0) R T(-p_0)$$

其中 R 是本题刚刚构造出的斜轴旋转矩阵。这样：先把 p_0 搬到原点 (轴随之过原点) $\rightarrow$ 用 R 绕斜轴转 $\rightarrow$ 把 p_0 搬回去。

这道题想讲明白的核心：再"任意"的旋转，都被拆成了**我们已经会的**绕坐标轴旋转 + 平移。 α, β 不是凭空的角度，而是"把斜轴拧到 z 轴"算出来的，且用的就是 2.2 的同一套旋转矩阵。整条公式 = 扳正 $\rightarrow$ 简单转 $\rightarrow$ 扳回 $\rightarrow$ (如需) 平移回中心。

4. 相机变换 gluLookAt

【先假设你不懂】 相机可以放在世界任意位置、朝任意方向。但所有投影公式都假设"相机在原点、朝 $-z$ 看"。怎么把"相机乱放"的场景，变回"相机在原点"的标准情形？

【一个生活类比】你坐在房间里拍照。要让某个物体出现在照片正中，有两个办法：(a) 你端着相机走过去对准它；(b) 你不动，让整个房间反方向滑过来对准你。两种办法拍出的照片一模一样。计算机选 (b)——**相机不动，反向搬动整个世界**。相机往右 1 米，等价于世界往左 1 米；相机左转 30° ，等价于世界右转 30° 。这就是相机变换的全部秘密：**把"相机的运动"反过来作用到世界上**。

【分两步：先平移，再旋转】

要把"相机在 E 、朝某方向"搬成"相机在 origin、朝 $-z$ "，对世界做两件事：

第一步——平移：相机现在在位置 E ，先把整个世界平移 $-E$ ，相机就回到了原点。这步是 $T(-E)$ 。

第二步——旋转对齐：相机现在虽在 origin，但它的"右、上、后"三个朝向还是歪的，要把它们转正、对齐到世界的 x, y, z 轴。要做这步，得先知道相机自己的三根轴朝哪——这就是构造相机的正交基 u, v, w 。

先搞清 u, v, w 到底是啥（身体版）：把你自己当相机。你的**正右手方向**就是 u 、**头顶方向**就是 v 、**后脑勺方向（视线正后方）**就是 w 。这三根是"长在相机身上、随相机一起转"的轴，和世界的 x, y, z 轴一般不重合。相机变换要干的，就是把这三根"长歪了"的轴转到和世界 x, y, z 对齐。所以第一件事是：用世界坐标把 u, v, w 写出来。

【怎么求相机的三根轴 u, v, w 】输入只有三样：相机位置 eye 、看的目标点 at 、一个"大概朝上"的提示向量 up 。下面一边讲直觉、一边用一个具体例子当场算。

贯穿例子： $eye = (0, 0, 0)$ ， $at = (1, 0, -1)$ ， $up = (0, 1, 0)$ 。即相机站在 origin，斜着朝"右前方（ $+x$ 、 $-z$ 之间）"看。

第 1 步：后方轴 w （先定"朝哪看"）。相机看的方向是 $at - eye$ （从眼睛指向目标）。但 OpenGL 的相机是顺着自己的 $-z$ 看的，所以相机的 $+z$ 轴（我们叫"后方" w ）指向**看的反方向**： $w = \frac{eye - at}{\|eye - at\|}$ 。

- 算： $eye - at = (0, 0, 0) - (1, 0, -1) = (-1, 0, 1)$ ，长度 $\sqrt{(-1)^2 + 1^2} = \sqrt{2}$ ， $w = \left(-\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}\right)$
- 画面：相机朝右前方看，它的"背后"自然指向左后方（ $-x$ 、 $+z$ 之间），对上了。

第 2 步：右方轴 u （叉乘造垂直方向）。相机的"右手边"必须同时垂直于"后方 w "和"上方"。怎么得到一个同时垂直于两个已知向量的向量？——**叉乘**（预备 0.14）。用提示 up 和刚得到的 w 叉乘： $u = \frac{up \times w}{\|up \times w\|}$ 。用行列式算：

$$up \times w = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 0 & 1 & 0 \\ -\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \end{vmatrix} = \left(1 \cdot \frac{1}{\sqrt{2}} - 0, -(0 - 0), 0 - 1 \cdot \left(-\frac{1}{\sqrt{2}}\right)\right) = \left(\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}\right)$$

（已是单位向量，无需再归一化。）

- 画面：相机斜着看，它的右手边指向右前方（ $+x$ 、 $+z$ 之间）的斜方向——看，叉乘真的造出了一根倾斜的轴，不再是规规矩矩的坐标轴。
- 为什么是 $up \times w$ 而不是 $w \times up$ ？顺序决定方向（叉乘不可交换）。按右手定则， $up \times w$ 指向相机的右手边；写反了右就变左了。

第3步：上方轴 v （再叉乘一次，修正 up ）。我们一开始给的 up 只是个“大概朝上”的提示，它不一定恰好垂直于视线（好比你歪着头拍照）。所以不能直接拿 up 当相机的上轴，而要用已经互相垂直的 w, u 再叉乘一次，得到精确、且保证垂直、且单位长的上方向： $v = w \times u$ 。

$$v = w \times u = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \end{vmatrix} = (0, 1, 0)$$

- 这里 v 正好等于原来的 $up = (0, 1, 0)$ ，因为本例的 up 恰好已经垂直视线。但如果 up 是歪的，这一步算出的 v 就会和 up 不同——这正是第3步存在的意义：自动把歪的 up “扶正”。这也是考点“当 up 不平行投影面时怎么办”的答案：不用手动调，叉乘自动搞定。

【小结这三步】一句话： w 取视线反方向 $\rightarrow u = up \times w$ （造右） $\rightarrow v = w \times u$ （造正的上）。三者互相垂直、都归一化，就是相机的正交基。

【公式（背这三行）】

$$w = \frac{eye - at}{\|eye - at\|}, \quad u = \frac{up \times w}{\|up \times w\|}, \quad v = w \times u$$

【关键：怎么把世界里的点 P 换算成“相机眼里”的坐标】这是整节最容易飘的一步，我们用最实在的方式想。

“ P 在相机眼里的坐标”无非是回答三个问题：

- P 在我右手方向上离我多远？ $\rightarrow$ 这就是相机坐标的 x 。
- P 在我头顶方向上离我多远？ $\rightarrow$ 相机坐标的 y 。
- P 在我前后方向上离我多远？ $\rightarrow$ 相机坐标的 z 。

“离我多远”要从相机位置 E 量起，所以看的是向量 $(P - E)$ ；“在某个方向上走了多远”= 把 $(P - E)$ 往那个单位方向上投影 = 和那个方向点乘（预备 0.14）。于是：

$$x_{\text{相机}} = (P - E) \cdot u, \quad y_{\text{相机}} = (P - E) \cdot v, \quad z_{\text{相机}} = (P - E) \cdot w$$

把它拆开看，矩阵就自己冒出来了。以 x 为例：

$$x_{\text{相机}} = (P - E) \cdot u = \underbrace{u \cdot P}_{\text{用 } u \text{ 那一行去乘 } P} - \underbrace{u \cdot E}_{\text{常数，进平移列}}$$

- “ $u \cdot P$ ”正是“矩阵第一行 = u ”去乘点 P ——所以矩阵的三行就是 u, v, w （用哪根轴量，就把哪根轴摆成对应那一行）。

- $-u \cdot E$ 是个固定常数，正好就是平移列里的第一项。

三行合起来，视图矩阵 V 就是：

$$V = \begin{pmatrix} u_x & u_y & u_z & -u \cdot E \\ v_x & v_y & v_z & -v \cdot E \\ w_x & w_y & w_z & -w \cdot E \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

一句话读懂：前三行 = 相机三根轴（回答“在右 / 上 / 前后各走了多远”）；最后一列 = $-(\text{轴} \cdot E)$ （把“从相机量起”那个减法装进来）。

【端到端验证：喂一个点进去，看它变成什么】用贯穿例子（ $E = (0, 0, 0)$ ，故平移列为 0； $u = (\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})$ ， $v = (0, 1, 0)$ ， $w = (-\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})$ ）。我们把目标点 $at = (1, 0, -1)$ 喂进去，它应该出现在相机正前方正中央：

$$x_{\text{相机}} = at \cdot u = (1)(\frac{1}{\sqrt{2}}) + (-1)(\frac{1}{\sqrt{2}}) = 0$$

$$y_{\text{相机}} = at \cdot v = 0$$

$$z_{\text{相机}} = at \cdot w = (1)(-\frac{1}{\sqrt{2}}) + (-1)(\frac{1}{\sqrt{2}}) = -\sqrt{2}$$

结果 $(0, 0, -\sqrt{2})$ ：左右居中（ $x = 0$ ）、上下居中（ $y = 0$ ）、在相机前方 $\sqrt{2}$ 处（ $z = -\sqrt{2}$ ，相机朝 $-z$ 看，负 z 就是前方）。完全符合“目标点应在正前方正中”——整套机器跑通了。

易错点： w 指向视线的反方向（ $eye - at$ ，不是 $at - eye$ ），因为相机朝 $-z$ 看。

4.1 【例题 2】求 `gluLookAt(1,0,0, 1,0,-1, 0,1,0)` 的相机基与视图矩阵

写思路：`gluLookAt` 先用 eye, at, up 构造相机自己的三根单位轴 u, v, w ，再把这三根轴写进视图矩阵。这里相机从 $(1, 0, 0)$ 看向 $(1, 0, -1)$ ，实际仍然沿世界的 $-z$ 方向看，所以三根相机轴会和世界坐标轴对齐；唯一的变化是相机位置不在原点，视图矩阵需要把世界整体向左平移 1。

写公式：先写出完整流水线里模型、视图、投影、视口的矩阵链路。

$$w = \frac{eye - at}{\|eye - at\|}, \quad u = \widehat{up \times w}, \quad v = w \times u$$

把相机基和相机位置写进矩阵，视图矩阵为

$$V = \begin{pmatrix} u_x & u_y & u_z & -u \cdot E \\ v_x & v_y & v_z & -v \cdot E \\ w_x & w_y & w_z & -w \cdot E \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

套数据计算： $E = (1, 0, 0)$ ， $at = (1, 0, -1)$ ， $up = (0, 1, 0)$ 。

$$eye - at = (0, 0, 1) \Rightarrow w = (0, 0, 1)$$

$$u = \widehat{up} \times w = (0, 1, 0) \times (0, 0, 1) = (1, 0, 0)$$

$$v = w \times u = (0, 0, 1) \times (1, 0, 0) = (0, 1, 0)$$

$$u \cdot E = 1, \quad v \cdot E = 0, \quad w \cdot E = 0$$

$$V = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

5. 投影变换：正交规范化与透视

【先假设你不懂】投影就是把三维场景"拍扁"成二维图。但计算机偷了个懒：它不直接拍扁，而是先把眼前那块"看得见的空间"（叫**视景体**）统一**规范化**成一个中心在原点、边长为 2 的标准立方体 $[-1, 1]^3$ 。

为什么要多此一举？ 两个好处：① 不管原来是正交还是透视，规范化之后都长一个样，**后面的流水线只需写一套**；② 裁剪（判断点是否在视野内）变得超简单——只要看坐标是不是都落在 -1 到 1 之间就行。**正交规范化矩阵的推导是重点考点。**

5.1 平行投影 vs 透视投影（概念对比）

	正交（平行）投影	透视投影
投影线	互相平行、垂直于投影面	汇聚于投影中心 COP
大小	不随距离变化	近大远小（diminution）
角度/长度	保持，可直接测量	一般不保持，难测量
灭点	无	平行线交于灭点；立方体可有 1/2/3 个灭点
用途	CAD、工程图、三视图	动画、真实感渲染

5.2 正交投影规范化矩阵（务必会推导）

【先假设你不懂】 `glOrtho(l,r,b,t,n,f)` 给了一个长方体视景体（ $x \in [l, r], y \in [b, t], z \in [-n, -f]$ ）。为什么要把它变成中心在原点、边长为 2 的标准立方体 $[-1, 1]^3$ ？因为这样**同一条流水线就能同时处理正交和透视**，裁剪也只需判断"是否在 $[-1, 1]$ 内"，简单统一。

【思路：两步，对应两个矩阵相乘】把长方体变成标准立方体，只需两步，每步就是一个我们已经会的矩阵（第 2 节）：

1. **平移 T** : 把长方体的**中心移到原点**。中心是各区间的中点 $\left(\frac{l+r}{2}, \frac{b+t}{2}, \frac{-n-f}{2}\right)$ ，所以要平移它的相反数。
2. **缩放 S** : 把三条边长 $(r-l, t-b, f-n)$ 都**压成2**，即各乘 $\frac{2}{\text{边长}}$ 。其中 z 方向因相机看 $-z, n, f$ 取正距离，要带负号翻转。

合起来 $N_{ortho} = S \cdot T$ (先 T 后 S ，先做的写右边)。

【把两个矩阵写出来】

$$T = \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & \frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{-2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(T 的 z 平移是 $+\frac{n+f}{2}$: 因为眼睛空间里 $z \in [-n, -f]$, 中心在 $-\frac{n+f}{2}$, 要加 $\frac{n+f}{2}$ 才挪到 0。)

【相乘: S 乘 T 怎么乘出来】 S 是对角矩阵, **左乘** 一个矩阵的效果, 就是把被乘矩阵的**第 i 行整行乘以 S 的第 i 个对角元** (包括那一行的平移列)。所以:

- 第 1 行整体 $\times \frac{2}{r-l}$: 对角项变 $\frac{2}{r-l}$, 平移项 $-\frac{l+r}{2} \times \frac{2}{r-l} = -\frac{r+l}{r-l}$ 。
- 第 2 行整体 $\times \frac{2}{t-b}$: 平移项 $-\frac{t+b}{2} \times \frac{2}{t-b} = -\frac{t+b}{t-b}$ 。
- 第 3 行整体 $\times \frac{-2}{f-n}$: 平移项 $\frac{n+f}{2} \times \frac{-2}{f-n} = -\frac{f+n}{f-n}$ 。

【最终矩阵】

$$N_{ortho} = ST = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

对角线是三个缩放系数、平移列是"缩放系数 $\times$ 原平移量"。 z 行带负号是因为相机看 $-z, n, f$ 取距离 (正数)。如果要检查数值, 可以直接把近面 $z = -n$ 代入看是否得到 $z_n = -1$, 再把远面 $z = -f$ 代入看是否得到 $z_n = 1$ 。

5.3 透视投影矩阵

【先假设你不懂】透视就是"近大远小"——同样大的两个人, 离你近的看起来大、离你远的看起来小。这个效果到底怎么用数学算出来? 我们从最朴素的"小孔成像"一步步来。

【从例子出发: 相似三角形】想象相机在原点, 朝 $-z$ 看, 在它前方距离 n 处立一块"成像屏幕" (近平面, 位于 $z = -n$)。空间里有个点, 眼睛空间坐标 (x_e, y_e, z_e) ($z_e < 0$, 在相机前方)。这个点和原点连一条线, 线穿过屏幕的位置, 就是它在屏幕上的投影。

由**相似三角形** (大三角形: 点到相机; 小三角形: 屏幕到相机), 屏幕上的投影横坐标 x_p 满足:

$$\frac{x_p}{n} = \frac{x_e}{-z_e} \implies x_p = \frac{n x_e}{-z_e}$$

【用数字感受"近大远小"】 设屏幕距离 $n = 1$ 。两个点横坐标都是 $x_e = 2$ （一样靠右）：

- 近点 A: $z_e = -2$ （前方 2 米） $\rightarrow x_p = \frac{1 \times 2}{2} = 1$ 。
- 远点 B: $z_e = -4$ （前方 4 米，远一倍） $\rightarrow x_p = \frac{1 \times 2}{4} = 0.5$ 。

同样的真实横坐标，远的那个投影到屏幕上只有近的一半——这就是近大远小！关键动作是：把 x, y 除以深度 $-z_e$ 。越远（ $-z_e$ 越大），投影越往中间缩、越小。

【难点：矩阵不会"除以一个变量"】 现在卡住了：矩阵只能做乘和加（线性运算），没法"除以 z_e "（ z_e 是变量，不是固定数）。怎么办？

【神来之笔：用齐次坐标的 w 把除法"赊账"到最后】 回忆预备 0.5：齐次坐标最后要做透视除法——把 (x, y, z, w) 全部除以 w 。这正好是个"除法"！所以我们让矩阵别急着除，而是把"要除的那个数 $-z_e$ "塞进第四维 w 里，留到最后透视除法时一并除掉。

要让输出的 $w = -z_e$ ，矩阵最后一行就得是 $(0, 0, -1, 0)$ （它专门把 $-z_e$ 挑出来放进 w ）。验证：

$$(0, 0, -1, 0) \cdot (x_e, y_e, z_e, 1)^T = -z_e \checkmark$$

这样矩阵相乘后得到的齐次坐标，第四维就是 $-z_e$ ；最后做透视除法 $\div w$ ， x, y 就自动除以了 $-z_e$ ，近大远小达成。

【矩阵视角推导：先把视锥压成长方体，再套用 5.2 的 N_{ortho} 】

我们在 5.2 已经会把一个长方体规范化到 $[-1, 1]^3$ （那就是 N_{ortho} ）。可透视的可视空间是个视锥（截头金字塔），不是长方体， N_{ortho} 直接用不上。思路于是很清楚——先用一个矩阵 P 把视锥"压"成长方体，再用现成的 N_{ortho} 规范化。两步相乘：

$$N_{persp} = N_{ortho} \cdot P$$

这个透视矩阵 P 长这样：

$$P = \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & nf \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

它的每一块都对应前面讲过的东西：

- 最后一行 $(0, 0, -1, 0)$ ：让输出的 $w = -z_e$ （上面定的赊账除数），这是产生"近大远小"的唯一来源。
- 前两行的 n ：把 x, y 各乘 n 。之后做透视除法 $\div (-z_e)$ 就得到 $\frac{n x_e}{-z_e}$ ，正是相似三角形投到近平面的 x_p 。所以 P 的 x, y 行就是"投影到近平面"那一步。
- 第三行 $(0, 0, n+f, nf)$ ：专门让近、远平面位置不变（从而视锥的近 / 远两个平面正好变成长方体的

近/远两面)。验证： P 作用后 z 分量是 $(n+f)z+nf$ 、 $w=-z$ ，相除得 $z' = \frac{(n+f)z+nf}{-z}$ 。代 $z=-n$ 得 $z' = \frac{-n^2}{n} = -n\checkmark$ ；代 $z=-f$ 得 $z' = \frac{-f^2}{f} = -f\checkmark$ 。近、远平面都映射到自己，斜边被"扳直"，视锥就被压成了长方体 $[l, r] \times [b, t] \times [-n, -f]$ 。

把 N_{ortho} (5.2) 和 P 相乘 (N_{ortho} 是对角缩放+平移列，照常做矩阵乘法)：

$$N_{persp} = \underbrace{\begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{N_{ortho} (5.2)} \begin{pmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & nf \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

逐行乘出来 (举两行看怎么算)：

- 第1行： $(1, 1) = \frac{2}{r-l} \cdot n = \frac{2n}{r-l}$ ； $(1, 3) = \left(-\frac{r+l}{r-l}\right) \cdot (-1) = \frac{r+l}{r-l}$ ，其余 $0 \rightarrow \left(\frac{2n}{r-l}, 0, \frac{r+l}{r-l}, 0\right)$ 。
- 第3行： $(3, 3) = \frac{-2}{f-n}(n+f) + \left(-\frac{f+n}{f-n}\right)(-1) = -\frac{f+n}{f-n}$ ； $(3, 4) = \frac{-2}{f-n} \cdot nf = -\frac{2fn}{f-n} \rightarrow \left(0, 0, -\frac{f+n}{f-n}, -\frac{2fn}{f-n}\right)$ 。
- 第4行： $(0, 0, 0, 1) \cdot P = (0, 0, -1, 0)$ 。

【最终矩阵】 `glFrustum(l, r, b, t, n, f)` 对应：

$$N_{persp} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

一句话记住来历： $N_{persp} = N_{ortho} \cdot P$ —— P 把视锥压成长方体 (并令 $w=-z_e$)， N_{ortho} 再把长方体规范化到 $[-1, 1]^3$ 。第四行 $(0, 0, -1, 0)$ 正是透视与正交最本质的区别 (正交第四行是 $(0, 0, 0, 1)$ ， w 恒为1、不用除)。

【数字验证】 用对称视锥 $l=-1, r=1, b=-1, t=1, n=1, f=3$ ，矩阵的第三行是 $\left(0, 0, -\frac{3+1}{3-1}, -\frac{2 \cdot 3 \cdot 1}{3-1}\right) = (0, 0, -2, -3)$ 。

- 取近平面中心点 $(0, 0, -1, 1)$ ： $x_c = \frac{2 \cdot 1}{2} \cdot 0 = 0$ ， $z_c = -2(-1) - 3 = -1$ ， $w_c = -z_e = 1$ 。除以 w_c 得NDC $(0, 0, -1)$ ——正好贴近平面 $-1\checkmark$ 。
 - 取远平面中心点 $(0, 0, -3, 1)$ ： $z_c = -2(-3) - 3 = 3$ ， $w_c = 3$ ， $z_n = 3/3 = 1$ ——正好贴远平面 $+1\checkmark$ 。
- 近/远都落到立方体两端，矩阵无误。

【`gluPerspective`和`glFrustum`的关系——从矩阵看】

两个函数都生成透视投影矩阵，区别只在"用什么参数描述视锥"：

- `glFrustum(l,r,b,t,n,f)`：直接给近平面窗口的四条边 l, r, b, t ，能描述左右 / 上下不对称（歪）的视锥。它的矩阵就是 5.3 推出的 N_{persp}
- `gluPerspective(fovy, aspect, near, far)`：用竖直接视野角 $fovy$ 和宽高比 $aspect$ 描述，默认上下、左右都对称。

要看清关系，就把"对称"这个条件直接代进 5.3 的 N_{persp} 矩阵，看它变成什么。对称意味着 $l = -r$ 、 $b = -t$ ，于是矩阵里两个量发生变化：

$$\frac{r+l}{r-l} = \frac{r+(-r)}{r-(-r)} = \frac{0}{2r} = 0, \quad \frac{t+b}{t-b} = 0$$

$$\frac{2n}{r-l} = \frac{2n}{2r} = \frac{n}{r}, \quad \frac{2n}{t-b} = \frac{2n}{2t} = \frac{n}{t}$$

也就是说，第三列上面那两个"歪"的项 $\frac{r+l}{r-l}$ 、 $\frac{t+b}{t-b}$ 直接变成 0 了（因为对称视锥没有偏移），矩阵简化成：

$$\begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

再把 t, r 用 $fovy, aspect$ 表示：侧面看， $\tan \frac{fovy}{2} = \frac{t}{near}$ 所以 $t = near \cdot \tan \frac{fovy}{2}$ ，又 $r = t \cdot aspect$ 。于是

$$\frac{n}{t} = \frac{1}{\tan(fovy/2)} = \cot \frac{fovy}{2}, \quad \frac{n}{r} = \frac{\cot(fovy/2)}{aspect}$$

代进去就得到 `gluPerspective` 的标准矩阵：

$$N_{persp}^{gluPerspective} = \begin{pmatrix} \frac{\cot(fovy/2)}{aspect} & 0 & 0 & 0 \\ 0 & \cot \frac{fovy}{2} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

结论（从矩阵看一目了然）：`gluPerspective` 的矩阵 = `glFrustum` 矩阵把那两个偏移项清零、并把 x/y 缩放用 $fovy, aspect$ 改写后的样子。所以 `gluPerspective` 就是 `glFrustum` 的对称特例——`glFrustum` 更一般（偏移项可非零，描述歪视锥），`gluPerspective` 是它令偏移为 0 的子集。

【换算（做题用）】拿到 `gluPerspective(fovy, aspect, near, far)`，先换成 `glFrustum` 的四条边再套 5.3：

$$t = near \cdot \tan \frac{fovy}{2}, \quad b = -t, \quad r = t \cdot aspect, \quad l = -r$$

例：`gluPerspective(90°, 2, 1, 10)` $\rightarrow t = 1 \cdot \tan 45^\circ = 1, b = -1, r = 2, l = -2 \rightarrow$ 等价 `glFrustum(-2, 2, -1, 1, 1, 10)`。

5.4 【例题】把一个点做正交投影规范化并判断是否被裁剪

这道题把 5.2（正交规范化矩阵）+ 矩阵乘法 + 裁剪判断串起来。

题目：用 `glOrtho(l,r,b,t,n,f) = glOrtho(-2, 2, -2, 2, 1, 9)`，眼睛空间里有两个点 $A = (1, -1, -3)$ 、 $B = (3, 0, -5)$ （ z 为负，在相机前方）。分别求规范化设备坐标（NDC），并判断是否落在可视立方体 $[-1, 1]^3$ 内。

写思路：先按 5.2 写出正交规范化矩阵 N_{ortho} ，再把眼睛空间点写成齐次列向量并左乘矩阵，得到裁剪坐标。正交投影不会改变 w ，这里 $w = 1$ ，所以齐次除法后得到的就是 NDC；最后逐分量判断 x_n, y_n, z_n 是否都在 $[-1, 1]$ 内。

写公式：正交规范化矩阵的一般形式为

$$N_{ortho} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

点的计算也要写成矩阵形式，公式为

$$P_c = N_{ortho}P_e, \quad P_{ndc} = \left(\frac{x_c}{w_c}, \frac{y_c}{w_c}, \frac{z_c}{w_c} \right)$$

裁剪判断只看 NDC 三个分量是否都在标准立方体内，即

$$-1 \leq x_n \leq 1, \quad -1 \leq y_n \leq 1, \quad -1 \leq z_n \leq 1$$

套数据计算：代入 $l = -2, r = 2, b = -2, t = 2, n = 1, f = 9$ ，先得

$$N_{ortho} = \begin{pmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{4} & -\frac{5}{4} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

把这个矩阵展开，就得到本题使用的三个分量公式：

$$x_n = \frac{2}{r-l}x_e - \frac{r+l}{r-l} = \frac{2}{4}x_e - 0 = \frac{x_e}{2}, \quad y_n = \frac{y_e}{2}$$

$$z_n = \frac{-2}{f-n}z_e - \frac{f+n}{f-n} = \frac{-2}{8}z_e - \frac{10}{8} = -\frac{z_e}{4} - \frac{5}{4}$$

点 $A = (1, -1, -3)$ ：

$$N_{ortho} \begin{pmatrix} 1 \\ -1 \\ -3 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ -\frac{1}{4}(-3) - \frac{5}{4} \\ 1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.5 \\ -0.5 \\ 1 \end{pmatrix}$$

三个分量 0.5, -0.5, -0.5 都在 $[-1, 1]$ 内 $\rightarrow$ **A 在视景体内，保留**。（ $z_n = -0.5$ 也合理： $z_e = -3$ 介于近面 -1 和远面 -9 之间，规范化后应在 -1 到 1 中间。）

点 $B = (3, 0, -5)$:

$$N_{ortho} \begin{pmatrix} 3 \\ 0 \\ -5 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{3}{2} \\ 0 \\ -\frac{1}{4}(-5) - \frac{5}{4} \\ 1 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

$x_n = 1.5 > 1$ 超出范围 $\rightarrow$ **B 被裁剪（落在视景体右侧之外，不显示）**。

验证近 / 远平面：把 $z_e = -1$ （近面）代入 $z_n = -\frac{-1}{4} - \frac{5}{4} = \frac{1}{4} - \frac{5}{4} = -1 \checkmark$ ； $z_e = -9$ （远面）代入 $z_n = \frac{9}{4} - \frac{5}{4} = 1 \checkmark$ 。近面映射到 -1、远面到 +1，正好贴住立方体两端，说明矩阵没错。

6. 视口变换 glViewport

【先假设你不懂】 经过投影 + 透视除法后，点的坐标落在 $[-1, 1]^3$ 这个抽象的标准立方体里。可屏幕窗口是按“像素”算的（比如宽 800、高 600）。怎么把 $[-1, 1]$ 的抽象坐标，对应到具体的像素位置？

【先看一个具体窗口】 假设窗口是 `glViewport(0,0,800,600)`，也就是窗口左下角在 (0,0)，宽 800 个像素，高 600 个像素。NDC 的 $x = -1$ 应该落在窗口最左边，也就是 $\text{WinX} = 0$ ； $x = 1$ 应该落在窗口最右边，也就是 $\text{WinX} = 800$ ； $x = 0$ 在中间，所以应当落在 $\text{WinX} = 400$ 。这个例子先告诉我们一件事：视口变换不是投影，也不是重新决定可不可见，它只是把已经规范化好的坐标换成窗口像素坐标。

【逐步推导水平坐标】 先只看水平方向。NDC 的范围是 $[-1, 1]$ ，窗口水平方向的范围是 $[O_x, O_x + \text{width}]$ 。第一步把 $[-1, 1]$ 变成 $[0, 1]$ ，做法是 $\frac{x+1}{2}$ ；第二步把 $[0, 1]$ 拉伸到窗口宽度，再加上左下角的水平偏移 O_x ，于是得到 $\text{WinX} = O_x + \frac{x+1}{2} \cdot \text{width}$ 。这个式子代回刚才的窗口就能检查： $x = 0$ 时 $\text{WinX} = 0 + \frac{1}{2} \times 800 = 400$ ，正好在屏幕中间。

【再看竖直坐标和深度】 竖直方向完全按同样的想法处理，只是把宽度换成高度、把水平偏移换成竖直偏移，所以 $\text{WinY} = O_y + \frac{y+1}{2} \cdot \text{height}$ 。深度 z 不表示屏幕上横竖位置，而是要存进 Z-buffer；Z-buffer 是深度缓冲区，意思是每个像素额外存一个深度值，用来判断谁挡住谁。OpenGL 这里把 $z \in [-1, 1]$ 也线性映射到 $[0, 1]$ ，所以 $\text{WinZ} = \frac{z+1}{2}$ ，其中 $z = -1$ 对应最近， $z = 1$ 对应最远。

【从例子收束成公式】 把上面的三条推导合在一起，视口变换的分量公式为

$$\begin{aligned}\text{WinX} &= O_x + \frac{x+1}{2} \cdot \text{width} \\ \text{WinY} &= O_y + \frac{y+1}{2} \cdot \text{height} \\ \text{WinZ} &= \frac{z+1}{2} \quad (\text{深度规范化到 } [0, 1])\end{aligned}$$

【再写成矩阵】 如果题目要求用矩阵乘法来写，就把这三条线性关系装进一个齐次矩阵。齐次矩阵的意思是把平移也塞进矩阵最后一列，这样缩放和平移可以一次相乘完成：

$$\begin{pmatrix} \text{WinX} \\ \text{WinY} \\ \text{WinZ} \\ 1 \end{pmatrix} = D \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad D = \begin{pmatrix} \frac{\text{width}}{2} & 0 & 0 & O_x + \frac{\text{width}}{2} \\ 0 & \frac{\text{height}}{2} & 0 & O_y + \frac{\text{height}}{2} \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

两个关键易错点：

1. 视口坐标原点在窗口**左下角**， y 向上增加；而窗口系统（如鼠标）原点在**左上角**， y 向下增加——两者朝向相反，求"第几行"（从上往下数）时需 $\text{row} = \text{height} - \text{WinY}$ 。
2. 若题目问"相对于窗口的行列"，不要加窗口左下角偏移 (O_x, O_y) ；只有求"全局屏幕坐标"时才加。

6.1 【例题】NDC 点映射到窗口像素行列

把视口矩阵、分量公式和 y 翻转串起来算一遍。

题目：窗口 `glViewport(0, 0, 800, 600)`（左下角在原点，宽 800、高 600）。某点规范化设备坐标 NDC 为 $(x, y, z) = (-0.5, 0.5, 0)$ 。求它的窗口像素坐标 $(\text{WinX}, \text{WinY})$ ，以及它在第几列、从上往下第几行。

写思路：先把 `glViewport` 的四个参数写进视口矩阵 D ，再用 D 左乘 NDC 齐次列向量得到窗口坐标。窗口坐标的 y 轴从下往上，而题目问"从上往下第几行"，所以最后还要做一次 $\text{row} = \text{height} - \text{WinY}$ 的方向转换。

写公式：把视口矩阵和行列换算公式先写出来。

$$D = \begin{pmatrix} \frac{\text{width}}{2} & 0 & 0 & O_x + \frac{\text{width}}{2} \\ 0 & \frac{\text{height}}{2} & 0 & O_y + \frac{\text{height}}{2} \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad P_w = D \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

得到窗口坐标后，再按窗口的列和从上往下的行进行换算：

$$\text{col} = \text{WinX}, \quad \text{row} = \text{height} - \text{WinY}$$

套数据计算：代入 $O_x = 0, O_y = 0, \text{width} = 800, \text{height} = 600$,

$$D = \begin{pmatrix} 400 & 0 & 0 & 400 \\ 0 & 300 & 0 & 300 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

接着直接做矩阵乘法，得到窗口坐标和深度：

$$D \begin{pmatrix} -0.5 \\ 0.5 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 400(-0.5) + 400 \\ 300(0.5) + 300 \\ \frac{1}{2} \cdot 0 + \frac{1}{2} \\ 1 \end{pmatrix} = \begin{pmatrix} 200 \\ 450 \\ 0.5 \\ 1 \end{pmatrix}$$

为了和上面的矩阵结果互相检查，也可以把同一件事展开成分量公式：

$$\text{WinX} = 0 + \frac{-0.5 + 1}{2} \times 800 = \frac{0.5}{2} \times 800 = 0.25 \times 800 = 200$$

$$\text{WinY} = 0 + \frac{0.5 + 1}{2} \times 600 = \frac{1.5}{2} \times 600 = 0.75 \times 600 = 450$$

$$\text{WinZ} = \frac{0 + 1}{2} = 0.5$$

列（从左数）为 $\text{col} = \text{WinX} = 200$ ；行（从上往下数，注意翻转）为 $\text{row} = \text{height} - \text{WinY} = 600 - 450 = 150$ 。

直觉验证：NDC 的 $x = -0.5$ 偏左，算出 $\text{WinX}=200$ （在 800 宽的左侧四分之一处）✓； $y = 0.5$ 偏上，从下数 $\text{WinY}=450$ （偏上），从上数第 150 行（靠上）✓；深度 $z = 0$ 居中， $\text{WinZ}=0.5$ ✓。

7.【综合例题】完整顶点变换流水线

这是第一大题最典型的综合大题：一个点经过平移 → 旋转 → 相机变换 → 投影 → 视口，算出它在窗口中的位置。下面把第 2-6 节的工具串起来。

题目：点 $P = (1, 2, 3)$ 依次经过：① 平移 $(2, 2, 5)$ ；② 绕平行于 z 轴、过 $(1, 1, 0)$ 的轴逆时针转 30° ；相机 `gluLookAt(1,0,0, 1,0,-1, 0,1,0)`。先求复合模型矩阵 C 、模型视图矩阵 M 、点在眼睛坐标系下的坐标 P_e 。为了把后半条流水线也走完整，题目再追加给定投影 `glOrtho(-2,2,-2,2,1,9)` 和窗口 `glViewport(0,0,400,300)`，要求判断点是否可见；若点不可见，再另取一个可见的眼睛空间点 P'_e

$= (1, -1, -3)$ 演示它如何落到窗口像素行列。

写思路：完整流水线必须按矩阵链路写清楚。先把局部点写成齐次列向量，按"先平移、再旋转"得到模型矩阵 $C = RT$ ；再由相机矩阵得到模型视图矩阵 $M = VC$ ；最后用 $P_e = MP$ 算眼睛空间坐标。若继续问屏幕像素，就在 P_e 后面接投影矩阵 N 和视口矩阵 D ，即 $P_{ndc} = NP_e$ 、 $P_w = DP_{ndc}$ 。

写公式：先写出完整流水线里模型、视图、投影、视口的矩阵链路。

$$P = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad C = RT, \quad M = VC, \quad P_e = MP$$

若题目继续要求投影与窗口坐标，则使用

$$P_c = NP_e, \quad P_{ndc} = \left(\frac{x_c}{w_c}, \frac{y_c}{w_c}, \frac{z_c}{w_c} \right), \quad P_w = D \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix}$$

套数据计算：下面依次把题目中的平移、旋转、相机和点代入矩阵链路。

(1) 平移矩阵

$$T = T(2, 2, 5) = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(2) 旋转矩阵 R

旋转轴平行于 z 轴，并且经过点 $p_f = (1, 1, 0)$ 。为了绕这条轴旋转，先把 p_f 平移到原点，再绕 z 轴转 30° ，最后把 p_f 平移回去，所以

$$R = T(1, 1, 0)R_z(30^\circ)T(-1, -1, 0)$$

令 $c = \cos 30^\circ = \frac{\sqrt{3}}{2}$, $s = \sin 30^\circ = \frac{1}{2}$ ，中间的旋转和平移矩阵为

$$R_z(30^\circ) = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad T(-1, -1, 0) = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

两边平移夹住旋转后得到

$$R = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{3}{2} - \frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} - \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(3) 复合模型矩阵 $C = RT$

先平移再旋转，因此 $C = RT$ ，平移在右先作用。线性部分仍是 $R_z(30^\circ)$ ，新平移列 = $R_{\text{线性}} \cdot (2, 2, 5)^T + R_{\text{平移列}}$ ：

$$\begin{aligned} t_x &= \left(\frac{\sqrt{3}}{2} \cdot 2 - \frac{1}{2} \cdot 2\right) + \left(\frac{3}{2} - \frac{\sqrt{3}}{2}\right) = \frac{1}{2} + \frac{\sqrt{3}}{2} \\ t_y &= \left(\frac{1}{2} \cdot 2 + \frac{\sqrt{3}}{2} \cdot 2\right) + \left(\frac{1}{2} - \frac{\sqrt{3}}{2}\right) = \frac{3}{2} + \frac{\sqrt{3}}{2} \\ t_z &= 5 \end{aligned}$$

$$C = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{1}{2} + \frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{3}{2} + \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(4) 视图矩阵 V

相机参数为 $E = eye = (1, 0, 0)$ ， $at = (1, 0, -1)$ ， $up = (0, 1, 0)$ 。先直接计算相机三根轴：

$$w = \frac{eye - at}{\|eye - at\|} = \frac{(1, 0, 0) - (1, 0, -1)}{\|(0, 0, 1)\|} = (0, 0, 1)$$

$$u = \widehat{up \times w} = (0, 1, 0) \times (0, 0, 1) = (1, 0, 0), \quad v = w \times u = (0, 0, 1) \times (1, 0, 0) = (0, 1, 0)$$

再算视图矩阵的平移列：

$$-u \cdot E = -1, \quad -v \cdot E = 0, \quad -w \cdot E = 0$$

因此视图矩阵为

$$V = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(5) 模型视图矩阵 $M = VC$

模型视图矩阵的含义在 1.1 已经先讲过：它是把"模型变换 C "和"相机变换 V "合成一步。这里按同样规则实际算一次。因为列向量右乘时右边先作用，所以点先经过模型矩阵 C 变到世界坐标，再经过视图矩阵 V 变到眼睛坐标，总矩阵就是

$$M = VC$$

这里的

$$V = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

表示"整个世界沿 x 方向减 1"，也就是把相机从 $(1, 0, 0)$ 搬回原点。这里不要用口头简化去猜结果，直接把 V 和 C 写出来做矩阵乘法：

$$VC = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{1}{2} + \frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{3}{2} + \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

把结果矩阵的最后一列按行乘列算清楚：

$$\begin{aligned} m_{14} &= 1 \cdot \left(\frac{1}{2} + \frac{\sqrt{3}}{2} \right) + 0 \cdot \left(\frac{3}{2} + \frac{\sqrt{3}}{2} \right) + 0 \cdot 5 + (-1) \cdot 1 = \frac{\sqrt{3}}{2} - \frac{1}{2} \\ m_{24} &= 0 \cdot \left(\frac{1}{2} + \frac{\sqrt{3}}{2} \right) + 1 \cdot \left(\frac{3}{2} + \frac{\sqrt{3}}{2} \right) + 0 \cdot 5 + 0 \cdot 1 = \frac{3}{2} + \frac{\sqrt{3}}{2} \\ m_{34} &= 0 \cdot \left(\frac{1}{2} + \frac{\sqrt{3}}{2} \right) + 0 \cdot \left(\frac{3}{2} + \frac{\sqrt{3}}{2} \right) + 1 \cdot 5 + 0 \cdot 1 = 5 \end{aligned}$$

其余元素也按同样的行乘列规则计算，得到

$$M = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} - \frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{3}{2} + \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(6) 点在眼睛坐标系下的坐标 $P_e = M P$

前面 1.1 已经说明，合成出模型视图矩阵 M 后，还要用它去乘原始点 P ，才能得到眼睛坐标。这里的点 $P = (1, 2, 3)$ 要先写成齐次列向量：

$$P = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 1 \end{pmatrix}$$

于是

$$P_e = M P = \begin{pmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} - \frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} & 0 & \frac{3}{2} + \frac{\sqrt{3}}{2} \\ 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 1 \end{pmatrix}$$

矩阵乘点时，就是 M 的每一行分别和这个点列向量做点乘：

$$\begin{aligned} x_e &= \frac{\sqrt{3}}{2} \cdot 1 - \frac{1}{2} \cdot 2 + \left(\frac{\sqrt{3}}{2} - \frac{1}{2}\right) = \sqrt{3} - \frac{3}{2} \\ y_e &= \frac{1}{2} \cdot 1 + \frac{\sqrt{3}}{2} \cdot 2 + \left(\frac{3}{2} + \frac{\sqrt{3}}{2}\right) = 2 + \frac{3\sqrt{3}}{2} \\ z_e &= 1 \cdot 3 + 5 = 8 \end{aligned}$$

$$P_e = \left(\sqrt{3} - \frac{3}{2}, 2 + \frac{3\sqrt{3}}{2}, 8 \right)$$

(7) 投影：对 P_e 做正交规范化，判断是否可见

这里使用的是题目追加给定的投影 `glOrtho(-2,2,-2,2,1,9)`，也就是 $l = -2, r = 2, b = -2, t = 2, n = 1, f = 9$ 。它是为了继续演示“眼睛坐标 $\rightarrow$ NDC $\rightarrow$ 裁剪”这一步额外给出的条件。正交投影的规范化矩阵一般式为

$$N_{ortho} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

现在逐项代入这道题的数值：

$$\frac{2}{r-l} = \frac{2}{2-(-2)} = \frac{1}{2}, \quad -\frac{r+l}{r-l} = -\frac{2+(-2)}{4} = 0$$

$$\frac{2}{t-b} = \frac{2}{2-(-2)} = \frac{1}{2}, \quad -\frac{t+b}{t-b} = -\frac{2+(-2)}{4} = 0$$

$$-\frac{2}{f-n} = -\frac{2}{9-1} = -\frac{1}{4}, \quad -\frac{f+n}{f-n} = -\frac{9+1}{8} = -\frac{5}{4}$$

所以本题的正交规范化矩阵为

$$N_{ortho} = \begin{pmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{4} & -\frac{5}{4} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

矩阵展开后的分量公式为

$$x_n = \frac{x_e}{2}, \quad y_n = \frac{y_e}{2}, \quad z_n = -\frac{z_e}{4} - \frac{5}{4}$$

代入 $P_e = (\sqrt{3} - \frac{3}{2}, 2 + \frac{3\sqrt{3}}{2}, 8) \approx (0.23, 4.60, 8)$:

$$x_n \approx 0.12 \checkmark, \quad y_n \approx 2.30 (> 1, \text{超出}), \quad z_n = -\frac{8}{4} - \frac{5}{4} = -3.25 (< -1, \text{超出})$$

结论: $y_n > 1$ 且 $z_n < -1 \rightarrow$ 该点落在视景体外, 被裁剪、不显示。其中 $z_e = 8 > 0$ 意味着这点其实在相机背后 (前方应为负 z), $z_n = -3.25$ 越过了近平面——所以这套相机 / 投影下根本看不到它。这本身是个有意义的结论: 不是所有点都会出现在屏幕上。

(8)(9) 完整链路演示: 取一个可见点走到像素

为把"投影 $\rightarrow$ 视口 $\rightarrow$ 像素行列"整条链路算完整, 再单独取一个落在视景体内的眼睛空间点 $P'_e = (1, -1, -3)$, 投影仍明确写作 `glOrtho(-2,2,-2,2,1,9)`, 窗口明确写作 `glViewport(0,0,400,300)`:

写思路: 这一段只演示后半条链路, 但仍然要按流水线顺序来。先用 (7) 已经推出来的正交规范化矩阵 N_{ortho} 乘眼睛空间点 P'_e , 得到 NDC; 再判断 NDC 三个分量是否都在 $[-1, 1]$ 内; 如果可见, 再用 `glViewport(0,0,400,300)` 推出视口矩阵 D , 最后用 D 乘 NDC 点得到窗口坐标, 并把窗口坐标换成从左数的列和从上往下数的行。

写公式: 后半条链路仍然是矩阵乘法, 不直接跳分量公式。

$$P_{ndc} = N_{ortho} P'_e, \quad P_w = D \begin{pmatrix} x_n \\ y_n \\ z_n \\ 1 \end{pmatrix}, \quad \text{col} = \text{WinX}, \quad \text{row} = \text{height} - \text{WinY}$$

套数据计算第一步: 用投影矩阵求 NDC。 由 (7) 已经逐项代入得到

$$N_{ortho} = \begin{pmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{4} & -\frac{5}{4} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

把 $P'_e = (1, -1, -3)$ 写成齐次列向量：

$$P'_e = \begin{pmatrix} 1 \\ -1 \\ -3 \\ 1 \end{pmatrix}$$

于是

$$P_{ndc} = N_{ortho}P'_e = \begin{pmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{4} & -\frac{5}{4} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \\ -3 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ \frac{3}{4} - \frac{5}{4} \\ 1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.5 \\ -0.5 \\ 1 \end{pmatrix}$$

正交投影这里 $w = 1$ ，所以不用再除以 w ，NDC 就是 $(0.5, -0.5, -0.5)$ 。三个分量都在 $[-1, 1]$ 内，因此这个点可见，可以继续做视口变换。

套数据计算第二步：由 `glViewport` 推出视口矩阵。 题目给的是 `glViewport(0,0,400,300)`，所以 $O_x = 0, O_y = 0, width = 400, height = 300$ 。代入第 6 节推出的视口矩阵：

$$D = \begin{pmatrix} \frac{width}{2} & 0 & 0 & O_x + \frac{width}{2} \\ 0 & \frac{height}{2} & 0 & O_y + \frac{height}{2} \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 200 & 0 & 0 & 200 \\ 0 & 150 & 0 & 150 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

接着把 NDC 点写成齐次列向量并左乘 D ：

$$P_w = D \begin{pmatrix} 0.5 \\ -0.5 \\ -0.5 \\ 1 \end{pmatrix} = \begin{pmatrix} 200(0.5) + 200 \\ 150(-0.5) + 150 \\ \frac{1}{2}(-0.5) + \frac{1}{2} \\ 1 \end{pmatrix} = \begin{pmatrix} 300 \\ 75 \\ 0.25 \\ 1 \end{pmatrix}$$

所以窗口坐标是 $WinX = 300, WinY = 75, WinZ = 0.25$ 。

套数据计算第三步：把窗口坐标换成行列。 题目问列时，从左往右数，所以列号直接用 $WinX$ ：

$$col = WinX = 300$$

题目问行时，通常是从上往下数；但 OpenGL 的窗口坐标 $WinY$ 是从下往上数，所以要翻转：

$$row = height - WinY = 300 - 75 = 225$$

所以 P'_e 最终落在窗口第 300 列、第 225 行，深度 0.25。

答题要点（这类综合大题的标准流程）：① 写出并相乘模型矩阵 C 、视图矩阵 V 得 $M = VC$ ；② 算 $P_e = MP$ ；③ 用投影公式得 NDC、**逐分量判断是否在 $[-1, 1]$ 内**（决定是否被裁剪）；④ 视图公式得像素 WinX/WinY，并用 $row = height - WinY$ 做 y 翻转得行号。每步先写公式再代数。

8. Phong / Blinn 光照模型

算完顶点的**位置**后，还要算它的**颜色**——这就是着色。

8.1 四种光源与三类表面

【先从房间里的灯想起】 如果房间中央挂着一个灯泡，它有明确的位置，离灯越近越亮、越远越暗，这种灯就叫**点光源**（Point light，意思是把灯近似看成一个发光点）。假设灯本身强度是 I_0 ，被照点到灯的距离是 d ，光走得越远，会摊到越大的球面上；球面面积和 d^2 成正比，所以单位面积收到的光大约按 $1/d^2$ 变小：

$$I = \frac{I_0}{d^2}$$

这里 I_0 是光源本身强度， d 是光源到被照点的距离。实际渲染里常用更柔和的衰减 $\frac{1}{a + bd + cd^2}$ ，其中 a, b, c 是人为调节的常数，用来避免距离很小时亮度突然过大。

如果把灯泡装进手电筒里，光不再向四面八方发散，而是集中在一个锥形范围内，这就是**聚光灯**（Spotlight，意思是有方向和照射范围的光源）。先想一个具体画面：手电筒正对着桌面照，光斑中心最亮，越靠边越暗，出了光斑范围就基本没有光。计算机要模拟这个现象，就要回答两个问题：第一，被照点有没有落在手电筒的光锥里面；第二，如果在光锥里面，它离光束中心有多近。

设 s 是手电筒指向的中心方向， l 是从光源指向被照点的方向，二者都取单位向量。这里 s 像"手电筒正中间那条轴"， l 像"从灯泡射到这个点的那条线"。如果 l 和 s 完全重合，说明点就在光斑中心；如果 l 和 s 夹角变大，说明点越来越靠近光斑边缘。设夹角为 ϕ ，那么单位向量点乘满足

$$s \cdot l = \|s\| \|l\| \cos \phi = \cos \phi$$

因为 s, l 都是单位向量，长度都是 1，所以点乘就直接等于 $\cos \phi$ 。当 $\phi = 0^\circ$ 时， $s \cdot l = 1$ ，表示点在光束中心；当 ϕ 变大时， $\cos \phi$ 变小，亮度就应该下降；如果 ϕ 大到超过聚光灯允许的锥角，说明点已经跑出手电筒照射范围，强度直接按 0 处理。

只用 $\cos \phi$ 会让光斑边缘下降得比较平缓。为了控制"中心亮、边缘暗"的程度，聚光灯会再加一个指数 e ：

$$I \propto (\cos \phi)^e = (s \cdot l)^e$$

这里 $\propto$ 表示"正比于"，意思是亮度还可以再乘光源强度、距离衰减等其它因子，但方向上的亮暗变化由 $(s \cdot l)^e$ 控制。指数 e 越大，稍微偏离中心就会暗得更快，光斑越集中； e 越小，边缘衰减越慢，光斑看起来更散。举个数值感受：如果 $s \cdot l = 0.8$ ，当 $e = 1$ 时方向因子是 0.8；当 $e = 4$ 时变成 $0.8^4 = 0.4096$ ；当 $e = 10$ 时变成 $0.8^{10} \approx 0.107$ ，也就是同样偏离中心，指数越大越暗。

还有一种常见情况是太阳光。太阳离场景极远，在一个小场景里可以近似认为所有光线都平行，这叫**无穷远光源**或**方向光**（Directional light，意思是只关心方向，不关心光源位置）。方向光的光照方向 l 在场景里处处相同，强度也不随距离变化，所以计算最省。

最后还有**环境光**（Ambient light，意思是没有明确方向、到处都有的底光）。它用来模拟光在墙壁、地面和物体之间反弹很多次之后形成的间接照明；如果没有环境光，没被灯直接照到的地方会变成完全黑色，画面通常会显得很假。

【再看三类常见表面，它们决定光照怎么反应】

表面也要分类型，因为同一束光照到不同材料上会有不同反应。**镜面**（Specular，也就是光滑反射面）像镜子或抛光金属，会产生明显高光；**漫反射面**（Diffuse，也叫 Lambertian 面，指粗糙表面把光向各个方向散开）像粉墙或纸，没有明显高光，亮度主要由入射角决定；**透明面**（Translucent，指光能穿过材料）像玻璃或水，光穿过时会折射，也就是方向发生弯折，这部分由 8.4 的 Snell 定律描述。

大多数真实物体是这几类的混合，所以 Phong 模型才把"漫反射 + 镜面 + 环境"三项加在一起（下一节）。

8.2 Phong 光照模型（局部光照，可快速计算）

【先假设你不懂】一个物体表面某点该有多亮、什么颜色？Phong 模型不直接凭感觉给颜色，而是把光照拆成三块相加：**环境光**（背景底光）+ **漫反射**（物体本色）+ **镜面反射**（高光亮点）。这叫**局部光照模型**，意思是只看这个点、这盏灯、相机和表面法向，不去追踪光在场景里反弹很多次。

先把后面公式里会出现的符号一次讲清楚：

- I ：最后算出来的亮度 / 颜色强度。若是彩色，就对 R、G、B 三个通道各算一次。
- L_a, L_d, L_s ：光源给出的三类光强，分别对应环境光、漫反射光、镜面光。
- k_a, k_d, k_s ：物体材质对三类光的反射系数。比如红色橡胶 k_d 大、 k_s 小；金属 k_s 大。
- n ：被照点处的**表面法向**。它不是坐标轴，也不是一个点，而是一根从表面竖起来的小箭头，用来表示"这个位置的表面朝哪边"。比如水平桌面朝上，法向可以写成 $n = (0, 1, 0)$ ；若表面正朝屏幕外，法向可能是 $n = (0, 0, 1)$ 。它必须是单位向量，因为后面要用 $l \cdot n$ 直接表示光线和表面朝向的夹角余弦。三角形网格里，面法向通常由两条边叉乘得到；顶点法向通常由周围几个面法向平均得到。
- l ：从被照点指向光源的单位向量。
- v ：从被照点指向眼睛 / 相机的单位向量。
- r ：光线按镜面反射以后，理论上应该反射出去的单位方向。
- α ：高光指数，控制高光是宽还是尖。

注意这里 l, n, v, r 都默认已经单位化，这样点乘才可以直接当作夹角余弦来用，例如 $l \cdot n = \cos \theta$ 。

【第一项：漫反射 Diffuse —— 物体的"本色"】

先看例子：拿手电筒照墙。**正对着照**（光线垂直墙面）最亮；**斜着照**，同样多的光摊在更大一片墙上，每块地方就变暗；几乎**贴着墙面照**（掠射）时几乎不亮。所以亮度取决于"光线和墙面法向的夹角 θ_i "——夹角越小越亮。

把图形语言翻译成公式：法向 n 是墙面"正前方"，入射光方向 l 是光从哪里照过来。两者夹角记为 θ_i ，这个下标 i 表示 incident，入射。朗伯定律说：漫反射亮度正比于 $\cos \theta_i$ 。又因为 l, n 都是单位向量，由点乘定义（预备 0.14）直接有 $\cos \theta_i = l \cdot n$ 。所以漫反射先写成

$$I_{\text{diffuse}} = k_d L_d (l \cdot n)$$

其中 L_d 是这盏灯的漫反射光强， k_d 是物体吃进这种光以后反出来多少。**为什么要 $\max(\cdot, 0)$ ：**如果光在表面**背面**， $l \cdot n$ 算出来是负数，意味着"负的亮度"，没意义。所以真正使用时写成

$$I_{\text{diffuse}} = k_d L_d \max(l \cdot n, 0)$$

漫反射还有一个重要特点：它**与视点无关**。粗糙墙面从左看、从右看都差不多，因为光被向各个方向均匀散开。

数字感受：设 $k_d L_d = 1$ 。光正对法向（ $\theta_i = 0^\circ$ ， $l \cdot n = \cos 0^\circ = 1$ ） $\rightarrow$ 漫反射 = 1，最亮；光与法向成 60° （ $l \cdot n = \cos 60^\circ = 0.5$ ） $\rightarrow$ 只有 0.5，半亮；成 90° （掠射， $l \cdot n = 0$ ） $\rightarrow 0$ ，全黑。亮度随夹角余弦平滑下降。

【第二项：镜面反射 Specular —— 高光亮点】

先看例子：想象一盏台灯照在一个光滑苹果上。我们只盯着苹果表面的某一个小点看，这个点上有三根方向箭头：

- l ：从这个点指向灯的位置，表示光从哪个方向照来。
- n ：这个点表面竖起来的法向，表示表面朝哪边。
- v ：从这个点指向你的眼睛，表示你从哪个方向看它。

如果苹果表面很光滑，它就有点像小镜子。光照到这个小点后，不会像粗糙墙面那样均匀散开，而是主要沿着一个"镜子反弹出去的方向"走，这个方向就是**理想反射方向** r 。当你的眼睛方向 v 正好落在 r 附近时，那束反弹光刚好进眼睛，你就看到一块很亮的白色高光；你把头挪开， v 变了，不再对着 r ，进眼睛的反射光变少，高光就变暗，甚至换到苹果表面的别的位置出现。

所以镜面反射看的是：**眼睛方向 v 和理想反射方向 r 有多接近**。越接近，高光越亮；偏得越远，高光越暗。

把图形语言翻译成公式：先求理想反射方向 r ，再看视线方向 v 和 r 有多接近。两者夹角记为 ϕ ，由于 r, v 是单位向量，所以 $r \cdot v = \cos \phi$ 。当 v 正好等于 r 时，点乘是 1，高光最亮；当 v 偏远时，点乘变小，高光变暗。再把它**乘方 α 次**，让高光衰减得很快：

$$I_{\text{specular}} = k_s L_s \max(r \cdot v, 0)^\alpha$$

这里 L_s 是镜面光强, k_s 是材质的镜面反射系数, $\max(r \cdot v, 0)$ 仍然是为了防止负亮度。**高光指数 α 的含义**: α 越大, 高光越小越锐、物体越"亮锃锃"。 $\alpha \in [5, 10]$ 像塑料; $\alpha \in [100, 200]$ 像金属; $\alpha \rightarrow \infty$ 趋于完美镜子。

数字感受: 设视线略偏离理想反射方向, $r \cdot v = 0.9$ 。则 $\alpha = 1$ 时高光 = 0.9 (大而散); $\alpha = 20$ 时 $0.9^{20} \approx 0.12$ (明显收紧); $\alpha = 100$ 时 $0.9^{100} \approx 2.7 \times 10^{-5}$ (几乎只剩正对反射方向那一点最亮)。这就是为什么大 α 给出又小又锐的金属高光。

理想反射方向 r 怎么来: 先注意这里的 l 不是"光线前进方向", 而是**从被照点指向光源的方向**。也就是说, 如果灯在表面上方, l 就是从表面点往灯那边指的小箭头。Phong 公式要算的是: 如果光按镜面规律反射, 反射出去的方向 r 应该指向哪里。

把 l 拆成两部分来看会清楚很多:

1. 沿法向 n 的部分: 这是 l 在 n 方向上的投影。因为 n 是单位向量, 所以投影长度是 $l \cdot n$, 投影向量就是

$$l_{\parallel} = (l \cdot n)n$$

2. 贴着表面的部分: 也就是剩下那部分

$$l_{\perp} = l - l_{\parallel}$$

镜面反射的几何规律是: 反射前后关于法向 n 对称。用这两个分量说, 就是**沿法向的分量不变, 贴着表面的分量反向**。所以反射方向为

$$r = l_{\parallel} - l_{\perp}$$

把 $l_{\perp} = l - l_{\parallel}$ 代进去:

$$r = l_{\parallel} - (l - l_{\parallel}) = 2l_{\parallel} - l$$

再把 $l_{\parallel} = (l \cdot n)n$ 代进去, 就得到

$$r = 2(l \cdot n)n - l$$

数字验证一下: 设表面法向 $n = (0, 0, 1)$, 光从右上方照来, $l = (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2})$ 。先算

$$l \cdot n = \frac{\sqrt{2}}{2}$$

所以

$$r = 2 \cdot \frac{\sqrt{2}}{2} (0, 0, 1) - (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2}) = (-\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2})$$

结果的 z 分量仍然向上, 说明反射光还在表面外侧; x 分量从正变负, 说明它从右上方照来后, 被镜面弹向左上方, 符合"入射角 = 反射角"的直觉。

【第三项：环境光 Ambient —— 背景底光】

先看例子：现实里光会在墙壁、地面之间**反弹很多次**，所以即使没被灯直接照到的角落也不是漆黑。但逐次追踪每条反弹光太贵。Phong 偷懒：给每个点都加一个**恒定的底光**，模拟这些四处乱窜的间接光。

$$I_{\text{ambient}} = k_a L_a \quad (\text{一个常数, 处处相同})$$

没有它会怎样：阴影里会是纯黑死黑，很假。加上它，暗部也有细节。

【三项合起来（单光源完整公式）】

$$I = \underbrace{k_a L_a}_{\text{环境(底光)}} + \underbrace{k_d L_d \max(l \cdot n, 0)}_{\text{漫反射(本色)}} + \underbrace{k_s L_s \max(r \cdot v, 0)^\alpha}_{\text{镜面(高光)}}$$

- 对 R、G、B 三个颜色通道各算一遍（所以有 9 个光照系数 + 9 个反射系数）。
- **多光源**：每个光源单独算一份，结果直接相加。
- 可乘距离衰减因子 $\frac{1}{a + bd + cd^2}$ （ d 是光源到点的距离）柔化点光源。

8.3 Blinn 模型（Phong 的改进，OpenGL 实际用的版本）

【先假设你不懂】Blinn 模型只改 Phong 里面的**镜面反射项**，环境光和漫反射不变。Phong 的镜面项要先算理想反射方向 $r = 2(l \cdot n)n - l$ ，再算 $r \cdot v$ ；Blinn 换了一个更好算的角度：不问“视线是否接近反射方向”，而是问“法向是否正好位于光线和视线的中间”。

【半程向量 h 是什么】站在被照点上，一只手指向**光源**（方向 l ），另一只手指向**眼睛**（方向 v ）。这两个方向的“正中间那条线”，就是**半程向量 h** 。这里 half 的意思不是长度一半，而是**夹角的一半**，也就是角平分方向。

为什么 $l + v$ 能给出中间方向？因为 l 和 v 都是单位向量，把它们首尾放在同一个起点，相加以后得到的向量会指向二者夹角的中间。相加后的长度不一定是 1，所以还要除以自己的长度 $\|l + v\|$ ，这一步叫**归一化**：

$$h = \frac{l + v}{\|l + v\|}$$

【为什么用 $n \cdot h$ 代替 $r \cdot v$ 】想一想最亮的高光出现在什么时候：表面法向 n 正好摆到一个位置，使得光线按镜面反射以后刚好进眼睛。镜面反射有一个几何性质：在最亮的位置，法向 n 正好平分入射方向 l 和观察方向 v 的夹角。也就是说， n 越接近半程向量 h ，这个点越像“正好把光反进眼睛”。

所以 Phong 用 $r \cdot v$ 判断“视线离反射方向近不近”，Blinn 用 $n \cdot h$ 判断“法向离半程方向近不近”。二者表达的是同一个高光直觉，只是计算路径不同。镜面项变成：

$$I_{\text{spec}} = k_s L_s \max(n \cdot h, 0)^\alpha$$

这个式子里每个符号的意思是： k_s 是材质有多容易出高光， L_s 是光源的镜面光强， $n \cdot h$ 是法向与半程方向的接近程度， α 仍然控制高光尖锐程度。

【为什么更省】算 $h = l + v$ 再归一化，比算反射向量 $r = 2(l \cdot n)n - l$ 少几步；更重要的是，当光源在无穷远（ l 是常量）、视点也很远（ v 近似常量）时， h 整个画面几乎是**同一个常量**，可以一次算好反复用，省下大量计算。

【数字示例（Blinn 与 Phong 对比同一情形）】设法向 $n = (0, 0, 1)$ 、光 $l = (0, 0, 1)$ （正上方射来）、视线 $v = (0, 0, 1)$ （正上方看）。

- Phong：反射方向 $r = 2(l \cdot n)n - l = 2(1)(0, 0, 1) - (0, 0, 1) = (0, 0, 1)$ ，则 $r \cdot v = 1 \rightarrow$ 镜面满值。
 - Blinn： $h = \frac{l + v}{\|l + v\|} = \frac{(0, 0, 2)}{2} = (0, 0, 1)$ ，则 $n \cdot h = 1 \rightarrow$ 同样满值。
- 两者一致 $\checkmark$ 。一般情形下 Blinn 的高光会比 Phong 略宽，常通过把 α 取大一些（约 Phong 的 4 倍）来匹配。

OpenGL 固定管线实际用的就是这个 Blinn（Blinn-Phong）版本。

8.4 折射（Snell 定律）

先看一个生活例子：把筷子斜插进水里，从水面上看，筷子像是折了一下。原因是光从空气进入水时速度变了，传播方向也跟着变了，这种现象叫**折射**。空气、水、玻璃这类光可以穿过的材料叫**介质**。

公式出现前先把符号讲清楚：

- η ：折射率，用来描述光在某种介质里传播得有多慢。 η 越大，光在里面越慢。空气约 1，水约 1.33，玻璃约 1.5。
- 下标 i ：incident，入射侧，也就是光线原来所在的介质。
- 下标 t ：transmitted，透射侧，也就是光线进入的新介质。
- θ_i ：入射光线和法线的夹角。
- θ_t ：折射光线和法线的夹角。
- 法线 n ：垂直于界面的参考方向。注意角度永远是光线和**法线**的夹角，不是和水面 / 玻璃面的夹角。

Snell 定律说：光穿过两种介质的交界面时，入射侧的“折射率 $\times$ 入射角正弦”等于透射侧的“折射率 $\times$ 折射角正弦”：

$$\eta_i \sin \theta_i = \eta_t \sin \theta_t$$

把它改写成考试常用的“求折射角”形式：

$$\sin \theta_t = \frac{\eta_i}{\eta_t} \sin \theta_i$$

【怎么读这个式子】从空气进玻璃时， $\eta_i = 1, \eta_t = 1.5$ ，所以 $\frac{\eta_i}{\eta_t} < 1$ ，折射角的正弦变小，光线会**向法线靠近**；从玻璃回空气时， $\frac{\eta_i}{\eta_t} > 1$ ，折射角会变大，光线会**远离法线**。

【数字感受】光从空气进入玻璃，入射角 $\theta_i = 30^\circ$ ：

$$\sin \theta_t = \frac{1}{1.5} \sin 30^\circ = \frac{1}{1.5} \times 0.5 = \frac{1}{3}$$

所以 $\theta_t \approx 19.5^\circ$ 。角度从 30° 变小到约 19.5° ，说明光线进入折射率更大的玻璃后，确实向法线弯过去。

【全反射为什么会出现】从光密介质到光疏介质时，比如从玻璃到空气， $\frac{n_i}{n_t}$ 可能大于 1。若入射角太大，算出来的 $\sin \theta_t > 1$ ，但任何真实角度的正弦都不可能超过 1，于是折射光线不存在，光全部反射回原介质，这叫**全反射**。水下看水面像镜子，就是这个原因。

8.5 【例题 3】完整光照计算（一般化，串起整个第 8 节）

这道题用一个**斜射的光、斜看的视线**（不再是全部正对），把 8.1~8.3 的点光源衰减、漫反射、镜面（Phong 和 Blinn 两种算法）全部走一遍。

题目：被照点法向 $n = (0, 0, 1)$ 。点光源在该点的**正上偏右** 45° 方向，即入射光方向 $l = (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2})$ （单位向量）；视线方向 $v = (0, 0, 1)$ （正上方看）。系数 $k_a = 0.2, L_a = 1$ ； $k_d = 0.8, L_d = 1$ ； $k_s = 0.6, L_s = 1, \alpha = 2$ 。先不计距离衰减。

写思路：先算环境光，再用 $l \cdot n$ 算漫反射，接着分别用 Phong 的反射向量和 Blinn 的半程向量算镜面项，最后把环境、漫反射、镜面三项相加。这里先用 Phong 镜面项做总亮度，Blinn 作为对照。

写公式：把 Phong 和 Blinn 两种光照计算中要用的向量公式与亮度公式统一列出。

$$I_a = k_a L_a, \quad I_d = k_d L_d \max(l \cdot n, 0)$$

Phong 镜面反射要先求理想反射方向，它使用

$$r = 2(l \cdot n)n - l, \quad I_s^{\text{Phong}} = k_s L_s \max(r \cdot v, 0)^\alpha$$

Blinn 镜面反射不直接求反射向量，而是使用半程向量：

$$h = \frac{l + v}{\|l + v\|}, \quad I_s^{\text{Blinn}} = k_s L_s \max(n \cdot h, 0)^\alpha$$

最后把环境光、漫反射和镜面反射相加，总亮度为

$$I = I_a + I_d + I_s$$

套数据计算：先算环境光（恒定底光）。

$$I_a = k_a L_a = 0.2 \times 1 = 0.2$$

再算漫反射 ($\propto l \cdot n$)。

$$l \cdot n = \frac{\sqrt{2}}{2} \cdot 0 + 0 \cdot 0 + \frac{\sqrt{2}}{2} \cdot 1 = \frac{\sqrt{2}}{2} \approx 0.707$$

$$I_d = k_d L_d \max(l \cdot n, 0) = 0.8 \times 1 \times 0.707 \approx 0.566$$

(光斜射 45° ，比正照暗一些，符合直觉。)

接着算镜面反射的 Phong 算法。

先求理想反射方向 $r = 2(l \cdot n)n - l$:

$$r = 2 \cdot \frac{\sqrt{2}}{2} (0, 0, 1) - (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2}) = (0, 0, \sqrt{2}) - (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2}) = (-\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2})$$

得到反射方向后，再计算它和视线方向的点乘 $r \cdot v$:

$$r \cdot v = (-\frac{\sqrt{2}}{2})(0) + 0 + (\frac{\sqrt{2}}{2})(1) = \frac{\sqrt{2}}{2} \approx 0.707$$

$$I_s^{\text{Phong}} = k_s L_s \max(r \cdot v, 0)^\alpha = 0.6 \times 1 \times (0.707)^2 = 0.6 \times 0.5 = 0.3$$

再算镜面反射的 Blinn 算法作为对照。

半程向量 $h = \frac{l + v}{\|l + v\|}$:

$$l + v = (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2}) + (0, 0, 1) = (\frac{\sqrt{2}}{2}, 0, \frac{\sqrt{2}}{2} + 1)$$

$$\|l + v\| = \sqrt{\frac{1}{2} + (\frac{\sqrt{2}}{2} + 1)^2} = \sqrt{\frac{1}{2} + (\frac{1}{2} + \sqrt{2} + 1)} = \sqrt{2 + \sqrt{2}} \approx 1.848$$

$$n \cdot h = \frac{(\frac{\sqrt{2}}{2} + 1)}{1.848} \approx \frac{1.707}{1.848} \approx 0.924$$

$$I_s^{\text{Blinn}} = 0.6 \times (0.924)^2 \approx 0.6 \times 0.854 \approx 0.512$$

(同一情形 Blinn 高光比 Phong 略大，正是 8.3 说的"Blinn 高光偏宽，需把 α 调大才匹配".)

最后合计。若采用 Phong 镜面项，则

$$I = I_a + I_d + I_s = 0.2 + 0.566 + 0.3 = 1.066 \rightarrow \text{截断到 } 1.0$$

若加距离衰减：设光源距被照点 $d = 2$ ，点光源用 $1/d^2$ ，则漫反射与镜面项再乘 $1/4$ （环境光不衰减）。

若是彩色：把 k_d, L_d 等换成 RGB 三个分量，对 R、G、B 各算一遍这套式子，得到三个通道的亮度。

9. 光栅化阶段：隐藏面消除与多边形着色

顶点变换算完位置、光照算完颜色后，进入**光栅化**阶段。光栅化做的事很具体：把一个个三角形从连续的几何形状，变成屏幕上一堆离散的像素候选。每个候选像素通常叫**片元**（fragment），它还不一定能真的写进画面，因为同一个像素位置可能被多个三角形覆盖，最后只能留下最靠近相机的那个。

所以第 9 节其实解决两个问题：

1. **这个像素到底显示哪个物体**：用隐藏面消除 / Z-buffer。
2. **留下来的这个像素是什么颜色**：用 Flat、Gouraud 或 Phong 着色。

9.1 隐藏面消除与 Z-buffer

【先假设你不懂】 屏幕上一个像素，背后可能排着好几个物体（近的挡住远的）。怎么保证只画**最靠前**的那个、把被挡住的藏起来？

【最常用办法：**Z-buffer（深度缓冲）**】 除了存颜色的画布，再开一块同样大小的"深度表" Z-buffer。颜色缓冲存这个像素最终显示什么颜色；Z-buffer 存这个像素目前见过的**最近深度值**。

在 OpenGL 视口变换之后，深度通常落在 $[0, 1]$ ：0 更靠近近裁剪面，1 更靠近远裁剪面，所以这里可以直接理解成**数值越小越近**。一开始还没有任何三角形写入，Z-buffer 通常初始化为 1.0，也就是先假设每个像素都在最远处。

画每个三角形时，光栅化会枚举它覆盖到的每个片元。对每个片元做三步：

1. 算出这个片元自己的深度 z_{new} 。
2. 读出 Z-buffer 中旧的深度 z_{old} 。
3. 若 $z_{\text{new}} < z_{\text{old}}$ ，说明新片元更靠前，写入颜色并更新深度；否则说明它被前面的东西挡住，直接丢弃。

这样不管物体以什么顺序画进来，最后留下的都是每个像素**最靠前**的那个，远处被挡的自动消失。Z-buffer 的深度值就来自视口变换里的 $W_{\text{in}}Z = \frac{z+1}{2} \in [0, 1]$ （第 6 节）。

两种绘制策略对比（预备 0.11 的延伸）：基于物体（for each object，流水线 + Z-buffer 解决可见性）vs 基于图像（for each pixel，光线跟踪天然按"最近交点"解决可见性）。

【例子：一个像素被两个三角形争用】 某像素先后被红、蓝两个三角形覆盖（深度越小越近）：

1. 初始：该像素 Z-buffer 存 1.0（最远）、颜色空。
2. 画红三角形，它在该像素深度 0.7： $0.7 < 1.0 \rightarrow$ 写入，颜色=红，深度更新为 0.7。
3. 画蓝三角形，它在该像素深度 0.4： $0.4 < 0.7 \rightarrow$ 蓝在红前面，覆盖，颜色=蓝，深度更新为 0.4。
4. 若之后又来一个深度 0.9 的绿三角形： $0.9 > 0.4 \rightarrow$ 在蓝后面被挡，**丢弃**。

最终该像素显示蓝色。**画的先后顺序无所谓**，结果总是最近的蓝色——这就是 Z-buffer 的好处。

【容易混的点】 Z-buffer 解决的是"谁挡住谁"，不是"怎么变亮变暗"。颜色怎么来，是 9.2 的着色问题；深度测试只负责决定这个颜色有没有资格写进屏幕。

9.2 多边形着色：Flat / Gouraud / Phong

【先假设你不懂】一个三角形只有三个顶点，但它内部会覆盖很多像素。顶点上可以有位置、法向、颜色这些信息，内部像素没有现成信息，必须从顶点信息**插值**出来。这里的插值就是"按距离比例在两个或三个顶点之间平滑过渡"，和 12.2 的 lerp 是同一个思想。

【背景】已知多边形顶点的位置 (x_i, y_i, z_i) 、法向 N_i 、颜色 C_i 。这里 i 表示第几个顶点； N_i 是该顶点的法向，通常由相邻面法向求平均得到； C_i 是该顶点颜色，可以用第 8 节 Phong / Blinn 光照模型算出来。问题是：三角形内部每个片元的颜色怎么定？

三种方法各有侧重：**Flat** 最简单（不插值，整面同色）；**Gouraud** 插颜色（顶点光照，内部插值）；**Phong** 插法向（逐像素重算光照）。三者精度和速度恰好反向。下面逐一详解。

9.2.1 Flat 着色（平面着色）

【先假设你不懂】最省事的想法：一个三角形就代表一小块平面，整块平面理应是同一个颜色（或颜色变化极小）。那我只算一次颜色，然后把整个三角形涂成这一种颜色——不就行了？这正是 Flat 着色的思路。

【从例子出发】想象一座用三角面拼起来的低多边形山丘模型（Low-poly 风格）。每个三角形面朝不同方向，受光程度不同，所以每块颜色不同——但**同一块面内**却是一模一样的颜色，你能清楚地看到每条边界线。这就是 Flat 着色的视觉效果。

【怎么计算】计算哪个顶点的颜色来代表整个面？通常有两种做法：

- 用**面法向**（而不是顶点法向）算光照。面法向 = 面上两条边叉积归一化：

$$N_{\text{face}} = \frac{(V_1 - V_0) \times (V_2 - V_0)}{\|(V_1 - V_0) \times (V_2 - V_0)\|}$$

- 假设**光源和视点都在无穷远处**，则 $\mathbf{l}$ （光方向）、 $\mathbf{v}$ （视方向）在整个面上是常向量，代入 Blinn-Phong 公式只算一次，整个面共用同一个结果颜色。

【计算流程（三步）】

1. 由面顶点坐标叉积求面法向 N_{face} ，归一化。
2. 用 N_{face} 代入第 8 节光照公式（漫反射 + 镜面 + 环境光），算出该面的颜色 C_{face} 。
3. 三角形内每个像素都直接写 C_{face} ，无需插值。

【为什么会看到"棱角感"——**Mach Band 效应**】即使场景里的曲面用 Flat 近似，人眼对**颜色跳变边界**极为敏感：边界两侧颜色一深一浅时，大脑会夸大对比度，让边界线看上去比实际还要明显（这是人类视觉系统的空间频率响应造成的，叫**马赫带效应 Mach Band Effect**）。所以 Flat 的问题不只是"面内颜色固定"，而是面与面交界处会产生**过于刺眼的色块边**。

【Flat 什么时候合适】

- 物体本身就是真正的**多面体**（如骰子、建筑立面），面内颜色本来就应该一致。

- 追求"低多边形艺术风格"。
- **实时游戏早期**（算力极弱）或**测试阶段**（快速预览）。

9.2.2 Gouraud 着色（逐顶点光照 + 颜色插值）

【先假设你不懂】Flat 的问题是整个三角形同色，产生棱角感。能不能让颜色在面内**平滑过渡**？很自然的想法：先在三个顶点各算一个颜色，再把内部当成"颜色渐变"——离哪个顶点近，就更接近那个顶点的颜色。这正是 Gouraud 的思路。

【逐步推导】

第一步：逐顶点计算光照颜色

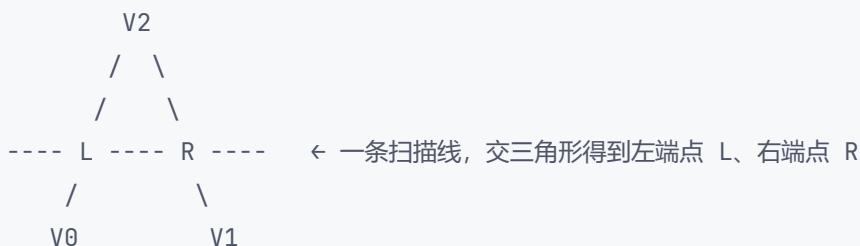
每个顶点 V_i 有一个法向 N_i （由该顶点周围所有相邻三角形的面法向平均后归一化得到）。对每个顶点，把 N_i 、顶点位置代入第 8 节光照公式：

$$C_i = k_a I_a + k_d I_d \max(N_i \cdot \mathbf{l}, 0) + k_s I_s \max(N_i \cdot \mathbf{h}, 0)^p$$

（RGB 三个通道各算一次。）这样三角形三个顶点各得到一个颜色 C_0, C_1, C_2 。

第二步：扫描线插值——把颜色平滑填到内部

光栅化三角形时，按行从上到下"扫"（每一行叫一条**扫描线**）：



- 对每条扫描线，用沿三角形**边**的线性插值，分别算出左端点 L 和右端点 R 的颜色（这一步是在边上插值，用两个顶点颜色）：

$$C_L = (1 - t_L)C_0 + t_L C_2, \quad C_R = (1 - t_R)C_1 + t_R C_2$$

其中 $t_L, t_R \in [0, 1]$ 是扫描线高度在该段边上的比例。

- 再沿这条扫描线**从 L 到 R** ，对每个像素做一次线性插值：

$$C(t) = (1 - t) C_L + t C_R, \quad t = \frac{x - x_L}{x_R - x_L}$$

更通用的方法：重心坐标（Barycentric Coordinates）

扫描线做了两次 lerp。等价地，三角形内部任意一点 P 都可以用**重心坐标** $(\lambda_0, \lambda_1, \lambda_2)$ 表示：

$$P = \lambda_0 V_0 + \lambda_1 V_1 + \lambda_2 V_2, \quad \lambda_0 + \lambda_1 + \lambda_2 = 1, \quad \lambda_i \geq 0$$

λ_i 的几何含义是 P 关于第 i 个顶点的"权重"——越靠近 V_i , λ_i 越大。算出 $(\lambda_0, \lambda_1, \lambda_2)$ 后, 直接加权插值颜色:

$$C_P = \lambda_0 C_0 + \lambda_1 C_1 + \lambda_2 C_2$$

λ_i 怎么算: 用面积法—— λ_i 等于"对面那个子三角形面积"除以"整个三角形面积":

$$\lambda_0 = \frac{S_{\triangle PV_1V_2}}{S_{\triangle V_0V_1V_2}}, \quad \lambda_1 = \frac{S_{\triangle V_0PV_2}}{S_{\triangle V_0V_1V_2}}, \quad \lambda_2 = \frac{S_{\triangle V_0V_1P}}{S_{\triangle V_0V_1V_2}}$$

三角形面积可用叉积的模算: $S = \frac{1}{2} \|(V_1 - V_0) \times (V_2 - V_0)\|$ 。

重心坐标不止插颜色: Gouraud 插颜色、Phong 插法向、甚至深度 (Z 值) 也用重心坐标插值 (不过深度要做透视校正插值, 见下文注意事项)。

【Gouraud 丢高光的具体原因】

一个典型场景: 三角形中心正对光源, 中心应出现镜面高光; 但三个顶点都斜着面向光源, 顶点颜色算出来都比较暗。Gouraud 把三个暗色插值到内部, 插值的结果是三种暗色的混合——**中间高光被抹平了**。

用数字感受一下: $C_0 = 0.1, C_1 = 0.1, C_2 = 0.1$ (三顶点都很暗), 则面中心插值结果:

$$C = \frac{1}{3} \times 0.1 + \frac{1}{3} \times 0.1 + \frac{1}{3} \times 0.1 = 0.1$$

但实际上中心该有高光, 真实颜色可能是 0.9。Gouraud 完全看不见这个高光。

【注意: 透视校正插值】 直接对屏幕坐标做线性插值会引入误差 (远处像素被"压扁")。正确做法是对 C/w 插值再除以插值后的 $1/w$ (透视校正插值), 但考试通常在正交投影或忽略此项, 实际算题时用普通线性插值即可, 除非题目特别说明。

9.2.3 Phong 着色 (逐像素光照, 插值法向)

【先假设你不懂】 Gouraud 丢高光, 原因是它只在顶点算光照, 内部只做颜色插值。能不能在**每个像素都重新算一次光照**? ——当然可以。但前提是每个像素要有自己的法向。顶点有法向, 内部像素的法向怎么来? 和 Gouraud 插颜色一样, **插值法向**就行了。这就是 Phong 着色 (注意: 它是 Phong **着色模型**, 与第 8 节的 Phong **反射模型**不同!)

【两个"Phong"的区分 (极易混淆)】

名称	概念层面	干什么
Phong 反射模型 (第 8 节)	光照计算	给定法向 N , 算漫反射 + 高光的公式 $k_d(N \cdot l) + k_s(N \cdot h)^p$

名称	概念层面	干什么
Phong 着色模型 (本节)	着色插值策略	逐像素地插值法向，再用某个光照公式（可以是 Phong 或 Blinn-Phong 反射模型）算颜色

结论： Phong 着色 = "插值法向 + 逐像素调用光照公式"。光照公式是 Phong 反射或 Blinn-Phong 均可，两者是独立的选择。

【逐步推导】

第一步：插值法向到每个像素

三角形三个顶点有法向 N_0, N_1, N_2 （由相邻面法向平均得到）。用重心坐标插值到片元点 P ：

$$N_P = \lambda_0 N_0 + \lambda_1 N_1 + \lambda_2 N_2$$

由于插值后 N_P 的长度一般不再是 1，**必须归一化**：

$$\hat{N}_P = \frac{N_P}{\|N_P\|}$$

为什么插值后要归一化： 两个单位向量的加权平均不一定还是单位向量。例如 $N_0 = (1, 0, 0)$ 、 $N_1 = (0, 1, 0)$ ，中点插值 $(0.5, 0.5, 0)$ 的长度是 $\frac{\sqrt{2}}{2} \neq 1$ 。不归一化就直接算光照， $N \cdot l$ 会偏小，导致颜色偏暗。

第二步：逐像素调用光照公式

用归一化后的 $\hat{N}_P$ 代入光照公式（Blinn-Phong 为例）：

$$C_P = k_a I_a + k_d I_d \max(\hat{N}_P \cdot \mathbf{l}, 0) + k_s I_s \max(\hat{N}_P \cdot \mathbf{h}, 0)^p$$

每个像素各算一次，RGB 三通道分别计算。

【Phong 为什么能保留高光】

还是前面的例子：三角形中心处插值法向 N_P 正好朝向光源，即使三个顶点法向都斜着，只要**中心的插值法向**让 $N_P \cdot \mathbf{h}$ 较大，那里就能算出高光。

直觉：Phong 在中心像素处，用"指向光源方向的法向"算出了强高光 → 中心像素颜色 ≈ 0.9 ，高光出现了。

【Phong 的代价】 内部每个像素都要：

1. 做一次重心坐标插值（计算 N_P ）
2. 做一次向量归一化

3. 完整计算光照公式（含向量点乘、幂次）

相比 Gouraud 内部只做一次 lerp，Phong 每个像素的工作量大约多 6–8 倍。在现代 GPU 上，这由片元着色器（Fragment Shader）逐像素执行，已经很快了，是实时渲染的主流方案之一。

【Phong 的局限】

- 若三角形面片很大（即多边形分辨率低），插值法向跨度大，可能在某些极端角度下出现"漏光"或黑斑（法向插值后与真实曲面法向偏差大）。
- 对非常尖锐的高光（ p 很大），若三角面片太大，高光仍可能被稀释。解决方法是提高网格细分程度，让每个三角形更小。

9.2.4 三种方法完整对比

对比项	Flat	Gouraud	Phong
插值对象	无	颜色 C	法向 N
光照计算次数	每面 1 次	每顶点 1 次	每像素 1 次
能否保留内部高光	否（高光按面算，位置不准）	否（高光被插值平掉）	能（逐像素算光照）
棱角感	严重（Mach Band 效应）	轻微（边界处有插值跳变）	最平滑
速度	最快	快	最慢（最多约 Gouraud 的 6–8 倍）
实现位置	顶点着色器或 CPU	顶点着色器（内插由硬件完成）	片元着色器
典型用途	低多边形艺术风/测试	早期实时渲染 / OpenGL 默认	现代实时渲染主流

高频简答题（必背两问）：

Q：Gouraud 为什么会丢高光？

A：Gouraud 只在三个顶点计算光照颜色，再把颜色插值到内部。若高光位置落在三角形内部（顶点处不亮），三个暗色插值的结果仍然是暗色，内部高光被完全抹掉。

Q：Phong 着色和 Phong 反射模型有什么区别？

A：Phong 反射模型是描述漫反射 + 镜面反射的光照公式；Phong 着色模型是一种着色策略，其核心是"对法向插值，逐像素重算光照"。两者是不同层面的概念，Phong 着色里用的光照公式也可以是 Blinn-Phong。

【扫描线是什么——更完整的解释】光栅化三角形时，按行从上到下逐行扫描。每一行水平扫描线与三角形相交，得到一个左端点 L 和右端点 R （三角形的两条边各交一点）。然后从 L 到 R 逐列填像素。

- Flat: 直接填整个面的颜色, 扫描线只用来判断哪些像素在三角形内。
- Gouraud: 扫描线进来时, 先沿两条边插值算出 L 和 R 的颜色; 再沿扫描线逐列在 C_L 和 C_R 之间插值。两层线性插值总共做了三次 lerp (等价于一次重心坐标插值)。
- Phong: 与 Gouraud 一样, 但插的是法向而非颜色; 插好法向后, 每个像素各调用一次光照公式算自己的颜色。

9.3 【例题】Gouraud 颜色插值

把"沿扫描线线性插值"算一遍。线性插值就是 12.2 节的 lerp。

题目: 一条三角形扫描线上, 左端点像素颜色(灰度) $C_L = 0.2$ 、右端点 $C_R = 0.8$ 。求这条线**正中间**那个像素、以及**距左端 1/4 处**像素的颜色。

写思路: Gouraud 着色已经在顶点算好了颜色, 扫描线内部的颜色由线性插值得到。先确定像素在左右端点之间的比例 t , 再把 C_L, C_R 代入 lerp 公式。

写公式: t 为从左到右的比例, 线性插值公式为

$$C(t) = (1 - t)C_L + tC_R$$

套数据计算: 正中间 $t = 0.5$, 所以 $C = (1 - 0.5)(0.2) + 0.5(0.8) = 0.1 + 0.4 = 0.5$ 。距左端 1/4 处 $t = 0.25$, 所以 $C = 0.75 \times 0.2 + 0.25 \times 0.8 = 0.15 + 0.2 = 0.35$ 。

颜色从左到右由 0.2 平滑过渡到 0.8, 中间 0.5、四分之一处 0.35, 符合线性渐变。**Gouraud 着色就是对每条扫描线、每个颜色通道这样插值。**若是 RGB 彩色, 则 R、G、B 三个分量各做一次这样的插值。

10. 光线跟踪

【先假设你不懂】 流水线(前面 1-9 节)是"站在物体的角度": 把每个物体投影、光栅化到屏幕。光线跟踪反过来, "站在像素的角度"想: 屏幕每个像素对应一条从眼睛射出去的光线, **这条光线第一个撞到的东西是什么颜色, 像素就是什么颜色**。这更接近真实世界里"光进入眼睛"的逆过程, 能自然产生阴影、镜面反射、玻璃折射, 但因为每个像素都要和场景求交, 慢。

一句话对比: 流水线 `for each object` (把物体画到屏幕); 光线跟踪 `for each pixel` (从像素发射光线找物体)。

10.1 三类光线

- **视线 (Primary / Viewing ray)**: 从相机过某个像素射出, 去找它第一个撞到的物体。
- **次级光线 (Secondary ray)**: 在撞击点继续射出的**反射**光线和**折射**光线, 递归地再去找它们撞到什么 (镜子里有镜子就是这么来的)。
- **阴影光线 (Shadow ray)**: 从撞击点射向光源; 若中途被别的物体挡住, 说明该点在**阴影**里, 不计这盏灯的直接光照。

10.2 光线方程与求交

【先假设你不懂】一条光线就是"从某点 $\mathbf{o}$ 出发、朝方向 $\mathbf{d}$ 一直走"。走多远用一个参数 t 表示—— $t = 0$ 在起点, t 越大走得越远。这就是光线方程:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d} \quad (t > 0)$$

【求交问题】想知道这条光线**会不会射中一个球、在哪射中**。

【逐步推导: 光线 × 球面】球心 $\mathbf{c}$ 、半径 R , 球面就是"所有到球心距离等于 R 的点": $\|\mathbf{P} - \mathbf{c}\|^2 = R^2$ 。光线上的点要射中球, 必须同时满足球面方程——把 $\mathbf{P} = \mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ 代进去:

$$\|\mathbf{o} + t\mathbf{d} - \mathbf{c}\|^2 = R^2$$

记 $\mathbf{e} = \mathbf{o} - \mathbf{c}$ (起点到球心的向量), 展开 (用 $\|\mathbf{a}\|^2 = \mathbf{a} \cdot \mathbf{a}$):

$$(\mathbf{e} + t\mathbf{d}) \cdot (\mathbf{e} + t\mathbf{d}) = (\mathbf{d} \cdot \mathbf{d})t^2 + 2(\mathbf{d} \cdot \mathbf{e})t + (\mathbf{e} \cdot \mathbf{e}) = R^2$$

整理成关于 t 的**一元二次方程**:

$$(\mathbf{d} \cdot \mathbf{d})t^2 + 2(\mathbf{d} \cdot \mathbf{e})t + (\mathbf{e} \cdot \mathbf{e} - R^2) = 0$$

【怎么读这个解】用初中的求根公式解 t , 看判别式:

- 判别式 < 0 : 方程无实根 → 光线**没碰到**球。
- 判别式 $= 0$: 一个根 → 光线**擦着**球边 (相切)。
- 判别式 > 0 : 两个根 → 光线穿过球 (一进一出); 取**较小的正根** t , 那才是离相机**最近**的那个交点 (先碰到的那面)。

10.3 光线与三角形求交 (网格场景常用)

球面求交只是为了把"代入方程解 t "讲清楚。真实模型大多是三角网格, 所以光线跟踪里更常见的是**光线 × 三角形**。不要把它硬塞进球面例题里, 先单独把思路讲明白。

设三角形三个顶点是 A, B, C , 光线是

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d} \quad (t > 0)$$

其中 $\mathbf{o}$ 是光线起点, $\mathbf{d}$ 是光线方向, t 是从起点沿方向走了多远。

【第一步：先和三角形所在的平面求交】 三角形一定躺在某个平面上。平面可以用一个点和一个法向表示：

$$\mathbf{n} \cdot (\mathbf{P} - \mathbf{P}_0) = 0$$

这里 $\mathbf{n}$ 是平面法向, $\mathbf{P}_0$ 是平面上的任意一个点, 通常直接取三角形顶点 A 。这个式子的意思是：如果点 $\mathbf{P}$ 在平面上, 那么从 A 指向 $\mathbf{P}$ 的向量 $\mathbf{P} - \mathbf{P}_0$ 一定和法向 $\mathbf{n}$ 垂直, 所以点乘为 0。

把光线点 $\mathbf{P} = \mathbf{o} + t\mathbf{d}$ 代入平面方程：

$$\mathbf{n} \cdot (\mathbf{o} + t\mathbf{d} - \mathbf{P}_0) = 0$$

展开后解 t ：

$$t = \frac{\mathbf{n} \cdot (\mathbf{P}_0 - \mathbf{o})}{\mathbf{n} \cdot \mathbf{d}}$$

若 $\mathbf{n} \cdot \mathbf{d} = 0$, 说明光线和平面平行, 要么不相交, 要么整条光线躺在平面里；考试里通常按不相交处理。若解出来 $t < 0$, 交点在相机后面, 也不能算射中。

【第二步：再判断交点是否在三角形内部】 第一步只保证光线打到了这个三角形所在的无限大平面, 但平面比三角形大得多, 所以还要检查交点是不是落在 A, B, C 围起来的范围内。常用方法是**重心坐标**：

$$\mathbf{P} = \alpha\mathbf{A} + \beta\mathbf{B} + \gamma\mathbf{C}, \quad \alpha + \beta + \gamma = 1$$

这里 α, β, γ 是三个顶点的权重。它们的意思很直观：点 $\mathbf{P}$ 由三个顶点按比例加权平均得到。若

$$\alpha \geq 0, \quad \beta \geq 0, \quad \gamma \geq 0$$

并且三者和为 1, 那么 $\mathbf{P}$ 就在三角形内部或边上；只要有一个权重为负, 点就跑到三角形外面去了。

所以光线与三角形求交的完整判断顺序是：先解平面交点 t , 再要求 $t > 0$, 最后用重心坐标判断交点是否在三角形内部。三个条件都满足, 才是真的射中这个三角形。

10.4 【例题】光线与球面求交

题目：相机在原点 $\mathbf{o} = (0, 0, 0)$, 光线方向 $\mathbf{d} = (0, 0, -1)$ (朝 $-z$ 看, 已单位化)。场景里有个球, 球心 $\mathbf{c} = (0, 0, -5)$ 、半径 $R = 1$ 。求光线与球的最近交点。

写思路：把光线方程代入球面方程, 得到关于 t 的一元二次方程。先用判别式判断有没有交点, 再取较小的正根作为最近交点, 最后把这个 t 回代到光线方程。

写公式：把光线方程、球面代入后的二次方程和交点回代公式写清楚。

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad \mathbf{e} = \mathbf{o} - \mathbf{c}$$

把光线方程代入球面方程后，可以得到关于 t 的二次方程：

$$(\mathbf{d} \cdot \mathbf{d})t^2 + 2(\mathbf{d} \cdot \mathbf{e})t + (\mathbf{e} \cdot \mathbf{e} - R^2) = 0$$

解出最近的正根以后，把它回代光线方程得到交点：

$$\mathbf{P} = \mathbf{r}(t)$$

套数据计算： $\mathbf{e} = \mathbf{o} - \mathbf{c} = (0, 0, 5)$ 。

- $\mathbf{d} \cdot \mathbf{d} = 0 + 0 + 1 = 1$
- $\mathbf{d} \cdot \mathbf{e} = (0)(0) + (0)(0) + (-1)(5) = -5$
- $\mathbf{e} \cdot \mathbf{e} - R^2 = 25 - 1 = 24$

方程： $1 \cdot t^2 + 2(-5)t + 24 = 0$ ，即 $t^2 - 10t + 24 = 0$ 。

判别式 $= (-10)^2 - 4(24) = 100 - 96 = 4 > 0 \rightarrow$ 有两个交点。

$$t = \frac{10 \pm \sqrt{4}}{2} = \frac{10 \pm 2}{2} = 6 \text{ 或 } 4$$

取较小的正根 $t = 4$ （最近的那面）。回代：

$$\mathbf{r}(4) = (0, 0, 0) + 4(0, 0, -1) = (0, 0, -4)$$

最近交点 $(0, 0, -4)$ ——正是球面朝相机这一侧的点（球心 $z = -5$ 、半径 1，朝相机的面在 $z = -4$ ） $\checkmark$ 。
另一根 $t = 6$ 对应穿出球的背面点 $(0, 0, -6)$ 。

10.5 Whitted 着色模型

【先假设你不懂】 光线跟踪找到最近交点以后，还没结束，因为交点的颜色不只来自它自己被灯照亮多少。若这个点在镜子上，你还会看到镜子里反射来的东西；若这个点在玻璃上，你还会看到玻璃后面透过来的东西。Whitted 模型就是把这些贡献分开加起来：

1. 直接光照：这个点被灯直接照亮的颜色，用第 8 节的 Phong / Blinn。
2. 反射光：沿镜面反射方向再发一条光线，看看它撞到什么。
3. 折射光：沿折射方向再发一条光线，看看玻璃后面是什么。

所以撞击点最终的颜色可以写成：

$$I = \underbrace{I_a + I_{\text{diffuse}} + I_{\text{specular}}}_{\text{直接光照（局部 Phong，第 8 节）}} + \underbrace{k_r I_{\text{reflect}} + k_t I_{\text{refract}}}_{\text{间接光照（沿次级光线递归算）}}$$

把每个符号讲清楚：

- I : 当前交点最终返回给相机的颜色。
- I_a : 环境光, 就是第 8 节的底光。
- I_{diffuse} : 漫反射, 决定物体本色亮不亮。
- I_{specular} : 镜面高光, 是灯在物体表面形成的亮斑。
- I_{reflect} : 反射光线递归追踪得到的颜色, 也就是"镜子里看到的颜色"。
- I_{refract} : 折射光线递归追踪得到的颜色, 也就是"透过玻璃看到的颜色"。
- k_r : 反射系数, 越大越像镜子。
- k_t : 透射 / 折射系数, 越大越透明。

【直接光照怎么来】 对每一盏灯, 从交点向光源发一条**阴影光线**。如果中途被别的物体挡住, 说明这盏灯照不到该点, 那么这盏灯的漫反射和镜面项不算; 如果没挡住, 就按第 8 节 Phong / Blinn 公式计算 I_{diffuse} 和 I_{specular} 。

【反射光怎么来】 若材质有镜面反射, 就按法向把入射光线弹出去。设当前追踪的入射方向是 $\mathbf{d}$ (从相机射向物体, 指向表面内部), 交点法向是 $\mathbf{n}$, 则反射方向常写成:

$$\mathbf{d}_{\text{reflect}} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$$

这个式子和第 8 节的镜面反射公式说的是同一件事, 只是第 8 节从"点指向光源的方向"出发, 这里从"当前光线前进方向"出发, 所以符号形式不同。这条反射光线会继续求最近交点, 那个交点再按 Whitted 模型算颜色, 结果就是 I_{reflect} 。最后乘上 k_r , 表示这个材质只接收其中一部分反射贡献。

【折射光怎么来】 若材质透明, 就按 8.4 的 Snell 定律求折射方向, 再沿折射方向发一条新光线。它穿过玻璃后撞到的东西贡献给 I_{refract} , 最后乘上 k_t 。如果发生全反射, 就没有折射光, 能量转到反射方向上。

【为什么叫递归】 因为反射光线或折射光线碰到新物体以后, 又会重复同一套流程: 直接光照、阴影光线、反射、折射。为了避免无限算下去, 程序会设置递归深度上限, 比如最多反射 / 折射 5 层; 超过上限就返回黑色或只返回环境光。

一句话记: Phong 只算"这个点被灯怎么照", Whitted 在此基础上再算"这个点还能从镜子方向、玻璃方向看见什么"。

第二大题：几何（贝塞尔 + 网格）

考核重点：贝塞尔曲线 / 曲面（de Casteljau + Bernstein）和 网格处理（去噪 / 细分 / 简化）。

11. 几何的表示方式

表示	特点	例子
隐式 (Implicit)	不直接给位置，只给满足的方程	球面 $x^2 + y^2 + z^2 = 1$
显式 (Explicit)	直接给出空间位置	$(x, y, z) = (1, 2, 3)$
参数 (Parametric)	坐标写成参数 $t \in [0, 1]$ 的函数	直线段 $P(t) = (1 - t)P_0 + tP_1$
点云	点的集合 $P = \{P_1, \dots, P_n\}$	三维扫描、双目视觉三角化、结构光、Kinect
体表示 (Voxel)	1=体内、0=体外；可做 CSG 布尔运算	像素的三维扩展
SDF (符号距离场)	每点存到表面的有符号距离	规整、抗噪、便于融合；但数据量大需稀疏化
网格 (Mesh)	顶点 + 拓扑连接关系	PLY/OBJ/STL/OFF

参数表示的优越性：形状不依赖坐标系（形状不变性）、易做平移 / 缩放 / 旋转、系数几何意义明确、便于交互控制形状。

12. 贝塞尔曲线

12.1 定义与控制点

贝塞尔曲线是一段 n 次多项式曲线，由 $n + 1$ 个控制点确定。**性质：**

- 一定经过首末控制点，但一般不经过中间控制点（它们只提供方向）。
- 首末端点处的切线方向与首段 / 末段控制多边形边一致。
- 仿射变换不变性：**对控制点做仿射变换 = 对曲线做仿射变换。
- 凸包性：**曲线落在控制点的凸包内。

12.2 de Casteljau 算法（递归线性插值，几何作图法）

【先从一条直线上的取点说起】 如果有两个点 $A = (0, 0)$ 和 $B = (4, 0)$ ，现在说“取从 A 走到 B 的四分之一处”，那这个点显然是 $(1, 0)$ 。把“走了多少比例”记成 t ，当 $t = 0$ 时还在 A ，当 $t = 1$ 时到达 B ，当 $t = 0.25$ 时就是刚才的四分之一处。这个按比例取中间点的操作叫**线性插值**，英文常写作 lerp，意思就是在两点之间按直线比例混合：

$$\text{lerp}(A, B, t) = (1 - t)A + tB$$

代回刚才的例子， $\text{lerp}(A, B, 0.25) = (0.75 \cdot 0 + 0.25 \cdot 4, 0) = (1, 0)$ ，正好在 $A \rightarrow B$ 的 $1/4$ 处。后面的 de Casteljau 算法没有别的新动作，本质上就是把这个"按比例取中间点"一层一层嵌套起来。

【从例子出发：三个控制点怎么画出弯曲的线】 给三个控制点 P_0, P_1, P_2 ，固定一个比例 t （比如 $t = 0.5$ ），做两层 lerp：

1. 第一层：在 P_0P_1 上取 t 处得 D ，在 P_1P_2 上取 t 处得 E 。
2. 第二层：再在 DE 上取 t 处，得到一个点——它就是曲线在参数 t 处的点 $B(t)$ 。

让 t 从 0 滑到 1，这个点划过的轨迹就是一条**光滑的弯曲**线。这套"对相邻点反复取中间点"的几何作图法，就是 **de Casteljau 算法**。控制点越多，层数越多（ $n + 1$ 个点要做 n 层）。

【把作图过程写成公式】 第 k 层第 i 个点由上一层相邻两点 lerp 而来，所以递归形式是

$$P_i^{(k)}(t) = (1 - t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}, \quad P_i^{(0)} = P_i$$

做满 n 层后只剩一个点，就是曲线上的点 $B(t) = P_0^{(n)}(t)$ 。

12.3 Bernstein 多项式形式（代数定义）

【先假设你不懂这串系数哪来的】 把上面的嵌套 lerp **展开**会发生什么？以二次为例， $B(t) = (1 - t)[(1 - t)P_0 + tP_1] + t[(1 - t)P_1 + tP_2]$ ，乘开合并：

$$B(t) = (1 - t)^2P_0 + 2(1 - t)tP_1 + t^2P_2$$

看每个控制点前面的系数： $(1 - t)^2$ ， $2(1 - t)t$ ， t^2 。注意那个 **2**——它来自 P_1 出现了两次，也就是从两条插值路径里都经过了中间控制点。这种"出现次数"正是**组合数** C_n^i ，组合数就是从 n 个位置里选 i 个位置的选法数量。本文统一用常见考试写法：组合数写 C_n^i ，排列数写 A_n^i 。把这个规律推广到 n 次，就得到 **Bernstein 基函数**；基函数的意思是"每个控制点对应的权重函数"，它告诉我们某个 t 下每个控制点占多少分量：

$$B(t) = \sum_{i=0}^n B_{i,n}(t) P_i, \quad B_{i,n}(t) = C_n^i t^i (1 - t)^{n-i}, \quad 0 \leq t \leq 1$$

所有基函数**系数之和恒为 1**（因为 $[(1 - t) + t]^n = 1$ ）——这就是"曲线是控制点的加权平均""仿射不变"的根源（权重和为 1 的组合就是仿射组合）。

常用低次展开可以直接背下来，考试代数计算时很省时间：

- 一次（直线 / 就是 lerp）： $B(t) = (1 - t)P_0 + tP_1$
- 二次（3 个控制点）： $B(t) = (1 - t)^2P_0 + 2(1 - t)tP_1 + t^2P_2$
- 三次（4 个控制点，最常用）：

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

(系数 1, 3, 3, 1 正是杨辉三角第 3 行——组合数 $C_3^0, C_3^1, C_3^2, C_3^3$ 。)

12.4 曲线拼接与连续性

控制点过多会自由度差、难反映真实走向 → 拆成多段三次贝塞尔曲线**拼接**。连续性：

- C^0 ：位置连续（首尾点重合）。
- C^1 ：一阶导（切线）连续。
- C^2 ：二阶导（曲率）连续。

12.5 【例题 4】贝塞尔曲线求值（二次热身 + 三次完整）

(a) 二次曲线热身：控制点 $P_0 = (0, 0), P_1 = (2, 2), P_2 = (4, 0)$ ，求 $B(0.5)$ 。

写思路：二次贝塞尔用 de Casteljau 两层插值。第一层在相邻控制点之间插值得到两个中间点，第二层再在这两个中间点之间插值得到曲线点。

写公式：二次 de Casteljau 的两层插值可以写成下面三条式子。

$$D = (1-t)P_0 + tP_1, \quad E = (1-t)P_1 + tP_2, \quad B(t) = (1-t)D + tE$$

套数据计算：当 $t = 0.5$ 时，每次插值都是取中点。第一层得到 $D = (1, 1)$ 、 $E = (3, 1)$ ，第二层得到 $B(0.5) = (2, 1)$ 。用 Bernstein 形式验证： $0.25P_0 + 0.5P_1 + 0.25P_2 = (0, 0) + (1, 1) + (1, 0) = (2, 1)$ ✓。

(b) 三次曲线（最常用，完整走三层）：控制点 $P_0 = (0, 0), P_1 = (0, 3), P_2 = (3, 3), P_3 = (3, 0)$ ，求 $t = 0.5$ 处的点。

写思路：三次贝塞尔有 4 个控制点，de Casteljau 要做 $4 - 1 = 3$ 层。因为 $t = 0.5$ ，每层都是相邻两点取中点。

写公式：三次曲线每一层仍然只使用同一个线性插值公式。

$$\text{lerp}(A, B, t) = (1-t)A + tB$$

三次曲线也可以写成 Bernstein 形式：

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

套数据计算：第一层由 4 点变 3 点。

$$A = \frac{1}{2}P_0 + \frac{1}{2}P_1 = (0, 1.5)$$

$$B = \frac{1}{2}P_1 + \frac{1}{2}P_2 = (1.5, 3)$$

$$C = \frac{1}{2}P_2 + \frac{1}{2}P_3 = (3, 1.5)$$

第二层继续对相邻中间点取中点，由 3 个点变成 2 个点。

$$D = \frac{1}{2}A + \frac{1}{2}B = (0.75, 2.25), \quad E = \frac{1}{2}B + \frac{1}{2}C = (2.25, 2.25)$$

第三层再对最后两个中间点取中点，由 2 个点变成 1 个曲线点。

$$B(0.5) = \frac{1}{2}D + \frac{1}{2}E = (1.5, 2.25)$$

Bernstein 验证（三次系数 1, 3, 3, 1, $t = 0.5$ 时各为 $\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$ ）:

$$\begin{aligned} B(0.5) &= \frac{1}{8}P_0 + \frac{3}{8}P_1 + \frac{3}{8}P_2 + \frac{1}{8}P_3 = \frac{1}{8}(0, 0) + \frac{3}{8}(0, 3) + \frac{3}{8}(3, 3) + \frac{1}{8}(3, 0) \\ &= \left(\frac{3}{8} \cdot 3 + \frac{1}{8} \cdot 3, \frac{3}{8} \cdot 3 + \frac{3}{8} \cdot 3\right) = \left(\frac{9}{8} + \frac{3}{8}, \frac{9}{8} + \frac{9}{8}\right) = (1.5, 2.25) \checkmark \end{aligned}$$

两种方法结果一致。注意曲线点 (1.5, 2.25) **没经过任何中间控制点**，但居于这堆控制点的"凸包"内部，符合 12.1 的性质。

13. 贝塞尔曲面

13.0 从曲线到曲面：建立直觉

【先假设你不懂】 贝塞尔曲线用"一排控制点 + 一个参数 t "描述一条线。那如果想描述一张光滑的面，该怎么推广？

【从例子出发：一张面需要两个方向】 想象一块橡皮泥。一条线只需要知道"沿线走了多远"（一个参数 t ）；一张面上的任何一点，需要知道"沿横方向走了多远"和"沿纵方向走了多远"——**两个参数**。我们把它们记作 u 和 v ，各自在 $[0, 1]$ 之间变化。

固定 u, v 各一个值，就确定曲面上一个点。让 u, v 从 0 到 1 扫一遍，就铺出整块曲面片。

【参数 u, v 的几何直觉】

情形	含义	等价于什么
固定 $v = 0.5$ ，让 u 从 0 到 1 变化	在曲面上沿横方向走一条曲线	曲面上纵向参数为 0.5 处的"等参线"
固定 $u = 0$ ，让 v 从 0 到 1 变化	曲面左边界那条曲线	控制网格最左列决定的贝塞尔曲线
$u = 0, v = 0$	曲面的一个角	控制点 P_{00} 本身 （角点恰好在曲面上）
$u = 0.5, v = 0.5$	曲面正中心点	需要计算得到，不等于任何控制点

记住：四个角 $(u, v) \in \{(0, 0), (1, 0), (0, 1), (1, 1)\}$ 对应的曲面点，正好等于四个角控制点 $P_{00}, P_{m0}, P_{0n}, P_{mn}$ 。只有角控制点保证在曲面上，内部控制点一般不在曲面上，它们只是"拉力把手"。

13.1 控制网格详解

【先假设你不懂】 曲线用一排控制点 $P_0, P_1, \dots, P_n$ ；曲面用一张**二维控制点网格** P_{ij} 。下标 i 是行号（对应 v 方向，从 0 到 m ）， j 是列号（对应 u 方向，从 0 到 n ）。

以最常见的 4×4 控制网格（双三次， $m = n = 3$ ）为例，共 16 个点：

$$\begin{pmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{pmatrix}$$

每个控制点的作用（直觉描述）：

- **四个角点** $P_{00}, P_{03}, P_{30}, P_{33}$ ：曲面片四个角的真实位置，曲面一定经过这四点。
- **四条边上的非角控制点**（如 P_{01}, P_{02} ）：决定四条边界曲线的形状。四条边界曲线就是固定 $u = 0, u = 1, v = 0, v = 1$ 时得到的四条贝塞尔曲线。
- **内部四个点** $P_{11}, P_{12}, P_{21}, P_{22}$ ：不在曲面上，像"隐形把手"，把曲面中间往某个方向拉扯。

【用一个比喻】 把控制网格想成用细线绷紧的一块帆布框架：四个角是框架的四个钉子（真正固定），边上的点是框架钢丝的弯折方向（决定边的弧度），中间的点是从背后顶着帆布的手指（把面往里或往外推）。

13.2 核心算法：两次 de Casteljau ("曲线的曲线")

这是整节最重要的计算方法。不要上来背双重求和公式——先把这个两步流程搞懂，公式自然就懂了。

【关键思想】 贝塞尔曲面 = 先在一个方向做一批贝塞尔曲线，再在另一个方向把这批曲线的结果当成新控制点再做一次贝塞尔曲线。

【以 3×3 网格（二次贝塞尔曲面）为例，固定 (u, v) 求曲面点】

控制网格三行各三个点：

行0: P00 P01 P02
行1: P10 P11 P12
行2: P20 P21 P22

第一阶段：对每一行，沿 u 方向做二次贝塞尔曲线

把每行的 3 个点当成一条二次贝塞尔曲线的控制点，在参数 u 处求值：

$$Q_0 = (1 - u)^2 P_{00} + 2u(1 - u)P_{01} + u^2 P_{02} \quad \leftarrow \text{第 0 行算出的中间点}$$

$$Q_1 = (1-u)^2 P_{10} + 2u(1-u)P_{11} + u^2 P_{12} \quad \leftarrow \text{第 1 行}$$

$$Q_2 = (1-u)^2 P_{20} + 2u(1-u)P_{21} + u^2 P_{22} \quad \leftarrow \text{第 2 行}$$

现在得到了三个中间点 Q_0, Q_1, Q_2 。这三个点不是曲线上的点，而是"过渡结果"——把原本 3×3 的网格"压缩"成了一列 3 个点。

第二阶段：把 Q_0, Q_1, Q_2 当新控制点，沿 v 方向再做一次二次贝塞尔曲线

$$B(u, v) = (1-v)^2 Q_0 + 2v(1-v)Q_1 + v^2 Q_2$$

这个结果才是曲面在 (u, v) 处真正的点。

【为什么叫"曲线的曲线"】 Q_0, Q_1, Q_2 本身各是"一条关于 u 的曲线上的一点"，而最终 $B(u, v)$ 是"用 v 把三条 u -曲线的结果连起来"。所以贝塞尔曲面是**参数曲线族的包络**，即"一族曲线扫出一张面"。

【先行后列、先列后行，结果一样吗？】 完全一样。可以先沿 u （每行做曲线），再沿 v （结果做曲线）；也可以先沿 v （每列做曲线），再沿 u 。两种顺序等价，是张量积结构的对称性保证的。

13.3 代数公式的推导（从两次 de Casteljau 推出双重求和）

【先假设你看不懂双重求和公式】 教材里的公式长这样，很多人直接跳过：

$$B(u, v) = \sum_{i=0}^m \sum_{j=0}^n B_{i,m}(u) B_{j,n}(v) P_{ij}$$

其实它不神秘，就是两次 de Casteljau 展开后自然出来的结果。下面一步一步推。

【以 2×2 （双线性， $m = n = 1$ ）为例推导】

4 个控制点 $P_{00}, P_{01}, P_{10}, P_{11}$ 。

第一阶段沿 u 做线性插值（ $m = 1$ ，每行只有 2 个点）：

$$Q_0 = (1-u)P_{00} + uP_{01}$$

$$Q_1 = (1-u)P_{10} + uP_{11}$$

第二阶段沿 v 做线性插值：

$$B(u, v) = (1-v)Q_0 + vQ_1$$

把 Q_0, Q_1 代入展开：

$$B(u, v) = (1-v)[(1-u)P_{00} + uP_{01}] + v[(1-u)P_{10} + uP_{11}]$$

$$= (1-u)(1-v)P_{00} + u(1-v)P_{01} + (1-u)vP_{10} + uvP_{11}$$

对照双重求和公式，这里 $B_{0,1}(u) = (1-u)$, $B_{1,1}(u) = u$, $B_{0,1}(v) = (1-v)$, $B_{1,1}(v) = v$, 完全一致：

$$\sum_{i=0}^1 \sum_{j=0}^1 B_{i,1}(u) B_{j,1}(v) P_{ij} = B_{0,1}(u)B_{0,1}(v)P_{00} + B_{1,1}(u)B_{0,1}(v)P_{10} + B_{0,1}(u)B_{1,1}(v)P_{01} + B_{1,1}(u)B_{1,1}(v)P_{11}$$

结论：双重求和不是新东西，就是两次 lerp/de Casteljau 展开后自然出来的形式。 $B_{i,m}(u) \cdot B_{j,n}(v)$ 是控制点 P_{ij} 的总权重，等于" P_{ij} 在 u 方向的权重 $\times$ 在 v 方向的权重"。

【一般情况（任意次数 m, n ）的公式】

u 方向次数 m , v 方向次数 n , 控制点共 $(m+1)(n+1)$ 个：

$$B(u, v) = \sum_{i=0}^m \sum_{j=0}^n B_{i,m}(u) B_{j,n}(v) P_{ij}$$

其中伯恩斯坦基函数：

$$B_{i,m}(u) = C_m^i u^i (1-u)^{m-i}, \quad B_{j,n}(v) = C_n^j v^j (1-v)^{n-j}$$

记忆方法：把一维贝塞尔曲线公式 $\sum_i B_{i,m}(t)P_i$ 对两个方向各用一次，控制点改成二维下标 P_{ij} , 权重改成两个方向权重相乘，就是曲面公式。

常用次数的展开权重：

- 线性 ($n=1$): $B_0 = (1-t)$, $B_1 = t$
- 二次 ($n=2$): $B_0 = (1-t)^2$, $B_1 = 2t(1-t)$, $B_2 = t^2$
- 三次 ($n=3$): $B_0 = (1-t)^3$, $B_1 = 3t(1-t)^2$, $B_2 = 3t^2(1-t)$, $B_3 = t^3$

13.4 曲面的重要性质（继承自贝塞尔曲线）

【性质 1：角点插值——只有四个角点保证在曲面上】

把 $(u, v) = (0, 0)$ 代入公式：所有 $B_{i,m}(0) = 0$ 除了 $B_{0,m}(0) = 1$, 所有 $B_{j,n}(0) = 0$ 除了 $B_{0,n}(0) = 1$, 所以 $B(0, 0) = P_{00}$ 。同理 $B(1, 0) = P_{m0}$, $B(0, 1) = P_{0n}$, $B(1, 1) = P_{mn}$ 。

考试常考：哪些控制点在曲面上？——**只有四个角控制点**, $P_{00}, P_{m0}, P_{0n}, P_{mn}$ 是曲面的四个角。其余控制点不在曲面上，它们的角色是拉扯曲面的形状。

【性质 2：边界曲线——固定一个参数得到边界贝塞尔曲线】

- 固定 $u = 0$, 让 v 变化: $B(0, v) = \sum_j B_{j,n}(v)P_{0j}$, 这正好是以第 0 行控制点为控制点的贝塞尔曲线。
- 固定 $v = 0$, 让 u 变化: $B(u, 0) = \sum_i B_{i,m}(u)P_{i0}$, 这是以第 0 列控制点为控制点的贝塞尔曲线。

四条边界曲线分别由四条边上的控制点决定。曲面与控制网格在角点和边上的"连接方向"完全由边控制点给出。

【性质 3: 凸包性——曲面不会跑出控制网格的凸包】

因为所有权重 $B_{i,m}(u)B_{j,n}(v) \geq 0$ 且所有权重之和为 1 (两组伯恩斯坦基函数各自和为 1, 乘积之和也为 1), 所以曲面点是控制点的凸组合, 曲面一定落在所有控制点张成的凸包内。

直觉: 曲面不会跑到控制点"外面去", 控制网格包围着曲面。

【性质 4: 仿射不变性——先变换控制点再生成曲面, 等价于先生成曲面再变换】

对控制点 P_{ij} 做任意平移、旋转、缩放 (仿射变换), 再生成曲面, 结果和先生成曲面再做相同仿射变换完全一致。所以只需要对控制点做变换, 不需要对曲面每个点逐一变换。

13.5 计算步骤总结 (考场用)

拿到一道"贝塞尔曲面求点"的题, 按下面顺序来:

1. 确认控制网格大小和次数: $m \times n$ 控制点 $\rightarrow u$ 方向 $m - 1$ 次, v 方向 $n - 1$ 次 (行数 $-1 =$ 次数)。
2. 写出对应次数的伯恩斯坦权重 (线性/二次/三次, 见上面常用次数表)。
3. 第一阶段: 对每一行, 用权重 $B_{j,n}(u)$ (j 遍历列号) 做加权求和, 得到中间点 Q_i (共 m 个)。
4. 第二阶段: 把 $Q_0, Q_1, \dots$ 当新控制点, 用权重 $B_{i,m}(v)$ (i 遍历行号) 做加权求和, 得到曲面点 $B(u, v)$ 。
5. 验证: 若 $u = 0$ 或 $v = 0$ 则可简单验证角点是否等于相应控制点。

常见坑: 行是 u 方向还是 v 方向? ——约定好就行, 只要两个阶段用的参数和方向对应一致即可。
本文档约定: 行内的列号 j 对应 u , 行号 i 对应 v (即先沿 u 对每行插值, 再沿 v 合成)。两道例题都用这个约定。

13.6 【例题】最简双线性曲面求一点

用最简单的 2×2 控制网格 (每个方向只 1 次, 即 lerp), 把"先行后列"两次插值走一遍。 3×3 只是把每行的 lerp 换成二次 de Casteljau, 套路完全一样。

题目: 4 个控制点 $P_{00} = (0, 0, 0)$, $P_{01} = (1, 0, 1)$, $P_{10} = (0, 1, 1)$, $P_{11} = (1, 1, 0)$ 。求曲面在 $(u, v) = (0.5, 0.5)$ 处的点。

写思路：双线性曲面就是两个方向各做一次线性插值。先固定行方向，沿 u 对每一行插值得到 Q_0, Q_1 ；再沿 v 对 Q_0, Q_1 插值，得到曲面点。

写公式：双线性曲面先沿 u 方向插值，再沿 v 方向插值。

$$Q_0 = (1-u)P_{00} + uP_{01}, \quad Q_1 = (1-u)P_{10} + uP_{11}$$

$$B(u, v) = (1-v)Q_0 + vQ_1$$

套数据计算：先沿 u 方向每一行各做一次 lerp，且 $u = 0.5$ 。

$$\text{行0: } Q_0 = \frac{1}{2}P_{00} + \frac{1}{2}P_{01} = (0.5, 0, 0.5)$$

$$\text{行1: } Q_1 = \frac{1}{2}P_{10} + \frac{1}{2}P_{11} = (0.5, 1, 0.5)$$

再把两个中间点沿 v 做 lerp，且 $v = 0.5$ 。

$$B(0.5, 0.5) = \frac{1}{2}Q_0 + \frac{1}{2}Q_1 = (0.5, 0.5, 0.5)$$

曲面中心点是 $(0.5, 0.5, 0.5)$ 。注意四个角的 z 是 $0, 1, 1, 0$ ，中心 $z = 0.5$ ，正是这块"马鞍形"小片的中间高度，符合直觉。

13.7 【例题】二次贝塞尔曲面求一点

这题用 3×3 控制网格走一遍，目的不是增加难度，而是让你看到"曲面 = 行方向算曲线 + 列方向再算曲线"到底怎么落到数字上。

题目：给 3×3 控制点网格：

$$P_{00} = (0, 0, 0) \quad P_{01} = (1, 0, 1) \quad P_{02} = (2, 0, 0)$$

$$P_{10} = (0, 1, 1) \quad P_{11} = (1, 1, 2) \quad P_{12} = (2, 1, 1)$$

$$P_{20} = (0, 2, 0) \quad P_{21} = (1, 2, 1) \quad P_{22} = (2, 2, 0)$$

求曲面在 $(u, v) = (0.5, 0.5)$ 处的点。

写思路：这是二次贝塞尔曲面，因为每个方向都有 3 个控制点。先沿每一行用二次贝塞尔权重 $(1-u)^2, 2(1-u)u, u^2$ 算出 Q_0, Q_1, Q_2 ；再把 Q_0, Q_1, Q_2 当成一条二次贝塞尔曲线的三个控制点，沿 v 再算一次。

写公式：二次贝塞尔曲线公式为

$$B(t) = (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2$$

因此曲面先沿 u 方向算三行：

$$Q_i = (1-u)^2 P_{i0} + 2(1-u)u P_{i1} + u^2 P_{i2} \quad (i = 0, 1, 2)$$

再沿 v 方向合成：

$$B(u, v) = (1-v)^2 Q_0 + 2(1-v)v Q_1 + v^2 Q_2$$

套数据计算：当 $u = 0.5$ 时，二次权重是 $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$ 。先算每一行：

$$Q_0 = \frac{1}{4}(0, 0, 0) + \frac{1}{2}(1, 0, 1) + \frac{1}{4}(2, 0, 0) = (1, 0, 0.5)$$

$$Q_1 = \frac{1}{4}(0, 1, 1) + \frac{1}{2}(1, 1, 2) + \frac{1}{4}(2, 1, 1) = (1, 1, 1.5)$$

$$Q_2 = \frac{1}{4}(0, 2, 0) + \frac{1}{2}(1, 2, 1) + \frac{1}{4}(2, 2, 0) = (1, 2, 0.5)$$

再沿 $v = 0.5$ 合成，权重仍然是 $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$ ：

$$\begin{aligned} B(0.5, 0.5) &= \frac{1}{4}Q_0 + \frac{1}{2}Q_1 + \frac{1}{4}Q_2 \\ &= \frac{1}{4}(1, 0, 0.5) + \frac{1}{2}(1, 1, 1.5) + \frac{1}{4}(1, 2, 0.5) = (1, 1, 1) \end{aligned}$$

所以曲面中心点是 $(1, 1, 1)$ 。这个结果也符合直觉：中间控制点 P_{11} 的高度是 2，会把曲面中心往上拉，但周围点高度较低，所以最后中心高度不是 2，而是加权平均后的 1。

14. 网格处理

14.1 拓扑与几何

- **几何：**顶点的具体坐标位置。
- **拓扑：**顶点之间的连接关系（不关心具体坐标）。
- **流形（Manifold）：**局部具有欧氏空间性质。二维流形即"表面"：用小球去切，切到的是一个二维圆盘。非流形如：一条边被 3 个面共享。

14.2 网格去噪：Laplacian 平滑

【先假设你不懂】 扫描得到的网格表面常有很多小毛刺（噪声）——某些顶点莫名其妙地凸出或凹进。怎么把表面"磨平"？

【从例子出发】 想象一个顶点孤零零地凸在它周围邻居的上方（一根毛刺）。直觉做法：把它往**邻居们的平均位置**拉一点，它就不那么扎眼了。对每个顶点都这么做，整个面就光滑了。这就是 Laplacian 平滑。

【先把符号讲清楚】 设当前要更新的顶点是 p_i 。它周围和它直接连边的顶点叫邻居，邻居集合写成 $N(i)$ ； $|N(i)|$ 表示邻居个数。比如一个顶点连着 4 条边，就有 4 个一环邻居， $|N(i)| = 4$ 。

【把直觉写成公式】 先算邻居平均位置：

$$\bar{p}_i = \frac{1}{|N(i)|} \sum_{j \in N(i)} p_j$$

这里的 $\sum_{j \in N(i)} p_j$ 表示把所有邻居顶点坐标加起来，再除以邻居个数，就是平均位置。当前顶点应该往哪里挪？就是从自己 p_i 指向邻居平均位置 $\bar{p}_i$ 的方向：

$$\delta_i = \bar{p}_i - p_i$$

把 $\bar{p}_i$ 展开，就得到常见写法：

$$\delta_i = \frac{1}{|N(i)|} \sum_{j \in N(i)} (p_j - p_i)$$

这个方向叫 **Laplacian 向量**。这里不用把 Laplacian 想成很抽象的算子，只要记成"邻居平均位置减去自己"，也就是"该往哪挪"。然后顶点朝这个方向挪一小步：

$$p'_i = p_i + \lambda \delta_i \quad (0 < \lambda < 1)$$

这里 λ 是步长，也叫平滑强度。 $\lambda = 0$ 表示完全不动； $\lambda = 1$ 表示直接跳到邻居平均位置；实际常取中间值，比如 0.2、0.5。

【完整算法顺序】

1. 对每个顶点找到一环邻居。
2. 计算邻居平均位置 $\bar{p}_i$ 。
3. 计算 $\delta_i = \bar{p}_i - p_i$ 。
4. 用 $p'_i = p_i + \lambda \delta_i$ 更新位置。
5. 所有顶点最好同步更新，也就是先算完所有新位置，再统一替换旧位置，避免前面更新过的点影响后面还没算的点。

副作用：迭代太多次会**过度收缩**（整个物体缩小、变胖变圆），因为每个点都在往邻居平均位置靠。要减轻收缩，可以减少迭代次数、调小 λ ，或使用带约束 / 保体积的改进平滑方法。

【例题】把一根毛刺磨平：顶点 $p = (0, 0, 2)$ 像根刺竖在邻居上方，它的 4 个邻居都在平面 $z = 0$ 上： $(1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0)$ ，取步长 $\lambda = 0.5$ 。

写思路：先用邻居平均位置减去当前顶点位置，得到 Laplacian 向量 δ ；再沿 δ 方向移动 λ 倍，得到平滑后的顶点。

写公式：Laplacian 平滑先算邻居平均方向，再按步长更新顶点。

$$\delta = \frac{1}{|N(i)|} \sum_{j \in N(i)} (p_j - p_i), \quad p'_i = p_i + \lambda \delta$$

套数据计算：把题目给的四个邻居逐个代入 Laplacian 向量。

$$\delta = \frac{1}{4} \left[(1, 0, -2) + (-1, 0, -2) + (0, 1, -2) + (0, -1, -2) \right] = \frac{1}{4} (0, 0, -8) = (0, 0, -2)$$

$$p' = p + \lambda \delta = (0, 0, 2) + 0.5(0, 0, -2) = (0, 0, 1)$$

这根刺被往邻居平面方向拉下来一半（ z 从 2 降到 1），表面更平了。再迭代会继续逼近 $z = 0$ 。

14.3 网格细分 (Subdivision): Catmull-Clark

14.3.0 为什么要细分，细分做了什么

【先假设你不懂】 建模软件里做一个角色，通常先用几十个面的粗糙“低多边形”把大形勾出来——速度快、容易调整。但最终渲染时，几十个面的棱角太分明，不够光滑。怎么在保持大形的前提下让面变多、形状变圆？

办法：细分 (Subdivision) ——在每个面内插入新点、把一个面切成好几个小面，同时把原来的顶点也稍微挪一挪，让整体更圆润。多做几轮，面越来越多，表面越来越光滑，最终趋近一个光滑曲面（叫**极限曲面**）。

【Catmull-Clark 细分的特点】

- 专门为**四边形网格**设计（也能处理三角形等任意多边形，但最终都会变成四边形网格）。
- 一轮细分后，**每个原来的面都变成若干个四边形**，无论原来是三角形、四边形还是五边形。
- 多轮迭代后收敛到一个 C^2 光滑曲面（在普通顶点处），视觉效果非常平滑。

【一次细分做的四件事（全局视角）】

原网格

- ↓ ① 每个面算一个“面点” F
- ↓ ② 每条边算一个“边点” E
- ↓ ③ 每个旧顶点挪到新位置 V'
- ↓ ④ 重新连边：每个原来的面被切成若干个四边形

新网格（面数更多，更光滑）

前三步是**算出新点的位置**，第四步是**用这些点重新拼出网格**。下面逐步讲清每一步。

14.3.1 第一步：面点 (Face Point)

【先假设你不懂】 每个面中间插入一个新点，用来“撑开”这个面，之后把原来的大面切成小面。这个点放在哪里最合理？——放在这个面的**几何中心**（重心）。

【公式】 若一个面有 m 个顶点 $p_1, p_2, \dots, p_m$ ，面点为：

$$F = \frac{p_1 + p_2 + \dots + p_m}{m}$$

【理解】这就是这 m 个顶点坐标的算术平均。四边形面就除以 4，三角形面就除以 3。

从例子感受一下：一个正方形面，四顶点是 $(0,0,0), (1,0,0), (1,1,0), (0,1,0)$ ，面点 $F = \frac{(0,0,0)+(1,0,0)+(1,1,0)+(0,1,0)}{4} = (0.5, 0.5, 0)$ ，正好是正方形中心。符合直觉。

14.3.2 第二步：边点 (Edge Point)

【先假设你不懂】每条边上也需要插入一个新点 (叫"边点")，用来把这条边拆成两段。边点放在哪里？

直觉：如果只取边的中点，整体太"硬" (不够圆滑)。边点除了要靠近这条边的中间，还应该受这条边两侧两个面的"拉力"——两侧面越"鼓"，边点就稍微往外偏，让弯折处更光滑。所以边点取边的两个端点 + 两侧面点，四者平均。

【公式】设这条边连接顶点 p_k 和 p_l ，边两侧的两个面的面点分别是 f_r 和 f_s ：

$$E = \frac{p_k + p_l + f_r + f_s}{4}$$

【理解每一项的作用】

项	含义	占比
$\frac{p_k + p_l}{2}$ (合并看)	边的几何中点，基础位置	各占 $\frac{1}{4}$
$\frac{f_r + f_s}{2}$ (合并看)	两侧面中心的平均，把边点往面内"拉"	各占 $\frac{1}{4}$

【为什么不只用中点】若只取 $\frac{p_k + p_l}{2}$ ，连续细分时顶点和边点不协调，边界处会出现"折痕"。把面点也纳入平均，边点就"知道"两侧面的形状，保持曲面的光滑连续。

【边界情况】若这条边是网格的边界边 (只有一侧有面，比如洞的边缘)，没有第二个面点 f_s ，通常退化成：

$$E_{\text{边界}} = \frac{p_k + p_l}{2}$$

即边界边的边点只取中点，不受面影响。

例子：边 $p_k = (0,0,0), p_l = (2,0,0)$ ，两侧面点 $f_r = (1,1,0), f_s = (1,-1,0)$ 。

$$E = \frac{(0,0,0)+(2,0,0)+(1,1,0)+(1,-1,0)}{4} = \frac{(4,0,0)}{4} = (1,0,0)$$

边点恰好落在边的中点上，因为两个面点在边的两侧对称，拉力互相抵消。不对称时边点就会偏向某一侧。

14.3.3 第三步：旧顶点更新（Vertex Point）

【先假设你不懂】原来的旧顶点也要**挪动**，挪向"周围邻域的加权平均"，让自己不那么尖锐。但不能挪太多，否则形状就变了。

先把几个概念讲清楚：

价（Valence） n ：一个顶点连了几条边，就叫"价"为 n 。规则的四边形网格内部顶点每个连 4 条边，所以 $n = 4$ 。如果一个顶点周围围了 6 个三角形，那 $n = 6$ 。价决定"这个顶点周围有多大的邻域"，从而决定平滑时的权重分配。

相邻面点的平均 F ：围绕这个旧顶点的所有相邻面，每个面都在第一步算出了一个面点。把这些面点全部加起来再除以面数（= n ，因为每条边对应一个相邻面），得到 F ：

$$F = \frac{1}{n} \sum_{\text{相邻面}} f_i$$

相邻边中点的平均 R ：注意这里不是用第二步算出的"边点 E "，而是用每条相邻边的**普通几何中点**（即旧边两端点的平均）。把所有相邻边中点加起来再除以边数（= n ）：

$$R = \frac{1}{n} \sum_{\text{相邻边}} \frac{p_k + p_l}{2}$$

为什么用旧边中点而不是新边点 E ？ 这是数学推导决定的，保证细分收敛到光滑极限曲面。用 E 会引入面点的成分，导致权重算错。

【公式】旧顶点 P （价为 n ）更新为 V' ：

$$V' = \frac{F + 2R + (n - 3)P}{n} = \frac{1}{n}F + \frac{2}{n}R + \frac{n - 3}{n}P$$

【解读三个权重】

项	权重	含义
P （自身原位置）	$\frac{n - 3}{n}$	保住自己，防止形状变形
R （相邻边中点均值）	$\frac{2}{n}$	被周围邻居"拉"一点，让顶点往边靠
F （相邻面点均值）	$\frac{1}{n}$	被周围面的中心"拉"一点，让顶点往面内靠

三个权重加起来： $\frac{1}{n} + \frac{2}{n} + \frac{n-3}{n} = \frac{n}{n} = 1$ ✓ 这是一个加权平均，顶点不会飞到奇怪的地方。

【特殊情况 $n = 4$ （规则内部顶点）】代入 $n = 4$ ：

$$V' = \frac{1}{4}F + \frac{2}{4}R + \frac{1}{4}P = \frac{F}{4} + \frac{R}{2} + \frac{P}{4}$$

权重 = $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$ ——面邻域和自身各占四分之一，边邻域占一半。

【 $n < 3$ 会怎样？】若 $n = 3$ ，则 $\frac{n-3}{n} = 0$ ， $V' = \frac{1}{3}F + \frac{2}{3}R$ ，自身权重为 0；若 $n = 2$ 则出现负权重，顶点需按边界规则处理（价 < 3 的顶点是尖点或边界顶点，Catmull-Clark 有专门的退化规则）。

符号汇总（容易混淆，务必分清）：

- P = 这个旧顶点自己的原坐标
- n = 这个旧顶点的价（连了几条边 = 周围有几个相邻面）
- f_i = 第 i 个相邻面的面点（第一步算出来的 F ，但这里要加下标区分每个面）
- F = 所有相邻面点的平均 = $\frac{1}{n} \sum f_i$
- R = 所有相邻边的几何中点的平均（不是边点 E ！）
- V' = 旧顶点更新后的新坐标

14.3.4 第四步：重新连边（Connectivity）

【先假设你不懂】算出了三类新点的位置：面点 F 、边点 E 、更新后的旧顶点 V' 。现在要把它们连成新的网格。怎么连？

【每个旧面被切成若干个四边形的规则】

以一个四边形旧面 $\{V_0, V_1, V_2, V_3\}$ 为例（按顺序围一圈），它有：

- 1 个面点 F （第一步算的，在面中间）
- 4 个边点 $E_{01}, E_{12}, E_{23}, E_{30}$ （分别在 4 条边上，第二步算的）
- 4 个更新后的旧顶点 V'_0, V'_1, V'_2, V'_3 （第三步算的）

连法：把面点 F 和四个边点连起来，再把每个边点和两侧的旧顶点连起来，就把原来一个四边形切成了 4 个小四边形：

```

V0' --- E30 --- V3'
|       |       |
E01    F    E23
|       |       |
V1' --- E12 --- V2'

```

四个小四边形分别是：

- $\{V'_0, E_{01}, F, E_{30}\}$
- $\{E_{01}, V'_1, E_{12}, F\}$
- $\{F, E_{12}, V'_2, E_{23}\}$
- $\{E_{30}, F, E_{23}, V'_3\}$

【一般多边形的情况】若原来的面有 m 条边 (m 边形)，则该面被切成 m 个四边形，每个四边形含 F 、两个相邻边点、一个旧顶点。无论原来是三角形（切成 3 个四边形）还是五边形（切成 5 个四边形），结果都是四边形。

重要结论：经过一轮 Catmull-Clark 细分，整张网格的**所有面都变成四边形**。若继续迭代，每轮面数扩大约 4 倍，边和顶点数也大幅增加，形状越来越圆润。

14.3.5 一轮细分的完整流程（考场答题顺序）

1. **算面点：**对每个面，取所有顶点坐标的平均 $F = \frac{\sum p_i}{m}$ 。
2. **算边点：**对每条内部边，取两端点 + 两侧面点的平均 $E = \frac{p_k + p_l + f_r + f_s}{4}$ ；边界边取中点 $E = \frac{p_k + p_l}{2}$ 。
3. **算更新顶点：**对每个旧顶点，确定其价 n ，求相邻面点均值 F 和相邻边中点均值 R ，代入 $V' = \frac{F + 2R + (n-3)P}{n}$ 。
4. **重新连边：**每个旧面内部，用面点和边点把该面切成 m 个四边形 ($m =$ 原面的边数)。

做题常见坑：

- 算 R 时用的是旧边两端点的**中点**，不是第二步算出来的边点 E 。
- 算旧顶点的 F 和 R 时， n 是围绕该旧顶点的**相邻面数 (= 相邻边数)**，不是面的顶点数。
- 三类新点的位置都要**用旧网格的坐标算**，不要用算到一半的"新点位置"去更新另一个点（各点是并行计算，互不依赖）。

14.4 网格简化 (Simplification)

【先假设你不懂】细分是把网格变密，简化正好反过来：把太密的网格变稀。比如扫描模型有 87 万个三角形，显示和传输都很慢，但很多小三角形对整体外形影响不大，就可以删掉一部分，变成 5 万面左右，同时尽量保持形状看起来差不多。

【简化要同时顾两件事】

1. 面数要减少：顶点、边、三角形都变少。
2. 形状不能塌：轮廓、尖锐边、孔洞、重要特征尽量保留。

【方法一：顶点聚类】 把空间划成很多小格子，落在同一个格子里的顶点合并成一个代表点。代表点可以取这些顶点的平均位置，也可以取最接近原形状的位置。然后把原来连到这些顶点的边和面重新连到代表点上。这个方法很快，适合大幅度降面，但细节可能损失明显。

【方法二：边折叠】 选一条边，把它的两个端点合并成一个新点，这条边就消失了，周围相邻的两个三角形通常也会消失。不断挑“折叠后误差最小”的边来折叠，就能逐步减少面数。很多简化算法的核心都是边折叠。

【误差怎么理解】 简化不是随便删点，而是问：删掉 / 合并之后，新网格离原网格有多远？这个“离得远不远”就是误差。误差小，说明这次简化不太影响形状；误差大，说明可能把重要棱角或轮廓弄坏了。

【考试答法】 网格简化的核心目标是减少顶点和面数，同时尽量保持原形状。常见做法是顶点聚类或边折叠：前者快但粗，后者更精细；实际系统会用误差度量决定哪些点 / 边可以优先合并。

14.5 网格正则化 (Regularization)

【先假设你不懂】 有些网格虽然面数不多，但三角形质量很差：有的特别细长，有的特别小，有的特别大。这种网格看起来可能还行，但后续做平滑、细分、物理模拟、数值计算时容易出问题。**正则化**就是调整网格分布，让三角形大小更均匀、形状更接近正三角形。

【为什么细长三角形不好】 细长三角形像一根针，面积很小但边很长。它会让法向、曲率、插值和模拟都变得不稳定：一点点移动就可能造成很大的角度变化。正三角形最均匀，数值性质最好，所以正则化常常希望三角形不要太扁、不要大小差太多。

【常见操作】

- 移动顶点：把顶点往邻居平均位置附近调整，让边长更均匀。这和 Laplacian 平滑有关系，但正则化更关心三角形质量。
- 翻边：两个三角形共用一条边时，可以把对角线换到另一边，让两个三角形更接近正三角形。
- 分裂长边：特别长的边中间插入新点，避免大三角形太大。
- 合并短边：特别短的边折叠掉，避免小三角形过密。

【和去噪、简化的区别】 去噪主要让表面更平；简化主要减少面数；正则化主要让网格元素更均匀。三者经常一起用，但目标不同。

14.6 【例题 5】Catmull-Clark 边点 / 顶点更新

一条内部边端点 $p_k = (0, 0, 0)$, $p_l = (2, 0, 0)$ ，两侧面点 $f_r = (1, 1, 0)$, $f_s = (1, -1, 0)$ ，求边点 E 。

写思路： 内部边点取边两端点和两侧面点的平均，所以先把四个点相加再除以 4。旧顶点更新时，要先确认价 n ，再把原顶点 P 、相邻边中点平均 R 、相邻面点平均 F 按 Catmull-Clark 权重合成。

写公式： Catmull-Clark 的内部边点和旧顶点更新分别使用下面两个平均公式。

$$E = \frac{p_k + p_l + f_r + f_s}{4}, \quad V' = \frac{(n-3)P + 2R + F}{n}$$

套数据计算：先算边点，四个点分别是边的两个端点和左右两个面点。

$$\begin{aligned} E &= \frac{(0, 0, 0) + (2, 0, 0) + (1, 1, 0) + (1, -1, 0)}{4} \\ &= \frac{(4, 0, 0)}{4} = (1, 0, 0) \end{aligned}$$

这个结果正好在原边中点，因为两侧面点关于 x 轴对称，上下拉力抵消。

再设某价 $n = 4$ 的旧顶点 $P = (0, 0, 0)$ ，其相邻面点平均 $F = (1, 1, 0)$ 、相邻边中点平均 $R = (0.5, 0.5, 0)$ ，则

$$V' = \frac{(4 - 3)(0, 0, 0) + 2(0.5, 0.5, 0) + (1, 1, 0)}{4} = \frac{(2, 2, 0)}{4} = (0.5, 0.5, 0)$$

这里 $n = 4$ ，所以原顶点权重是 $\frac{1}{4}$ ，边中点平均权重合起来是 $\frac{2}{4}$ ，面点平均权重是 $\frac{1}{4}$ 。结果 $(0.5, 0.5, 0)$ 表示旧顶点被邻域平均位置拉过去了一些，网格因此变得更圆滑。

第三大题：动画（弹簧系统）

考核重点：质点-弹簧系统（受力分析 + 数值积分）。

15. 动画基础概念

【先假设你不懂】静态建模只关心"这一帧物体长什么样"，动画关心的是"物体随时间怎么变"。所以动画的本质可以理解成一个随时间变化的状态函数：

$$\text{状态} = \text{state}(t)$$

这里的状态可以是物体位置、旋转角度、骨骼姿态、表情权重、布料顶点位置等。计算机每隔一小段时间取一次状态，渲染成一张图，连续播放就形成运动。帧率就是每秒播放多少张图：电影常见 24fps，视频常见 30fps，VR 往往要 90fps 左右才不容易晕。

15.1 关键帧、插值与形状变化

【关键帧动画】最朴素的动画做法不是每一帧都手画，而是只指定几个关键时刻的状态，叫**关键帧**。比如第 0 帧手臂垂下，第 30 帧手臂抬起，中间第 1 到 29 帧由计算机自动补出来，这个"补中间"就叫**插值**。

若某个量从 A 变到 B ，线性插值仍然是前面反复用过的 lerp：

$$X(t) = (1 - t)A + tB, \quad 0 \leq t \leq 1$$

$t = 0$ 时是起点， $t = 1$ 时是终点， $t = 0.5$ 时在正中间。若想让动作先慢后快、先快后慢，就不用线性 t ，而是用缓入缓出的曲线重新控制 t ，这就是非线性插值。

【BlendShape：表情怎么插值】人脸表情常用 BlendShape。先准备一个基础表情 F_0 ，再准备若干个目标表情 $F_1, F_2, \dots$ ，比如笑、皱眉、张嘴。新表情不是重新建模，而是把这些形状按权重混合：

$$F = F_0 + w_1(F_1 - F_0) + w_2(F_2 - F_0) + \dots$$

这里 $w_1, w_2, \dots$ 是权重。 $w_1 = 0$ 表示不笑， $w_1 = 1$ 表示完全变成笑脸目标形状，中间值表示半笑。这个公式的意思很简单：从基础脸出发，沿着各个目标表情的差值方向挪动。

15.2 骨骼动画、蒙皮与 FK / IK

【Rigging 和 Skinning 先分清】角色动画通常不会直接拖每个顶点，而是先给模型装一套骨架。Rigging 是绑定骨架、设置关节层级和控制器；Skinning 是让网格顶点跟着骨骼动，也叫蒙皮。

【LBS：一个顶点跟着多根骨骼动】线性混合蒙皮 LBS 的直觉是：一个顶点可能同时受多根骨骼影响。比如手肘附近的皮肤，一部分跟上臂走，一部分跟前臂走，所以要按权重混合多个骨骼变换：

$$p' = \sum_k w_k M_k p, \quad \sum_k w_k = 1$$

这里 p 是原顶点, p' 是变形后的顶点, M_k 是第 k 根骨骼的变换矩阵, w_k 是这根骨骼对该顶点的影响权重。权重和为 1, 表示这个点的最终位置是多根骨骼作用结果的加权平均。

【动作捕捉】 动作捕捉是把真人动作采下来, 再映射到角色骨架上。常见方式有光学、惯性、机械、磁系统。它解决的是"动作数据从哪里来"; 而 FK / IK 解决的是"有了骨架以后怎么控制关节"。

【FK: 前向运动学】 FK 的输入是每个关节角度, 输出是末端位置。比如先知道肩转多少、肘转多少, 再算手腕在哪里。它的计算方向是从根关节一路乘到末端:

关节角度 $\rightarrow$ 末端位置

矩阵角度看, 就是父骨骼变换乘子骨骼变换, 一节一节累乘。

【IK: 反向运动学】 IK 反过来, 输入是末端目标位置, 输出是关节角度。比如先说"手要抓到杯子", 再反求肩和肘应该转多少:

末端目标位置 $\rightarrow$ 关节角度

IK 通常比 FK 难, 因为同一个末端位置可能有多个姿势能达到, 也可能根本够不到, 所以要解方程或做优化。

【一句话区别】 FK 是"给角度, 算手在哪", 适合摆姿势、机械臂正算; IK 是"给手的位置, 反求角度", 适合脚踩地、手抓物体、角色互动。

15.3 【例题】两段手臂的前向运动学 (FK)

把"已知关节角 $\rightarrow$ 求末端位置"用一个二维两连杆手臂算清楚, 正是变换复合 (第 2、3 节) 在动画里的应用。

题目: 肩关节在原点, 第一段手臂长 $L_1 = 2$ 、相对水平转 $\theta_1 = 90^\circ$; 肘关节接第二段长 $L_2 = 1$ 、相对第一段再转 $\theta_2 = -90^\circ$ 。求手腕 (末端) 位置。

写思路: FK 就是一节一节地"旋转 + 沿局部骨骼方向平移", 本质是矩阵复合。第一段由 $R(\theta_1)T(L_1, 0)$ 给出肘的位置, 第二段在第一段坐标系后继续接 $R(\theta_2)T(L_2, 0)$, 所以末端点要用从右往左的矩阵链计算。

写公式: 二维齐次坐标中, 旋转和平移仍然写成矩阵相乘。

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad T(L, 0) = \begin{pmatrix} 1 & 0 & L \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

用这两个矩阵表达每一段骨骼变换, 肘关节和手腕位置分别为

$$P_{\text{elbow}} = R(\theta_1)T(L_1, 0) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad P_{\text{wrist}} = R(\theta_1)T(L_1, 0)R(\theta_2)T(L_2, 0) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

套数据计算：先看肘位置，它等于肩部加上第一段沿 θ_1 方向的位移。

$$\text{肘} = (0, 0) + L_1(\cos 90^\circ, \sin 90^\circ) = (0, 0) + 2(0, 1) = (0, 2)$$

第二段的绝对方向为 $\theta_1 + \theta_2 = 90^\circ + (-90^\circ) = 0^\circ$ ，指向 $+x$ 。

$$\text{腕} = \text{肘} + L_2(\cos 0^\circ, \sin 0^\circ) = (0, 2) + 1(1, 0) = (1, 2)$$

如果用矩阵乘法看同一件事，完整链路就是

$$P_{\text{wrist}} = R(90^\circ)T(2, 0)R(-90^\circ)T(1, 0) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

所以手腕在 $(1, 2)$ 。**FK 的本质：**末端位置 = 各段变换的累乘（每段“先按累计角度转、再沿自己长度走”），方向就明确给出、直接算；而 IK 反过来——给你腕在 $(1, 2)$ ，反求 θ_1, θ_2 ，往往有多个解、要解方程，更难。

16. 质点-弹簧系统（柔性物体模拟）

【先假设你不懂】 布料、头发、软绳这类东西不是一个刚体。刚体只要整体平移、旋转就能描述；布料每个地方都可能不同运动，所以要把它拆成很多小点来算。每个小点有位置、速度和质量，叫**质点**；相邻质点之间用弹簧连接，弹簧负责把被拉长或压短的地方拉回去。

这一步叫**离散化**：现实里的布料是连续表面，计算机把它近似成有限个质点和有限根弹簧。离散化之后，问题就变成：每一帧算每个质点受力多少，再用这些力更新位置和速度。

16.1 系统里到底存什么

【质点存什么】 每个质点至少要存三样东西：

- 位置 x_i 或 p_i ：这个点现在在哪里。
- 速度 v_i ：这个点正在往哪里动、动多快。
- 质量 m_i ：质量越大，同样的力越不容易把它推快。

有些质点还会被固定住，比如布料上边缘挂在衣架上，这些固定点位置不更新，只作为约束存在。

【弹簧存什么】 每根弹簧连接两个质点 i, j ，至少要存：

- 两端点：知道它连的是哪两个质点。
- 原长 L_0 ：弹簧不拉不压时的自然长度。

- 刚度 k_s : 越大越硬, 拉长一点就产生很大的恢复力。
- 阻尼 k_d : 越大越容易停下来, 不会一直弹来弹去。

【每一帧总流程】 质点-弹簧系统每一帧基本按这个顺序算:

1. 把每个质点的合力先清零。
2. 加外力, 比如重力、风力。
3. 遍历每根弹簧, 计算弹力和阻尼力, 把力分别加到两端质点上。
4. 对每个非固定质点, 用 $a = F/m$ 求加速度。
5. 用数值积分更新速度和位置。
6. 处理固定点、碰撞、地面约束等额外限制。

这条流程先记住, 后面的公式都只是把第 3、4、5 步写细。

16.2 受力分析

设一根弹簧连接质点 i, j , 位置分别是 p_i, p_j 。从 j 指向 i 的弹簧向量写成

$$L = p_i - p_j$$

当前长度是 $\|L\|$, 弹簧方向单位向量是

$$\hat{L} = \frac{L}{\|L\|}$$

注意 $\hat{L}$ 只表示方向, 不表示长度。下面的力通常先写作用在质点 i 上的力; 作用在质点 j 上的力方向相反。

① 弹力 (胡克定律)

【从直觉开始】 拉一根弹簧: 拉得越长, 它越想缩回去; 压得越短, 它越想撑开; 正好等于原长时, 它不想动。这三个情况都由"当前长度和原长的差"决定:

$$\|L\| - L_0$$

这个量叫形变量。它为正, 说明弹簧被拉长; 它为负, 说明弹簧被压短; 它为 0, 说明刚好原长。

把弹力拆成四块看:

1. **形变量**: $\|L\| - L_0$ 。
2. **弹簧硬度**: 乘 k_s , 得到力的大小尺度。
3. **方向**: 沿弹簧方向, 用 $\hat{L}$ 。
4. **恢复力负号**: 弹簧总是反抗形变, 所以前面有负号。

于是作用在质点 i 上的弹力写成:

$$F_{\text{spring},i} = -k_s(\|L\| - L_0)\hat{L}$$

作用在质点 j 上的力相反：

$$F_{\text{spring},j} = -F_{\text{spring},i}$$

【方向怎么验证】 若 p_i 在右边、 p_j 在左边，且弹簧被拉长，则 $\hat{L}$ 指向右， $-k_s(\|L\| - L_0)\hat{L}$ 指向左，所以质点 i 被往回拉；质点 j 受到相反方向，被往右拉。两端互相靠近，符合弹簧直觉。

② 阻尼力

【为什么需要阻尼】 只有弹力的话，弹簧会一直来回振荡。现实中布料不会永远抖下去，因为空气阻力、材料内摩擦会消耗能量。阻尼力就是模拟这种“把振动慢慢消掉”的力。

最简单的阻尼写法是：

$$F_{\text{damp}} = -k_d v$$

它适合把一个质点在空气里运动时的阻力粗略写出来：速度越大，阻力越大，方向和速度相反。

但弹簧连接的是两个质点。真正让这根弹簧变长或变短的，是两端沿弹簧方向的**相对速度**，不是两点一起平移。比如两端点都以同样速度向右飞，弹簧长度不变，就不应该因为这根弹簧产生拉伸阻尼。

所以更常用的弹簧阻尼按三步写：

1. 先取相对速度： $v_i - v_j$ 。
2. 再投影到弹簧方向： $(v_i - v_j) \cdot \hat{L}$ 。
3. 最后乘回方向并加负号：

$$F_{\text{damp},i} = -k_d((v_i - v_j) \cdot \hat{L})\hat{L}$$

同样地，作用在质点 j 上的阻尼力相反：

$$F_{\text{damp},j} = -F_{\text{damp},i}$$

【这个点乘怎么读】 $(v_i - v_j) \cdot \hat{L} > 0$ ，说明两点沿弹簧方向越离越远，阻尼要阻止继续远离；小于 0，说明两点越靠越近，阻尼要阻止继续靠近；等于 0，说明没有沿弹簧方向的相对运动，这根弹簧不给阻尼。

③ 外力与合力

外力就是不来自弹簧本身的力。常见有重力、风力、碰撞力、约束力。重力最常写成：

$$F_g = (0, -m_i g, 0)$$

其中 $+y$ 通常表示向上，所以重力是负 y 方向。

一个质点可能连着好几根弹簧，所以最后要把所有力加起来：

$$F_i = \sum F_{\text{spring}} + \sum F_{\text{damp}} + F_{\text{ext}}$$

求出合力后，用牛顿第二定律得到加速度：

$$a_i = \frac{F_i}{m_i}$$

16.3 布料弹簧的三种结构

【先看为什么不只连一种弹簧】 想象一块方格布。如果只连横竖相邻点，布能抵抗被拉长，但很容易像平行四边形一样被斜着扭歪；如果加对角线，就能抵抗这种斜向扭歪；如果再连接隔一个点的远邻，布就不容易折出很尖的角。

因此布料网络上常放三类弹簧：

结构弹簧 (Structural)：连接水平或竖直相邻质点，主要抵抗拉伸。它保证布料不会轻易被拉长或压短，是最基础的一类。

剪切弹簧 (Shear)：连接方格对角线上的相邻质点，主要抵抗剪切变形。没有它，方格容易变成歪斜的平行四边形。

弯曲弹簧 (Bending / Flexion)：连接隔一个点的质点，主要抵抗过度弯折。没有它，布料会过软，容易折出不自然的尖角。

【考试怎么答】 三类弹簧分别负责三种变形：结构弹簧抗拉伸，剪切弹簧抗斜向扭歪，弯曲弹簧抗折弯。它们一起让布料既能动，又不会像一团没有形状的点乱飞。

16.4 数值积分（时间步进更新）

【先假设你不懂】 现在每个质点的合力 F_i 算出来了，但我们真正想要的是下一帧的位置。物理规律是连续的，计算机只能一帧一帧算，所以要用一个短时间步 Δt 近似推进。这就叫**数值积分**。

从合力到运动分两步：

$$a_i = \frac{F_i}{m_i}$$

先由力得到加速度；再由加速度更新速度和位置。

① 显式欧拉（前向欧拉）

显式欧拉只用当前时刻已经知道的量。先用当前加速度更新速度，再用当前旧速度更新位置：

$$v_i^{t+1} = v_i^t + \Delta t a_i^t$$

$$x_i^{t+1} = x_i^t + \Delta t v_i^t$$

它最容易算，但稳定性差。若 Δt 太大或 k_s 太大，弹簧力变化很猛，每一步迈得太粗，系统可能越算越抖，最后"爆炸"。

② 半隐式欧拉（辛欧拉）

半隐式欧拉只改一步：位置不用旧速度，而用刚更新出来的新速度：

$$v_i^{t+1} = v_i^t + \Delta t a_i^t$$

$$x_i^{t+1} = x_i^t + \Delta t v_i^{t+1}$$

它仍然简单，但通常比显式欧拉稳定，所以实时布料模拟里很常用。

③ 隐式欧拉、Verlet、RK4 怎么记

隐式欧拉用未来时刻的力更新，稳定但要解方程，适合刚度大、步长大的情况。Verlet 常用于粒子系统，不显式依赖速度，用前后位置推进，稳定性较好。RK4 在一步里多算几次斜率取加权平均，精度高，但计算更贵。

【本节例题用哪个】 为了让计算最清楚，例题用显式欧拉。考试遇到"一步欧拉"时，先看题目说的是显式还是半隐式：显式位置用旧速度，半隐式位置用新速度。

16.5 【例题 6】单弹簧单质点一步显式欧拉

这题之前先记住模板：先算弹簧向量和长度，再算弹力和阻尼，合成总力，除以质量得加速度，最后按题目指定的积分方法更新速度和位置。

题目：质点 $m = 1$ ，初始位置 $x = (2, 0, 0)$ 、速度 $v = (0, 0, 0)$ ，由弹簧连接到固定点 $x_0 = (0, 0, 0)$ 。弹簧刚度 $k_s = 10$ ，原长 $L_0 = 1$ ；阻尼 $k_d = 0$ ，忽略重力，时间步长 $\Delta t = 0.1$ 。用**显式欧拉**求一步后的位置和速度。

写思路：固定点不动，只更新质点。先用 $L = x - x_0$ 算弹簧当前方向和长度；再用 Hooke 定律算弹力。因为 $k_d = 0$ 且忽略重力，所以合力就是弹力。然后用 $a = F/m$ 算加速度，最后按显式欧拉更新：速度用加速度更新，位置用旧速度更新。

写公式：单质点连固定点时，弹簧向量、弹力、加速度和显式欧拉公式为

$$L = x - x_0, \quad \hat{L} = \frac{L}{\|L\|}$$

$$F = -k_s(\|L\| - L_0)\hat{L} - k_d(v \cdot \hat{L})\hat{L} + F_{\text{ext}}$$

$$a = \frac{F}{m}, \quad v' = v + \Delta t a, \quad x' = x + \Delta t v$$

这里最后一式的位置更新用的是旧速度 v ，因为题目指定的是显式欧拉。

套数据计算：先算弹簧当前向量和长度。

$$L = x - x_0 = (2, 0, 0) - (0, 0, 0) = (2, 0, 0)$$

$$\|L\| = 2, \quad \hat{L} = \frac{(2, 0, 0)}{2} = (1, 0, 0)$$

当前长度 2，大于原长 1，说明弹簧被拉长了 1 个单位，应该把质点往原点方向拉回去。

阻尼和外力都为 0，所以合力就是弹力：

$$F = -k_s(\|L\| - L_0)\hat{L} = -10(2 - 1)(1, 0, 0) = (-10, 0, 0)$$

加速度为

$$a = \frac{F}{m} = \frac{(-10, 0, 0)}{1} = (-10, 0, 0)$$

显式欧拉先更新速度：

$$v' = v + \Delta t a = (0, 0, 0) + 0.1(-10, 0, 0) = (-1, 0, 0)$$

再用旧速度更新位置：

$$x' = x + \Delta t v = (2, 0, 0) + 0.1(0, 0, 0) = (2, 0, 0)$$

所以一步之后速度变成 $(-1, 0, 0)$ ，说明质点已经开始向原点运动；但位置仍是 $(2, 0, 0)$ ，因为显式欧拉这一小步的位置用旧速度，而旧速度为 0。

若改用半隐式欧拉，位置会用新速度 v' ，所以 $x' = (2, 0, 0) + 0.1(-1, 0, 0) = (1.9, 0, 0)$ 。这就是两种欧拉在同一道题里的差别。

附录 A：高频公式速查表

向量运算是一切几何和光照计算的基础

名称	公式
点乘	$a \cdot b = a_x b_x + a_y b_y + a_z b_z = \ a\ \ b\ \cos \theta$
叉乘（行列式式，好记）	$a \times b = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$
叉乘（展开式）	$(a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$
归一化	$\hat{a} = a / \ a\ $
正交矩阵	$R^{-1} = R^T$ （旋转矩阵都满足）

变换矩阵（齐次坐标，列向量右乘）

名称	矩阵 / 公式
平移 $T(d)$	单位阵右列填 d
绕 Z 旋转 R_z	$\begin{bmatrix} c & -s \\ s & c \end{bmatrix}$ 放左上
绕 X 旋转 R_x	$\begin{bmatrix} c & -s \\ s & c \end{bmatrix}$ 放 (y, z)
绕 Y 旋转 R_y	$\begin{bmatrix} c & s \\ -s & c \end{bmatrix}$ 放 (x, z) （符号相反）
缩放 $S(s)$	对角 $s_x, s_y, s_z, 1$
绕任意中心轴	$T(p_0)R(\theta)T(-p_0)$
逆变换	$T^{-1} = T(-d), R^{-1} = R^T, S^{-1} = S(1/s)$

相机、投影和视口共同决定一个三维点最终落到哪里

名称	关键公式
gluLookAt	$w = \widehat{eye - at}, u = \widehat{up \times w}, v = w \times u$
视图矩阵	上 3 行为 u, v, w ，平移列 $-u \cdot E, -v \cdot E, -w \cdot E$
正交规范化	$S(\frac{2}{r-l}, \frac{2}{t-b}, \frac{-2}{f-n}) \cdot T(\text{中心移原点})$
透视 glFrustum	末行 $(0, 0, -1, 0)$ ，使 $w = -z_e$
gluPerspective	$t = n \tan \frac{fovy}{2}, r = t \cdot aspect$ （对称特例）
视口	$Win = \frac{x+1}{2} \cdot size + offset$ ；行号 $= height - WinY$

光照、几何和动画部分常用公式集中在这里

名称	公式
理想反射	$r = 2(l \cdot n)n - l$
Phong	$I = k_a L_a + k_d L_d \max(l \cdot n, 0) + k_s L_s \max(r \cdot v, 0)^\alpha$
Blinn 半程向量	$h = \widehat{l + v}$, 镜面用 $\max(n \cdot h, 0)^\alpha$
Snell	$\eta_i \sin \theta_i = \eta_t \sin \theta_t$
贝塞尔 de Casteljau	$P_i^{(k)} = (1 - t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}$
Bernstein	$B_{i,n} = C_n^i t^i (1 - t)^{n-i}$
三次贝塞尔	$(1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3$
Catmull-Clark 边点	$E = \frac{1}{4}(p_k + p_l + f_r + f_s)$
Catmull-Clark 顶点	$V' = \frac{(n-3)P + 2R + F}{n}$
Laplacian 平滑	$p'_i = p_i + \lambda \cdot \frac{1}{ N } \sum (p_j - p_i)$
弹力	$F = -k_s(\ L\ - L_0)\hat{L}$
阻尼	简单版 $F = -k_d v$; 弹簧方向版 $F_{\text{damp}} = -k_d((v_i - v_j) \cdot \hat{L})\hat{L}$
显式欧拉	$v' = v + \Delta t \frac{F}{m}, x' = x + \Delta t v$
半隐式欧拉	$v' = v + \Delta t \frac{F}{m}, x' = x + \Delta t v'$

附录 B：易错点清单

1. **变换顺序**： BA 表示先 A 后 B ；最右先作用。矩阵乘法不可交换。
 2. **点 vs 向量**：齐次坐标第 4 维点为 1、向量为 0。平移对向量无效（第 4 维 0）。
 3. R_y 的符号：负号在左下角，和 R_x, R_z 相反。
 4. **gluLookAt 的 w** ：指向视线**反方向** ($eye - at$)，不是 $at - eye$ 。
 5. **up 不垂直于视线**：用叉乘 $up \times w$ 自动正交化即可，不用手动调整。
 6. **正交矩阵 z 行的负号**：相机看 $-z$ ， n, f 取距离绝对值。
 7. **透视必须做透视除法**（除以 $w = -z_e$ ）；正交 $w = 1$ 不用除。
 8. **gluPerspective 是 glFrustum 的对称特例**，只能定义对称视景体。
 9. **视口的 y 翻转**：视口原点在左下、 y 向上；窗口 / 鼠标原点在左上、 y 向下。求"行号"要 $height - WinY$ 。"相对窗口"不加左上角偏移。
 10. **漫反射 / 镜面用 $\max(\cdot, 0)$** ：防止背光出现负光照。
 11. **Gouraud 丢高光**：插颜色；Phong 插法向逐像素算光照才保高光。OpenGL 默认 Gouraud。
 12. **贝塞尔不过中间控制点**，只过首末点；首末切线沿控制多边形首末边。
 13. **Catmull-Clark 顶点权重** ($n - 3$) : 2 : 1 分别对应 旧顶点 P : 边中点平均 R : 面点平均 F ，再除以价 n 。
 14. **显式欧拉不稳定**： k_s 大或 Δt 大会爆炸，需小步长或换隐式 / 半隐式。
 15. **法向要归一化**，且顶点法向 = 相邻面法向的平均。
-

附录 C：知识点覆盖对照表（按三大题核对）

按"三道大题 = 图形学三大方向"逐条核对，确认复习无死角。✅ 表示本文档已覆盖、并标出所在章节。

第一大题·渲染（重点：变换 + 上色）

知识点	章节
✅ 齐次坐标、点 vs 向量	0.4, 0.5
✅ 顶点变换流水线总览	1
✅ 平移 / 旋转 / 缩放矩阵	2.1–2.3
✅ 刚体 vs 仿射、错切、逆变换、复合与顺序	2.4–2.6
✅ 绕任意轴 / 任意中心旋转	3（含完整斜轴例题 §3.5）
✅ 相机变换 gluLookAt (u,v,w + 视图矩阵)	4
✅ 正交规范化矩阵 glOrtho（推导）	5.2
✅ 透视投影 glFrustum / gluPerspective	5.3
✅ 视口变换 glViewport / gluProject	6
✅ 完整流水线综合计算	7
✅ 光源类型 / 表面类型	8.1
✅ Phong 光照模型（环境 / 漫反射 / 镜面）	8.2
✅ Blinn 改进（半程向量）	8.3
✅ 折射 Snell 定律	8.4
✅ 隐藏面消除 / Z-buffer	9.1
✅ 多边形着色 Flat / Gouraud / Phong	9.2
✅ 光线跟踪（Whitted、求交、阴影 / 反射 / 折射）	10

第二大题·几何（重点：贝塞尔 + 网格）

知识点	章节
✅ 几何表示（隐式 / 显式 / 点云 / 体 / SDF / 网格）	11
✅ 贝塞尔曲线：de Casteljau（线性插值）	12.2
✅ 贝塞尔曲线：Bernstein 多项式	12.3
✅ 贝塞尔性质、拼接与连续性	12.1, 12.4

知识点	章节
✓ 贝塞尔曲面	13
✓ 拓扑与几何、流形	14.1
✓ 网格去噪: Laplacian	14.2
✓ 网格细分: Catmull-Clark	14.3
✓ 网格简化、正则化	14.4, 14.5

第三大题·动画（重点：弹簧系统）

知识点	章节
✓ 动画概念、帧率、关键帧插值	15
✓ 关键帧插值、BlendShape	15.1
✓ LBS、Rigging / Skinning、动作捕捉、FK / IK	15.2
✓ 质点-弹簧系统：质点、弹簧、每帧流程	16.1
✓ 质点-弹簧系统：受力分析（弹力 / 阻尼 / 重力）	16.2
✓ 布料三类弹簧（结构 / 剪切 / 弯曲）	16.3
✓ 数值积分（显式 / 半隐式 / 隐式欧拉 / Verlet / RK4）	16.4

带解例题分布（几乎每节末尾都有一道串公式的计算例题）：

- **第一大题**：§2.7 平移+旋转+缩放复合、§3.4 绕 z 平行轴任意中心旋转、§3.5 绕真正斜轴旋转 30°（完整）、§4.1 gluLookAt、§5.4 正交投影+裁剪、§6.1 视口到像素行列、§7 完整流水线综合大题、§8.5 完整光照（Phong+Blinn）、§9.3 Gouraud 颜色插值、§10.4 光线与球面求交。
- **第二大题**：§12.5 贝塞尔（二次+三次）、§13.1 双线性曲面、§14.2 Laplacian 平滑、§14.6 Catmull-Clark 边点/顶点。
- **第三大题**：§15.3 两连杆 FK、§16.5 弹簧系统一步欧拉。

考试 10 道小题即从这三大题的上述知识点中抽取，全部按“先写思路 → 再写公式 → 最后套数据计算”作答，方便拿步骤分、公式分、结果分。